

PERSONAL CYBERSECURITY HOMELAB

Technical Documentation

Consolidated, Corrected, and Professionally Rebuilt

Field	Detail
Project Title	Personal Cybersecurity Homelab
Server Hardware	Dell Inspiron N4030
Workstation	Acer Nitro V15
Base Operating System	Ubuntu Server 22.04.5 LTS
Vulnerable Targets	DVWA · OWASP Juice Shop · Metasploitable 2
Focus	SOC Monitoring · Detection Engineering · Attack Simulation
Classification	Personal Portfolio Documentation

Table of Contents

1. Introduction	13
1.1 What This Project Is	13
1.2 Why This Homelab Was Built.....	13
1.3 What This Documentation Covers.....	14
1.4 What This Documentation Does Not Cover	14
1.5 Key Learning Outcomes.....	15
2. Hardware Specifications	16
2.1 Dell Inspiron N4030 - The Server	16
Hardware Considerations and Constraints	16
2.2 Acer Nitro V15 - The Workstation.....	17
3. System Architecture.....	18
3.1 The Design Philosophy	18
3.2 Full Architecture Diagram.....	19
3.3 Role Definitions	20
4. Operating System Installation	22
4.1 Why Ubuntu Server?	22
4.2 Design Decisions Before Installation.....	22
No Graphical Interface.....	22
Full Disk Dedication to the Homelab.....	22
LVM Disabled	23
OpenSSH Enabled During Installation.....	23
4.3 Prerequisites	23
4.4 Creating the Bootable USB	23
4.5 Booting from the USB.....	24
4.6 Installation Walkthrough	24
Language and Keyboard	25
Installer Type	25
Network Configuration	25
Storage Configuration.....	25
Server Identity Setup	25
OpenSSH Installation	25
Featured Snaps	26
Completing Installation and First Reboot.....	26
4.7 First Login and System Update	26
4.8 Troubleshooting - OS Installation	27

USB Boot Failure on First Attempt.....	27
Confusion Between Ubuntu 22.04 and 24.04	27
Installer Freezing at 'Reading Package Lists'	27
Installer Crash Loop.....	28
'Failed to Unmount /cdrom' on Reboot	28
Boot Hang at Network Initialisation After First Reboot.....	28
5. Initial Server Configuration.....	29
5.1 Static IP Planning	29
5.2 Netplan Configuration.....	29
5.3 SSH Access	30
5.4 Timezone Configuration	31
5.5 Troubleshooting: Initial Server Configuration	32
SSH disconnected after applying Netplan	32
Wrong network interface name.....	32
YAML indentation issue	32
Wrong timezone in Grafana logs	33
6. Docker Platform Setup.....	34
6.1 Why Docker Was Used	34
6.2 Docker Installation.....	34
6.3 Docker Network: labnet.....	36
6.4 Docker Container Lifecycle	37
6.5 Restart Policy Strategy.....	38
6.6 Troubleshooting: Docker	39
Permission denied when running Docker commands.....	39
Container cannot reach another container by name.....	39
Wrong volume mount path.....	39
7. Tailscale Remote Access.....	40
7.1 Why Tailscale Was Chosen	40
7.2 Installing Tailscale on the Dell.....	41
7.3 Installing Tailscale on the Acer.....	41
7.4 Remote Access Model	42
7.5 Tailscale and Metasploitable Routing Note	42
7.6 Troubleshooting: Tailscale	43
Acer cannot ping the Dell Tailscale IP	43
Tailscale works at home but not on another network	43
Tailscale not active after reboot.....	43

8. Firewall and Security Baseline	44
8.1 UFW Purpose	44
8.2 Firewall Design Rules	44
8.3 Firewall Setup with UFW	44
8.4 Security Hygiene	46
8.5 Troubleshooting: UFW	46
SSH locked out after enabling UFW	46
Service not reachable later after adding a firewall rule	46
Rule is too broad	47
9. Logging Stack Overview: Loki, Alloy, Grafana	48
9.1 Why Logging Matters	48
9.2 Why Loki, Alloy, and Grafana	48
Why Not ELK Stack?	49
9.3 Promtail Replacement Note	49
9.4 Logging Directory Structure	50
9.5 Docker Network for the Logging Stack	50
9.6 Loki Configuration	51
Understanding the Key Configuration Options	51
Create the Loki Configuration File	52
Fix Directory Permissions	53
Start the Loki Container	53
Verify Loki is Running	54
9.7 Alloy Configuration	54
Base Alloy Configuration	54
Understanding the Alloy Configuration Structure	54
Create the Alloy Configuration File	55
Start the Alloy Container	56
Verify Alloy is Working	57
9.8 Grafana Configuration	57
Start the Grafana Container	57
Access Grafana from the Acer	58
Add Loki as a Data Source	58
9.9 Grafana Dashboard Persistence	59
Recommended: Volume Mount for Persistence	59
Minimum: Dashboard JSON Export	59
9.10 Troubleshooting: Logging Stack	60

Loki Container Starts but Writes No Data	60
Alloy Cannot Connect to Loki	60
Grafana Cannot Connect to Loki Data Source	60
Logs Appear with Incorrect Timestamps	60
10. Homelab Monitoring Dashboard	62
10.1 Purpose of the Monitoring Dashboard.....	62
10.2 Data Sources.....	62
10.3 Panel Structure.....	64
10.4 Visual Settings.....	66
10.5 What This Dashboard Confirms	67
10.6 Troubleshooting: Monitoring Dashboard	68
11. DVWA Deployment.....	69
11.1 Why DVWA Was Added First.....	69
11.2 Why Docker for DVWA.....	69
11.3 DVWA Container Deployment.....	70
Initial Database Setup.....	71
Setting the Security Level	71
11.4 DVWA Management.....	71
11.5 Troubleshooting: DVWA Deployment.....	72
HTTPS Error When Testing DVWA Locally	72
DVWA Shows Database Setup Page on First Load	72
No DVWA Logs Appearing in Grafana	72
12. DVWA Detection Dashboard	74
12.1 Dashboard Purpose	74
12.2 Data Source	74
12.3 Design Philosophy.....	74
12.4 Dashboard Variable.....	75
12.5 Row 1: Overview Panels	75
Panel: Total Requests per Minute	75
Panel: Request Rate Over Time.....	75
Panel: Top Attacker IPs	76
Panel: DVWA Module Activity Indicator	76
12.6 Row 2: Brute Force Detection	77
Panel: Login Attempts Over Time.....	77
Panel: Login Attempts by IP	77
Panel: Login Attempt Evidence.....	78

12.7 Row 3: SQL Injection Detection	78
Panel: SQL Injection Attempts Over Time	78
Panel: SQLi by IP	79
Panel: SQLi Raw Evidence.....	79
12.8 Row 4: XSS Detection.....	79
The URL Encoding Problem	79
Panel: XSS Attempts Over Time	80
Panel: XSS by IP	80
Panel: XSS Raw Evidence	80
12.9 Row 5: Behaviour Panels	81
Panel: Request Burst.....	81
Panel: Sensitive Endpoint Activity	81
Panel: Host Authentication Activity	81
Panel: DVWA Raw Log Stream	82
12.10 Complete Panel Reference	82
12.11 Testing and Observations	83
Brute Force Testing	83
SQL Injection Testing	83
XSS Testing and the URL Encoding Discovery.....	84
12.12 Troubleshooting: DVWA Detection Dashboard	85
XSS Panels Return No Data Despite Active Testing.....	85
SQLi Panels Show Low-Level Matches on Idle System.....	85
Attack Score Panel Not Updating	85
13. OWASP Juice Shop Deployment.....	86
13.1 Why Juice Shop Was Added	86
13.2 Difference Between DVWA and Juice Shop	87
13.3 Juice Shop Container Deployment.....	87
13.4 Important Port Decision.....	88
13.5 Troubleshooting: Juice Shop Deployment.....	88
Juice Shop Not Loading on First Start	88
Juice Shop Container Not Discovered by Alloy	89
14. Caddy Reverse Proxy for Juice Shop	89
14.1 The Logging Problem	89
14.2 Why Caddy Was Added	89
14.3 Caddy Directory Structure	90
14.4 Final Caddyfile.....	91

14.5	Why JSON Format Was Required	91
14.6	Start Caddy Container	92
14.7	Clear Old Console-Format Logs	93
14.8	Verify Caddy Access Logs	93
14.9	Limitation: Caddy Does Not Capture Full Request Bodies	94
14.10	Troubleshooting: Caddy	94
	JSONParserErr When Querying Juice Shop Logs	94
	Caddy Cannot Reach Juice Shop - 502 Bad Gateway	94
	No Log Entries Appearing After Visiting Juice Shop Through Caddy	95
15.	Juice Shop Logs to Alloy and Loki	96
15.1	Why File-Based Collection	96
15.2	Alloy Caddy Log Source	96
15.3	Recreate Alloy With Caddy Mount	97
15.4	Verify Pipeline	98
	Step 1: Verify Alloy Started Without Errors	98
	Step 2: Generate Test Traffic Through Caddy	98
	Step 3: Verify Logs Appear in Grafana	98
	Step 4: Verify JSON Parsing Works	98
	Confirm All Running Containers	99
15.5	Troubleshooting: Juice Shop Log Pipeline	99
	Alloy Shows No Such File or Directory for caddy-logs	99
	Alloy Config Fails to Start with 'missing comma in field list'	99
16.	Juice Shop Detection Dashboard	100
16.1	Dashboard Purpose	100
16.2	How It Differs From the DVWA Dashboard	100
16.3	Performance Note	100
16.4	Data Source and Base Query	101
16.5	Row 1: Overview	101
	Panel: Total Requests	101
	Panel: Request Rate Over Time	101
	Panel: Top Client IPs	101
	Panel: Attack Score	102
16.6	Row 2: Login Abuse and Brute Force	102
	Panel: Login Attempts Over Time	102
	Panel: Login Attempts by IP	103
	Panel: Login Evidence	103

16.7 Row 3: Injection Detection.....	103
SQL Injection Detection.....	104
XSS Detection	104
Combined Injection Evidence Panel	105
16.8 Row 4: API Abuse and Behaviour	105
Panel: API Request Burst	106
Panel: Sensitive Endpoint Access	106
Panel: HTTP Status Code Breakdown	106
16.9 Row 5: Investigation.....	107
Panel: Full Raw Log Stream	107
Panel: Parsed Clean Logs	107
Panel: Suspicious Activity Logs	107
16.10 Complete Panel Reference	108
16.11 Key Lessons.....	109
Applications Do Not Always Produce the Logs You Need.....	109
Log Format Determines What Detection Is Possible	109
Truncate Before Querying After a Format Change	109
Different Applications Require Different Detection Logic.....	109
Hardware Limits Must Shape Query Design.....	109
16.12 Troubleshooting: Juice Shop Detection Dashboard	110
'Too Many Outstanding Requests' Error in Grafana	110
json Queries Return No Fields Despite Logs Appearing in Raw View	110
request_remote_ip Shows Unexpected IP Values	110
17. Metasploitable 2 Design Decision and Virtualization Choice	111
17.1 Why Metasploitable 2 Was Added	111
17.2 Why Metasploitable 2 Instead of Metasploitable 3	111
17.3 Why VirtualBox Was Selected.....	112
17.4 What Not to Include as Installed.....	112
17.5 Troubleshooting and Design Notes	112
18. Metasploitable 2 Deployment.....	113
18.1 Install VirtualBox.....	113
18.2 Download and Extract Image	113
18.3 Create VM	113
18.4 Attach VMDK.....	114
18.5 Host-Only Network Setup.....	114
18.6 Start VM Headless	115

18.7 Connectivity Validation	115
18.8 SSH Compatibility Fix.....	115
18.9 Terminal Compatibility Fix	116
18.10 Troubleshooting: Metasploitable Deployment	116
SSH Connection Rejected Due to Deprecated Algorithms	116
VM Not Reachable After Start	116
19. Metasploitable Logging Architecture	117
19.1 Why Docker Log Collection Does Not Apply	117
19.2 Final Working Pipeline.....	117
19.3 Why Direct Alloy Syslog Was Rejected	117
19.4 Troubleshooting and Design Notes	118
20. Configure Metasploitable Syslog Forwarding	119
20.1 Identify Logging Service	119
20.2 Add Forwarding Rule.....	119
20.3 Restart syslogd.....	119
20.4 Generate Test Log	120
20.5 Troubleshooting: Metasploitable Syslog Forwarding.....	120
Forwarding Rule Has No Effect	120
syslogd Restart Fails.....	120
21. Configure Dell rsyslog Receiver.....	121
21.1 Install rsyslog.....	121
21.2 Create Dedicated Log File.....	121
21.3 Final rsyslog Receiver Config.....	121
21.4 Restart rsyslog	122
21.5 Verify Reception	123
21.6 UFW Rule for Metasploitable Syslog.....	123
21.7 Troubleshooting: rsyslog Receiver	123
UDP Port 514 Already in Use	123
rsyslog Writes Nothing to the Log File	124
Permission Denied Error in rsyslog Log	124
RFC 3164 Format Causing Alloy Parse Errors.....	124
22. Alloy Collection for Metasploitable Logs	125
22.1 Why File Tailing Was Used	125
22.2 Final Alloy Block	125
22.3 Recreate Alloy	125
22.4 Verify in Grafana	126

22.5 Troubleshooting: Alloy Metasploitable Collection	126
No Results for {job="metasploitable-syslog"} in Grafana	126
23. Metasploitable Detection Dashboard	128
23.1 Dashboard Purpose	128
23.2 Design Philosophy	128
23.3 Data Source	129
23.4 Row 1: Overview	129
Panel: Total Events	129
Panel: Event Rate Over Time	129
Panel: Suspicious Activity Score	129
23.5 Row 2: SSH and Authentication	130
Panel: Failed Login Attempts	130
Panel: Root Login Attempts	130
Panel: SSH Evidence Logs	131
23.6 Row 3: FTP and Telnet	131
Panel: FTP Activity	131
Panel: Telnet Activity	132
Panel: FTP and Telnet Evidence	132
23.7 Row 4: Reconnaissance	132
Panel: Service Connection Activity	132
Panel: Authentication Burst Detection (30s)	133
Panel: Reconnaissance Evidence	133
23.8 Row 5: Investigation	134
Panel: Suspicious Logs	134
Panel: Full Raw Logs	134
23.9 Complete Panel Reference	135
23.10 Attack Simulation and Validation	136
SSH Brute Force Simulation	136
Reconnaissance Scanning	136
Manual Log Injection for Pipeline Testing	136
23.11 Detection Limitation	137
23.12 What This Phase Adds to the Homelab	137
23.13 Troubleshooting: Metasploitable Dashboard	138
No Data for {job="metasploitable-syslog"}	138
Dashboard Shows Events but Panels Are Not Detecting Attack Activity	138
24. Multi-Source Investigation Workflow	139

24.1 Why Separate Dashboards Exist	139
24.2 Correlation by Timestamp and Source IP	139
24.3 Example Attack Storyline	141
Stage 1: Reconnaissance	141
Stage 2: Service Attack	141
Stage 3: Web Application Attack	141
Stage 4: API Abuse	141
24.4 Evidence Handling	141
24.5 Troubleshooting and Workflow Notes	142
Cross-Dashboard Timestamps Do Not Align	142
Grafana Time Range Does Not Show Expected Activity	142
25. Daily Operation Runbook.....	143
25.1 Start Core Stack	143
25.2 Start Vulnerable Targets	143
25.3 Verify Dashboards Before Testing	144
25.4 Stop Lab Safely	144
25.5 After Reboot Checklist.....	145
25.6 Troubleshooting: Operations	145
Core Container Does Not Start After Reboot.....	145
Grafana Shows No Data After Reboot.....	145
26. Log Rotation and Storage Management	147
26.1 Why Log Rotation Matters	147
26.2 Files Needing Rotation	147
26.3 Suggested logrotate Plan	147
27. Backup and Persistence	149
27.1 What Must Be Backed Up	149
27.2 Grafana Dashboard Export	149
27.3 Container Persistence	150
27.4 Troubleshooting: Backup and Persistence	150
Grafana Dashboards Lost After Container Rebuild	150
28. Lab Security Rules.....	151
28.1 Keep Vulnerable Targets Private	151
28.2 Start Vulnerable Targets Only When Needed	151
28.3 Lab Boundary	151
28.4 Tailscale Access Scope	151
28.5 Keep the Host System Updated	152

28.6 Troubleshooting and Safety Notes	152
UFW Rules Missing After Reboot	152
29. Detection Limitations.....	152
29.1 DVWA Limitations	152
29.2 Juice Shop Limitations	153
29.3 Metasploitable Limitations	153
29.4 Platform Limitations	153
29.5 What Would Improve Detection Later	153
30. Current Completed State	155
30.1 Completed Components.....	155
30.2 Final Service Table.....	156
31. Future Work and Roadmap.....	157
31.1 Short-Term Improvements	157
31.2 Security and Monitoring Improvements	157
31.3 Home Server Improvements	157
31.4 Advanced Security Improvements.....	158
31.5 Items Not Currently Planned	158

Part 1: Project Overview and Design

1. Introduction

1.1 What This Project Is

This documentation covers the complete design, build, and ongoing operation of a personal cybersecurity homelab running on repurposed consumer hardware. The server is a Dell Inspiron N4030 that has been converted into a permanently running headless Linux server meaning it operates without a monitor, keyboard, or mouse attached, and is managed entirely over the network via SSH from the Acer workstation.

The homelab exists at the intersection of two disciplines: offensive security, which focuses on understanding how attacks are executed, and defensive security, which focuses on detecting, investigating, and understanding those same attacks through their log evidence. Rather than studying these disciplines in isolation through courses or documentation, this lab creates a controlled private environment where both sides can be practised simultaneously on real infrastructure.

Beyond the security focus, the Dell server also serves as a platform for broader home-server learning. Services like Pi-hole for DNS filtering, Samba for file sharing, and Jellyfin for media streaming are planned for later phases each adding systems administration experience that complements the security curriculum.

1.2 Why This Homelab Was Built

There is a meaningful difference between understanding a concept conceptually and being able to demonstrate it on real infrastructure. Certifications and courses provide theoretical foundations, but practical experience on real systems the kind that produces the intuition to troubleshoot an unexpected error, design an architecture around real hardware constraints, or read a raw log line and understand what it means comes only from building and operating things yourself.

This homelab was built specifically to develop the following capabilities through direct practice:

- Attack-to-log correlation: running a real SQL injection attack against a running web application and then examining precisely what that attack looks like in the log record
- SOC workflow practice: investigating alerts, correlating events across multiple log sources, and reconstructing attack sequences from raw evidence
- Detection engineering: writing LogQL queries that identify specific attack patterns in structured and unstructured log data
- Linux server administration: managing services, users, firewall rules, network interfaces, and system configuration on a real operating system
- Docker container management: deploying, networking, and maintaining containerised services in a multi-service environment
- Networking practice: configuring VPNs, host-only networks, port mappings, and inter-container communication
- Portfolio value: producing documented, reproducible technical work that demonstrates real engineering capability

1.3 What This Documentation Covers

This document covers the complete homelab from initial hardware to final detection dashboard. The following areas are documented in full, including rationale, commands, configuration files, and troubleshooting:

- Ubuntu Server 22.04.5 LTS installation and post-installation configuration
- Docker platform setup and the labnet shared container network
- Tailscale remote access mesh
- UFW firewall configuration with service-specific and interface-specific rules
- Grafana Loki, Grafana Alloy, and Grafana deployment and configuration
- Homelab Monitoring Dashboard for pipeline health verification
- DVWA deployment and the DVWA Detection Dashboard
- OWASP Juice Shop deployment and Caddy reverse proxy for JSON access logging
- Juice Shop Detection Dashboard
- VirtualBox installation and Metasploitable 2 VM setup
- rsyslog receiver configuration and the Metasploitable logging pipeline
- Metasploitable Detection Dashboard
- Multi-source investigation workflow across all three detection dashboards
- Operational runbook, log rotation planning, backup and persistence guidance
- Future work and roadmap

1.4 What This Documentation Does Not Cover

The following are explicitly outside the scope of this homelab in its current phase. They are not installed and should not be understood as part of the current architecture:

- Production-grade SIEM platforms — Wazuh, Splunk, and the ELK stack are not installed. They were evaluated and rejected due to hardware constraints.
- Public internet exposure — no services in this lab are reachable from the public internet
- Enterprise alerting workflows — Grafana alerting is a planned future item, not currently configured
- Full packet inspection — Suricata and Zeek are not installed; this is a planned future option
- Malware analysis
- Active Directory attack lab
- Windows target systems — these may be added later if hardware permits

1.5 Key Learning Outcomes

By building and operating this homelab through its current completed state, the following skills have been directly practised and can be demonstrated:

- Linux server administration on Ubuntu 22.04 LTS including networking, services, and system management
- Docker container lifecycle management pulling, running, networking, persisting, and troubleshooting containers
- Log pipeline design collecting, forwarding, storing, and querying logs across multiple sources and formats
- LogQL query construction for detection writing queries that identify brute force, SQL injection, XSS, API abuse, and service-level attack patterns
- SOC-style dashboard design building panels that separate monitoring from detection, support IP attribution, and provide evidence panels for manual triage
- Attack simulation generating controlled attack traffic and correlating it with its log evidence
- Evidence-based troubleshooting identifying root causes from log output, error messages, and observable system behaviour
- Understanding the differences between web application logs, structured JSON access logs, and legacy syslog format and why each requires different detection logic

2. Hardware Specifications

Two physical machines form the foundation of this homelab. Understanding their specifications matters because every architectural and software decision made throughout this project is shaped by their capabilities and limitations.

2.1 Dell Inspiron N4030 - The Server

This is the homelab server. It is a repurposed consumer laptop that was no longer being used as a daily driver. Rather than discarding it, it has been given a second life as a permanently running headless server. The word 'headless' means it runs without a monitor, keyboard, or mouse attached - all management is done remotely over the network via SSH from the Acer.

Component	Specification
Model	Dell Inspiron N4030
CPU	Intel Core i3 (1st generation, dual-core)
RAM	16 GB DDR3
Storage	1 TB HDD (5400 RPM SATA)
Network	1 Gb/s Ethernet (RJ-45) + Wi-Fi (not used for server)
Display	14-inch (not used - headless operation)
Power	Always plugged in, lid closed
Boot Mode	Legacy BIOS (not UEFI)
Operating System	Ubuntu Server 22.04.5 LTS

Hardware Considerations and Constraints

Understanding the limitations of this hardware is important because they directly influenced every architectural decision made in this project. The CPU is an older first-generation Intel Core i3, which means CPU-intensive tasks - such as video transcoding, running multiple virtual machines simultaneously, or operating heavy enterprise log stacks - are not feasible. The HDD is also a mechanical hard drive rather than an SSD, which means disk read/write operations are significantly slower than modern systems. This is relevant for anything that involves frequent disk access, such as log writing or database queries.

The 16 GB of RAM is the strongest asset this machine has. It is more than sufficient to run multiple Docker containers simultaneously, and it provides enough headroom to run one lightweight virtual machine without starving the core services of memory.

i Note

These hardware constraints are not obstacles - they are the reason the architecture was deliberately designed to be lightweight and efficient. Docker containers were chosen over virtual machines precisely because they consume far less RAM and CPU overhead. Loki was chosen over Elasticsearch because Loki is vastly more resource-efficient while still providing meaningful log aggregation. Every tool selection in this project reflects a conscious trade-off tuned to this hardware.

2.2 Acer Nitro V15 - The Workstation

This is the main laptop used for all administrative, analytical, and offensive operations. It is a significantly more powerful modern machine. It hosts the Kali Linux virtual machine that will be used as the attacker system, and it is also the machine from which the Grafana dashboard is accessed for log analysis.

Component	Specification
Model	Acer Nitro V15
CPU	AMD Ryzen 5 / Intel Core (modern generation)
RAM	16 GB+ (sufficient for VirtualBox Kali VM)
Storage	SSD - fast I/O for VM and daily use
Role in Homelab	Attacker, SOC Analyst, Admin Workstation
OS (Host)	Windows (with VirtualBox installed)
OS (VM)	Kali Linux 2024 (in VirtualBox, Bridged mode)

3. System Architecture

3.1 The Design Philosophy

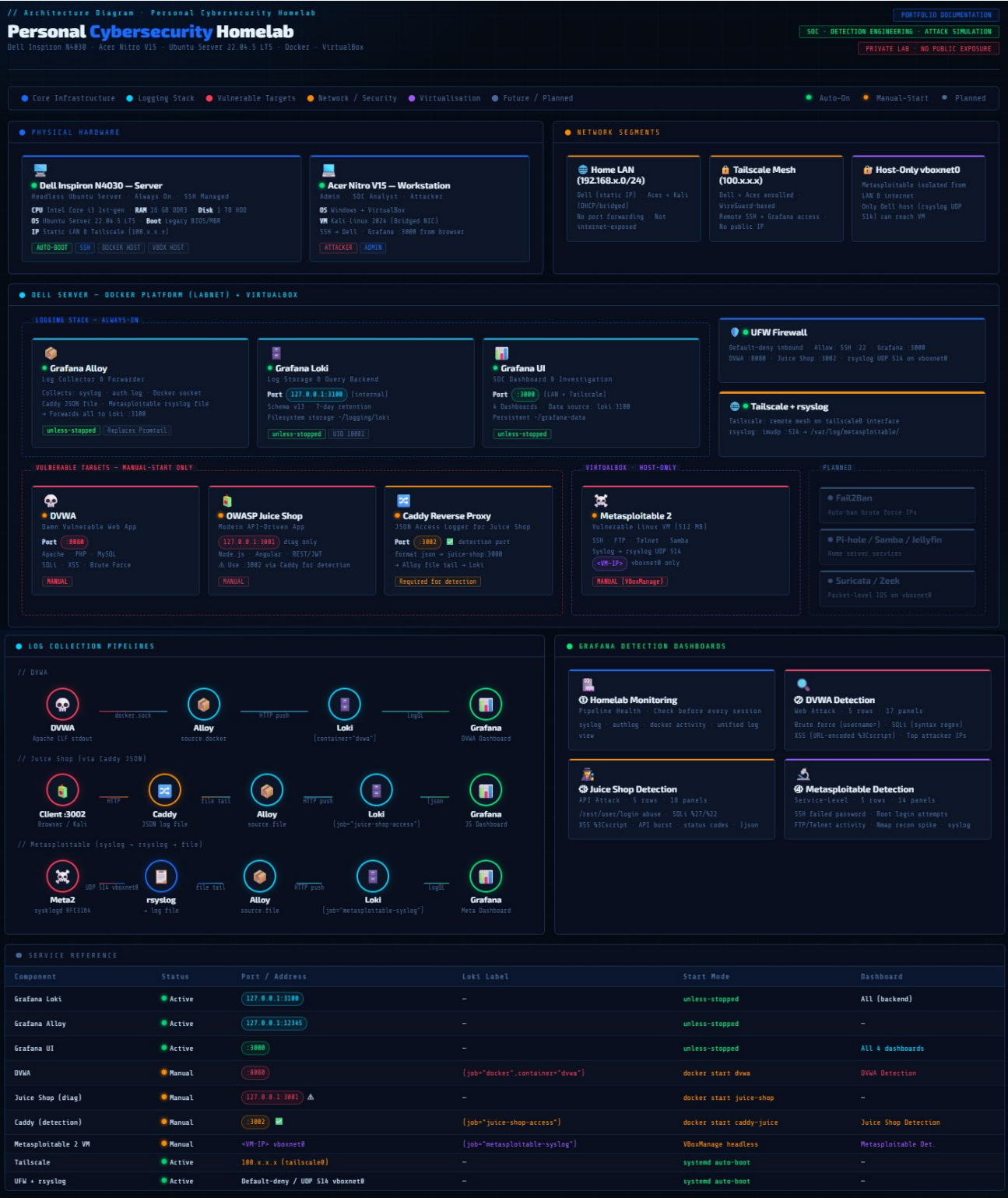
The architecture of this homelab is built around three principles: role separation, explicit network isolation, and deliberate start policies for vulnerable targets.

Role separation means that no single machine serves as both attacker and victim simultaneously. The Dell server hosts all services and targets. The Acer workstation handles administration and analysis. The Kali Linux VM (on the Acer) handles attack simulation. Grafana, Loki, and Alloy run on the Dell server because this is a single-server homelab. The Grafana web interface is accessed from the Acer browser, which separates the analyst workstation from the target interaction workflow. This is not full monitoring-host separation, but it is an acceptable homelab compromise and should be understood as one. This separation mirrors the physical separation that exists in professional SOC environments and makes the learning meaningful like the analyst role, the attacker role, and the target role are kept cleanly distinct.

Network isolation means that vulnerable targets operate on network segments that limit their exposure. DVWA and Juice Shop are Docker containers on the labnet network, accessible only on specific ports via explicit port mappings. Metasploitable 2 operates on a VirtualBox host-only network, shown in this document as <YOUR-HOST-ONLY-NETWORK>. The Dell host-only adapter is shown as <YOUR-HOST-ONLY-ADAPTER-IP>, and the Metasploitable VM is shown as <YOUR-VM-IP>.

Deliberate start policies mean that vulnerable services are not configured to auto-start on boot. DVWA, Juice Shop, Caddy, and Metasploitable are started manually before a lab session and stopped when work is complete. This minimises attack surface outside of active practice sessions.

3.2 Full Architecture Diagram



3.3 Role Definitions

The following table summarises the clearly defined role that each machine and each component plays within the homelab. This is worth understanding before diving into any setup phase, because it explains why things are placed where they are.

Component	Status	Role	Reason for This Placement
Dell Inspiron N4030	Current	Homelab server and monitored environment	Hosts Ubuntu Server, Docker services, vulnerable targets, logging components, rsyslog, and VirtualBox. This is the main system being monitored and documented.
Acer Nitro V15	Current	Workstation and analyst machine	Used for SSH administration, Grafana access, documentation work, and controlled attack simulation through a Kali Linux VM.
Kali Linux VM	Current	Attacker workstation	Used to run security tools for authorised testing against lab targets. Keeping Kali separate from the server creates a more realistic attacker-to-target setup.
Docker	Current	Container platform	Runs DVWA, Juice Shop, Caddy, Grafana, Loki, and Alloy in isolated containers without installing all services directly on the host OS.
Docker network: labnet	Current	Shared container network	Allows containers to communicate by name, such as Grafana reaching Loki and Alloy forwarding logs to Loki.
DVWA	Current	Vulnerable web application target	Provides a simple web target for practising brute force, SQL injection, XSS, and basic web attack detection.
OWASP Juice Shop	Current	Modern vulnerable web application target	Provides a more realistic API-driven web target with modern application behaviour, login activity, API routes, and injection opportunities.
Caddy	Current	Reverse proxy and access log generator for Juice Shop	Routes traffic to Juice Shop through port 3002 and creates structured JSON access logs for detection. Direct Juice Shop access on port 3001 is diagnostic only and bypasses Caddy logging.
Grafana Alloy	Current	Log collection agent	Collects host logs, Docker logs, Caddy access logs, and Metasploitable log files, then forwards them to Loki. Alloy replaces Promtail in the final setup.
Grafana Loki	Current	Log storage and query backend	Stores logs collected by Alloy and makes them searchable through LogQL. Loki was chosen because it is lighter than ELK, Splunk, or Wazuh for this hardware.

Grafana	Current	SOC-style dashboard and investigation interface	Visualises logs from Loki through dashboards. It is accessed from the Acer browser and acts as the main analyst console.
rsyslog	Current	Syslog receiver for Metasploitable logs	Receives syslog messages from Metasploitable over UDP 514 and writes them to a dedicated log file. Alloy then tails that file into Loki.
VirtualBox	Current	Virtualization platform	Runs Metasploitable 2 as a lightweight VM using a host-only network. VirtualBox is used instead of KVM or Proxmox in the current setup.
Metasploitable 2	Current	Vulnerable Linux VM target	Provides service-level attack practice, including SSH, FTP, Telnet, and other vulnerable services. It adds OS and network service visibility beyond web application testing.
Tailscale	Current	Secure remote access mesh	Provides encrypted remote access to the Dell without router port forwarding or public internet exposure.
UFW	Current	Host firewall	Controls inbound access to the server and restricts services to trusted networks or specific lab interfaces where possible.
Fail2Ban	Future Work	Automated brute force protection	Planned for later hardening and response testing. It is not part of the current active setup.
Pi-hole	Future Work	DNS filtering service	Planned for future network-wide DNS filtering and home server functionality.
Unbound	Future Work	Recursive DNS resolver	May be paired with Pi-hole later to provide local recursive DNS resolution. It is not part of the current setup.
Samba	Future Work	Local file sharing	Planned for future home server functionality, allowing shared folders to be accessed from the Acer and other trusted LAN devices.
Jellyfin	Future Work	Media server	Planned for local media streaming. If added, it should use direct play only because the Dell CPU is not suitable for heavy transcoding.

Part 2: Base Server Build

4. Operating System Installation

This section covers the complete installation of Ubuntu Server 22.04 LTS on the Dell Inspiron N4030. The instructions here go beyond simply listing commands, they explain what each decision means and why it was made, so that the choices made during installation are understood rather than simply followed.

4.1 Why Ubuntu Server?

Ubuntu Server is a Linux distribution produced by Canonical. It is designed for server use rather than desktop use, which means it comes without a graphical interface, without desktop applications, and without the associated performance overhead. For a machine that will run 24 hours a day as a headless server managed exclusively over SSH, this is exactly what you want.

The LTS (Long Term Support) designation is also important. LTS releases of Ubuntu are supported with security updates and fixes for five years. Ubuntu 22.04 LTS is a long-term support release. Ubuntu's official release documentation lists standard support for Ubuntu 22.04 LTS until June 2027, with extended support options available beyond that through Ubuntu Pro.

Ubuntu Server 22.04 was chosen over 24.04 specifically because 22.04 has had more time for its packages to mature and for community documentation to accumulate. On older hardware, this translates to a more stable and predictable experience.

4.2 Design Decisions Before Installation

Several important decisions were made before the installation began. Understanding these decisions will help you understand the layout of the installed system.

No Graphical Interface

The server edition of Ubuntu installs with no desktop environment whatsoever. This is intentional. A graphical interface would consume roughly 500 MB to 1 GB of RAM permanently just to render a display that will never be used, and it would require a monitor to be connected. The server will be managed entirely through SSH from the Acer, which means a graphical interface has no value here.

Full Disk Dedication to the Homelab

The entire 1 TB drive was dedicated to Ubuntu Server. No dual-boot setup was created. The rationale is straightforward, this machine is a dedicated server. There is no scenario where you

would want to boot it into another operating system. Allocating the full disk to the server installation means all 1 TB is available for container images, logs, media files, and other homelab content.

LVM Disabled

LVM stands for Logical Volume Manager. It is a system that allows you to manage disk partitions more flexibly - for example, resizing volumes without rebooting, or creating logical volumes that span multiple physical disks. While LVM is a useful enterprise tool, it adds a layer of abstraction that makes the disk layout harder to reason about for someone building their first server. Since there is only one disk and no immediate need to resize partitions dynamically, LVM was disabled in favour of a simple flat partition layout.

OpenSSH Enabled During Installation

SSH (Secure Shell) is the protocol used to connect to and manage remote Linux servers. Enabling the OpenSSH server during installation means the server is accessible over the network as soon as it finishes installing and reboots - no additional configuration required. Without this, you would need a physical keyboard and monitor connected to the Dell just to log in and enable SSH, which defeats the purpose of headless operation.

4.3 Prerequisites

Before starting the installation, you will need the following:

- The Dell Inspiron N4030 laptop, plugged into power
- A USB flash drive (minimum 4 GB capacity)
- The Acer Nitro V15 (or any other computer) to download the ISO and create the installer
- The Ubuntu Server 22.04 LTS ISO file, downloaded from ubuntu.com
- Rufus, a free USB bootable media creator for Windows (rufus.ie)
- An Ethernet cable to connect the Dell to your home router during installation

i Note

Download Ubuntu Server 22.04 LTS specifically - not the desktop edition, and not 24.04. The server edition has no graphical installer; it uses a text-based interface. This is normal and expected.

4.4 Creating the Bootable USB

The installer USB must be created with specific settings to ensure it works correctly with the Dell's legacy BIOS. This step was the source of the first major problem encountered during this project - using the wrong settings caused the Dell to fail to boot from the USB entirely.

Open Rufus on the Acer and configure it as follows:

Rufus Setting	Value to Use
Device	Your USB flash drive
Boot selection	Select the Ubuntu Server 22.04 ISO file
Partition scheme	MBR (not GPT)
Target system	BIOS or UEFI-CSM
File system	FAT32
Write mode prompt	Accept the recommended ISO image mode when prompted
GRUB download	Allow Rufus to download the GRUB files when it asks

Important

Why MBR and not GPT? The Dell N4030 uses legacy BIOS rather than UEFI. GPT partition schemes are primarily used with UEFI firmware. Using GPT on a legacy BIOS machine will cause the boot process to fail silently, which is exactly what happened on the first attempt. MBR is the partition scheme designed for legacy BIOS compatibility.

4.5 Booting from the USB

To boot from the USB on the Dell Inspiron N4030:

- Power on the Dell laptop
- Immediately and repeatedly press F12 as the Dell logo appears - this opens the one-time boot menu
- Select the USB drive from the boot menu
- When the Ubuntu boot menu appears, select the default option: 'Try or Install Ubuntu Server'

Tip

If the machine does not detect the USB, ensure it is inserted in a USB 2.0 port. Some older machines have trouble booting from USB 3.0 ports. The Dell N4030 has both types - try a different port if the first attempt fails.

4.6 Installation Walkthrough

The Ubuntu Server installer is text-based and navigated using the arrow keys and Enter. It is straightforward once you know what to expect. The following walks through each screen in order.

Language and Keyboard

Select English for both language and keyboard layout. Accept the defaults and continue.

Installer Type

When prompted to choose the base installer type, select the standard Ubuntu Server option. There is also a 'minimized' option that strips down the install further, but the standard option provides a more practical starting point for a homelab.

Network Configuration

The installer will detect your network interface and attempt to assign an IP address via DHCP from your home router. If you have an Ethernet cable connected, you should see an assigned IP address on this screen. Note it down - you will use it to SSH into the machine before a static IP is configured.

Important

Network connectivity during installation is not required to complete the base install. If the mirror step hangs or causes the installer to freeze (as it did during this project), you can safely skip past it. All packages can be updated after installation using 'sudo apt update && sudo apt upgrade'.

Storage Configuration

On the storage screen, select 'Use an entire disk'. Select the 1 TB internal HDD. When the LVM option appears, deselect it - you want a simple partition layout without LVM. Confirm the partition layout and accept the warning that the disk will be formatted.

Server Identity Setup

You will be asked to configure the server identity. Use the following values (or your own preferred equivalents):

- Your name: enter your name or any label
- Server name: homelab (this will be the hostname of the machine)
- Username: choose a username you will remember - this is your Linux login
- Password: set a strong password

OpenSSH Installation

This is one of the most important screens. You will see a checkbox option for 'Install OpenSSH server'. Make sure this is checked before continuing. Without this, the server will not be remotely accessible after installation.

Featured Snaps

The installer will show a list of optional software packages presented as 'featured snaps'. Skip all of these. The homelab uses Docker for service deployment, and adding snap packages at install time would introduce unnecessary complexity. All required software will be installed later in a controlled manner.

Completing Installation and First Reboot

Once the installation completes, the installer will prompt you to remove the USB drive and press Enter to reboot. Remove the USB when prompted, then allow the system to restart.

i Note

The first boot may take longer than expected - sometimes up to 2–3 minutes - because the system is initialising network services for the first time. If the boot appears to hang at a 'waiting for network configuration' message, this is normal on this hardware. If it does not resolve after 3 minutes, reboot with the Ethernet cable already connected before startup.

4.7 First Login and System Update

Once the server has booted, you will see a text login prompt. Log in with the username and password you set during installation. You will see the bash command line.

Before doing anything else, update the system. This refreshes the package database and installs any security patches or updates that have been released since the installation image was built. Run the following commands one at a time:

```
sudo apt update
sudo apt upgrade -y
```

The first command refreshes the list of available packages from Ubuntu's repositories. The second downloads and installs any packages that have newer versions available. The -y flag automatically confirms all prompts. This process may take several minutes on the first run.

✓ Result

After the update completes, reboot the server one more time with 'sudo reboot'. This ensures any kernel updates are applied cleanly. After the reboot, the server is ready for the next phase.

4.8 Troubleshooting - OS Installation

The following problems were encountered during the OS installation of this specific homelab. They are documented here so that if you encounter the same issues, you understand exactly what caused them and how to resolve them. These are real incidents, not hypothetical scenarios.

USB Boot Failure on First Attempt

On the first boot attempt, the Dell failed to boot from the installer USB. The system either displayed a blank screen or an error message and fell back to the BIOS splash screen.

Important

Root cause: The USB was created in Rufus using GPT partitioning and UEFI target settings. The Dell N4030 uses legacy BIOS, which cannot read a UEFI-oriented bootloader. The fix was to recreate the USB in Rufus with MBR partitioning, BIOS or UEFI-CSM target, FAT32 file system, and to accept the GRUB download when Rufus prompted for it. The machine booted correctly on the next attempt.

Confusion Between Ubuntu 22.04 and 24.04

During the USB recreation stage, there was brief confusion between two Ubuntu LTS server images being considered. This caused an additional delay while the correct ISO was confirmed.

Note

Resolution: The decision was made to use Ubuntu Server 22.04 LTS specifically. This was the correct choice for older hardware because 22.04 has more mature package support, more community documentation, and has been running in production environments for longer than 24.04.

Installer Freezing at 'Reading Package Lists'

During installation, the system stalled repeatedly at the mirror step, where the installer attempts to contact an online package repository. The installer eventually crashed or entered an unresponsive state.

Important

Root cause: The combination of slow HDD I/O, unstable install-time networking on older hardware, and the installer's dependency on a responsive mirror caused a timeout that the installer could not recover from gracefully. **Resolution:** The Ethernet cable was disconnected before restarting the installer. With no network, the installer skipped the mirror step and completed successfully. All package updates were applied afterward using `sudo apt update` and `sudo apt upgrade -y` once the OS had fully booted.

Installer Crash Loop

After the initial mirror failure, the installer entered a cycle where it crashed and restarted to the same failure point repeatedly without progressing.

Note

Resolution: The Dell was powered off completely, the USB was re-inserted, and the installation was restarted from the very beginning. With the Ethernet cable already unplugged, the crash did not recur.

'Failed to Unmount /cdrom' on Reboot

At the end of installation, when the system prompted to remove the USB and reboot, a 'failed to unmount /cdrom' error appeared.

Result

Root cause: The installer attempted to cleanly detach the USB device before rebooting, but the USB was still physically mounted. **Resolution:** The USB was removed manually and the reboot was allowed to continue. This error causes no damage to the installation whatsoever. The OS was fully intact after reboot.

Boot Hang at Network Initialisation After First Reboot

After the first successful reboot following installation, the system appeared to stall for several minutes on a message related to waiting for network configuration before continuing the boot sequence.

Important

Root cause: Because networking had been bypassed during installation, the network service configuration was incomplete. On first boot, systemd waited for a network interface to come up and timed out slowly before continuing. **Resolution:** The machine was rebooted with an Ethernet cable already connected before startup. With a physical connection present, the interface received a DHCP address, the network wait was satisfied, and the system reached the login prompt normally.

5. Initial Server Configuration

After Ubuntu Server was installed successfully, the next step was to prepare the system for stable remote administration and reliable log correlation. This section covers the basic host configuration that must be completed before deploying Docker, Tailscale, the firewall rules, or the logging stack.

The goal of this phase is simple: make the server reachable, predictable, and ready for future services.

5.1 Static IP Planning

A stable IP address is important for a homelab because most services will be accessed from another machine on the local network. Grafana, DVWA, Juice Shop, SSH, and other services are all reached through the Dell server's network address. If that address changes after every reboot, bookmarks, firewall rules, dashboard URLs, and documentation become harder to maintain.

A static IP configuration is therefore recommended. If a static IP is already configured from the router through DHCP reservation, the server does not need manual Netplan changes. If the IP is configured directly on Ubuntu Server, Netplan should be used.

5.2 Netplan Configuration

Ubuntu Server 22.04.5 LTS uses Netplan for network configuration. Netplan configuration files are stored in:

```
/etc/netplan/
```

Run the following to find your file name:

```
ls /etc/netplan/
```

You will see a file with a name like 00-installer-config.yaml. Open it for editing:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

Replace the contents with a configuration similar to the following (adjust the IP addresses to match your network):

```
network:
  version: 2
  ethernets:
    enp2s0:                                # Replace with your interface name
```

```
dhcp4: no
addresses:
  - <YOUR-SERVER-IP>/24      # Your chosen static IP
routes:
  - to: default
    via: <YOUR-ROUTER-IP>    # Your router's IP
nameservers:
  addresses: [8.8.8.8, 1.1.1.1]
```

To find your actual interface name, run:

```
ip link show
```

The interface name will be something like enp2s0 or eth0. After saving the file, apply the configuration:

```
sudo netplan apply
```

⚠ Important

If you lose SSH connectivity after applying Netplan, use the physical keyboard on the Dell to verify the interface came up with the correct IP using 'ip a'. Double-check the YAML indentation - Netplan configuration files are sensitive to incorrect spacing.

5.3 SSH Access

SSH is the main method used to manage the Dell server after installation. Since OpenSSH was enabled during Ubuntu Server installation, the server should be reachable from the Acer workstation.

From the Acer, connect to the Dell using:

```
ssh user@<YOUR-SERVER-IP>
```

Replace:

```
user
```

with the Linux username, and replace:

```
<YOUR-SERVER-IP>
```

with the Dell server's actual lab IP address.

The first connection may ask whether the host key should be trusted. Type:

```
yes
```

Then enter the Linux user password.

✓ Result

A successful SSH login confirms that the Dell can be managed remotely and does not require direct keyboard and monitor access for normal operation.

5.4 Timezone Configuration

Correct time settings are important for log analysis. If the server timezone is wrong, Grafana panels and raw logs may show events at confusing times. This becomes a problem when comparing attack activity with log timestamps.

Set the local timezone:

```
sudo timedatectl set-timezone <LOCAL_TIMEZONE>
```

Example format:

```
Region/City
```

To list available timezones:

```
timedatectl list-timezones
```

Verify the current timezone:

```
timedatectl
```

⚠ Important

Time accuracy matters in SOC work. If the attack happens at one time but the logs show another, investigation becomes harder. Set the timezone before collecting serious lab evidence.

5.5 Troubleshooting: Initial Server Configuration

SSH disconnected after applying Netplan

This may happen because the network interface restarts when Netplan applies the new configuration. If the static IP is different from the previous DHCP address, the old SSH session will no longer work.

Fix:

```
ip a
```

Use the Dell keyboard and screen to verify the new IP address, then reconnect from the Acer using the updated address.

Wrong network interface name

If the Netplan file uses the wrong interface name, the server may not get network connectivity.

Fix:

```
ip link show
```

Use the correct interface name in the Netplan YAML file.

YAML indentation issue

Netplan files require correct indentation. A spacing error can prevent the network configuration from loading.

Fix:

```
sudo netplan try
```

This command tests the configuration before making it permanent.

Wrong timezone in Grafana logs

If Grafana appears to show logs at the wrong time, check the server timezone:

```
timedatectl
```

Set the correct timezone if needed:

```
sudo timedatectl set-timezone <LOCAL_TIMEZONE>
```

Part 3: Core Platform Services

6. Docker Platform Setup

Docker is the main platform used to run the homelab services. Instead of installing every vulnerable application and monitoring service directly on the host operating system, Docker keeps services isolated inside containers.

This makes the lab easier to manage, easier to reset, and safer to operate.

6.1 Why Docker Was Used

Docker was chosen because it fits the hardware and learning goals of the project.

Docker provides:

- Isolation between services
- Fast deployment and removal
- Simple rebuilds when something breaks
- Lower overhead than full virtual machines
- Cleaner host operating system management
- Easy networking between containers through a shared Docker network

This is important because the Dell server has limited CPU power and uses a mechanical HDD. Running every service as a full virtual machine would create unnecessary overhead. Docker keeps the system lightweight enough to run the logging stack and vulnerable applications at the same time.

6.2 Docker Installation

Docker is the container runtime that underpins almost every service in this homelab. A container is a lightweight, isolated process that packages an application along with everything it needs to run - its code, runtime, configuration, and dependencies - into a single portable unit. Containers start and stop quickly, use far fewer resources than virtual machines, and can be removed and recreated cleanly without leaving residue on the host system.

For this homelab, Docker is used instead of installing services directly on the host OS for several reasons. First, it keeps the host system clean - if DVWA needs to be removed and rebuilt, you remove the container without touching the host. Second, it provides isolation - a vulnerable application running in a container cannot easily compromise the host. Third, it makes the setup reproducible - the exact same environment can be recreated from a docker run command.

Install Docker with the following commands:

```
sudo apt update
sudo apt install -y docker.io
```

Enable Docker so it starts automatically at every boot, and start it immediately:

```
sudo systemctl enable docker
sudo systemctl start docker
```

By default, Docker requires sudo for every command because the Docker daemon runs as root. This becomes inconvenient during regular lab work. Add your user to the docker group to allow running Docker commands without sudo:

```
sudo usermod -aG docker $USER
newgrp docker
```

The first command adds your current user to the docker group. The second command refreshes your shell session's group membership so the change takes effect immediately without needing to log out and back in.

Verify the installation works correctly by running the Docker test container:

```
docker run hello-world
```

✓ Result

If Docker is installed and working correctly, this command will pull a small test image from Docker Hub and print a success message. This confirms that Docker can pull images from the internet, create containers, and execute them.

6.3 Docker Network: labnet

A custom Docker network was created so containers can communicate with each other by container name.

Create the network:

```
docker network create labnet
```

This network is used by:

- Loki
- Grafana
- Grafana Alloy
- DVWA
- Juice Shop
- Caddy

The shared network matters because services need to communicate internally. For example, Grafana connects to Loki using:

```
http://loki:3100
```

This works because both containers are attached to the same Docker network.

i Note

If containers are not on the same Docker network, container-name resolution may fail. This can cause Grafana to fail when connecting to Loki, or Alloy to fail when forwarding logs to Loki.

6.4 Docker Container Lifecycle

These commands are used frequently during the homelab build and maintenance process.
List running containers:

```
docker ps
```

List all containers, including stopped containers:

```
docker ps -a
```

View logs for a container:

```
docker logs <container_name>
```

Start a stopped container:

```
docker start <container_name>
```

Stop a running container:

```
docker stop <container_name>
```

Restart a container:

```
docker restart <container_name>
```

Remove a container forcefully:

```
docker rm -f <container_name>
```

Inspect the lab network:

```
docker network inspect labnet
```

These commands are important because Docker services may need to be recreated when configuration files, volume mounts, or network settings change.

6.5 Restart Policy Strategy

Not every container should start automatically after a reboot. The lab separates core monitoring services from vulnerable targets.

Core services should auto-start:

- Loki
- Grafana
- Grafana Alloy
-

These services support logging and monitoring, so they should come back automatically when the Dell restarts.

Apply restart policy:

```
docker update --restart unless-stopped loki
docker update --restart unless-stopped grafana
docker update --restart unless-stopped alloy
```

The restart policy is applied directly in the docker run commands for Loki, Grafana, and Alloy using `--restart unless-stopped`. The docker update command is only needed if a container was already created without a restart policy.

Vulnerable targets should remain manually started:

- DVWA
- Juice Shop
- Caddy
- Metasploitable 2

The vulnerable targets should only run when they are needed for testing. This reduces unnecessary attack surface and keeps the server lighter when the lab is not active.

Important

Core monitoring services should be reliable. Vulnerable targets should be controlled. This separation helps keep the lab safer and easier to manage.

6.6 Troubleshooting: Docker

Permission denied when running Docker commands

If Docker commands show permission errors, the user may not be part of the Docker group.
Fix:

```
sudo usermod -aG docker $USER
newgrp docker
```

Then test again:

```
docker ps
```

Container cannot reach another container by name

If Grafana cannot reach Loki using `http://loki:3100`, the containers may not be on the same Docker network.

Check the network:

```
docker network inspect labnet

Connect a container to the network if needed:
docker network connect labnet <container_name>
```

Wrong volume mount path

If a container does not see a configuration file, check that the host path exists and is mounted correctly.

Example:

```
ls -l ~/logging/alloy/config.alloy
```

If the file does not exist, the container may have mounted an empty directory instead of the intended file.

7. Tailscale Remote Access

Tailscale is used to access the homelab securely when the Acer workstation is not on the same local network as the Dell server. It provides encrypted remote access without router port forwarding.

7.1 Why Tailscale Was Chosen

Tailscale was chosen because it provides secure remote access without exposing the homelab directly to the public internet.

It helps avoid:

- Router port forwarding
- Public IP exposure
- Dynamic DNS setup
- Manual WireGuard configuration
- Opening vulnerable services to the internet
-

Each device joined to the same Tailscale network receives a private Tailscale IP address, shown in sanitized form as:

```
100.x.x.x
```

This allows SSH and browser access to the Dell from outside the home network.

7.2 Installing Tailscale on the Dell

The standard Tailscale one-line install command uses a pipe symbol (|) to pass the downloaded script directly to bash. On the Dell, the keyboard was configured with a non-standard layout that made the pipe character difficult to type. To work around this, the installer was downloaded first and then executed in a separate step:

```
curl -fsSL https://tailscale.com/install.sh -o install.sh
chmod +x install.sh
sudo ./install.sh
```

After installation, bring Tailscale up and authenticate the device:

```
sudo tailscale up
```

This command will print a URL. Copy that URL and open it in a browser on any device. Log in with your Tailscale account (a free account suffices) and authorise the Dell. Once authorised, the Dell will receive a Tailscale IP address. Find it with:

```
tailscale ip -4
```

7.3 Installing Tailscale on the Acer

Download and install the Tailscale Windows client from the Tailscale website (tailscale.com). Log in using the same account you used on the Dell. Both devices must be logged into the same Tailscale account for them to appear on the same private network (called a 'tailnet').

Important

A common mistake is logging in to Tailscale on each device with different accounts. If you do this, the devices will be on separate tailnets and will not be able to see each other. Always use the same Google, Microsoft, or email account on both devices.

Once both devices are on Tailscale, verify connectivity by pinging the Dell's Tailscale IP from the Acer:

```
ping <YOUR-TAILSCALE-IP> # Replace with your Dell's actual Tailscale IP
```

Note

The Dell's LAN IP (e.g., <YOUR-SERVER-IP>) is used when both devices are on the same home network. The Tailscale IP (e.g., <YOUR-TAILSCALE-IP>) is used when accessing the server remotely from outside the home network. Both will work from inside the home network.

7.4 Remote Access Model

The Dell has two main access paths.
Local LAN access:

```
<YOUR-SERVER-IP>
```

Tailscale remote access:

```
100.x.x.x
```

When the Acer is on the same local network, local LAN access can be used. When the Acer is outside the local network, Tailscale access should be used.

Example Grafana access through Tailscale:

```
http://100.x.x.x:3000
```

Example SSH access through Tailscale:

```
ssh user@100.x.x.x
```

Note: Tailscale provides access to the Dell server, but it does not automatically provide access to every internal network behind the Dell. Metasploitable routing requires separate handling because it runs on a VirtualBox host-only network.

7.5 Tailscale and Metasploitable Routing Note

Metasploitable 2 runs on a VirtualBox host-only network using a sanitized example range:

```
<YOUR-HOST-ONLY-NETWORK>
```

The Dell can reach Metasploitable because the Dell owns the VirtualBox host-only adapter. The Acer and Kali VM cannot automatically reach Metasploitable through Tailscale unless routing is configured.

Possible future options include:

- Tailscale subnet route advertising from the Dell
- IP forwarding between tailscale0 and vboxnet0
- SSH tunneling through the Dell
- Running attack tools directly from the Dell
- Redesigning the network

Important: Metasploitable is intentionally isolated. It should not be treated as directly reachable from the Acer or Kali VM unless a controlled route is configured.

7.6 Troubleshooting: Tailscale

Acer cannot ping the Dell Tailscale IP

The most common cause is that only the Dell has been enrolled in Tailscale.

Fix:

- Install Tailscale on the Acer.
- Sign in with the same account used on the Dell.
- Confirm both devices appear in the same Tailscale network.
- Retry the ping test.

Tailscale works at home but not on another network

Some networks may restrict VPN traffic. This may happen on university or corporate networks.

Fix:

- Try another network or hotspot.
- Check Tailscale status:

```
tailscale status
```

Confirm the Dell is online in the Tailscale admin panel.

Tailscale not active after reboot

Check service status:

```
sudo systemctl status tailscaled
```

Enable it at boot:

```
sudo systemctl enable tailscaled  
Start it manually if needed:  
sudo systemctl start tailscaled
```

8. Firewall and Security Baseline

UFW is used as the host firewall on the Dell server. Since this homelab includes intentionally vulnerable targets, firewall control is important. The firewall does not make vulnerable applications secure, but it reduces unnecessary exposure and limits which systems can reach which services.

8.1 UFW Purpose

UFW provides a simpler way to manage Linux firewall rules. The goal is to deny inbound traffic by default, then allow only the specific services required for the lab.

This is important because the server hosts:

- Vulnerable web applications
- Logging services
- Remote access services
- A vulnerable VM target
- Future home server services

The firewall should therefore follow the principle of least privilege.

8.2 Firewall Design Rules

The firewall is configured with a default-deny inbound policy. Only required services are allowed, and those rules should be limited to trusted LAN, Tailscale, or host-only interfaces where possible. Vulnerable services should not be exposed broadly. Backend services such as Loki and Alloy should remain internal or admin-only.

8.3 Firewall Setup with UFW

UFW (Uncomplicated Firewall) is Ubuntu's front-end for managing the Linux kernel's netfilter firewall rules. It allows firewall rules to be configured without writing complex iptables commands manually.

For a server that will host intentionally vulnerable applications, a correctly configured firewall is a critical baseline control. It does not make vulnerable applications secure internally, but it reduces unnecessary exposure by allowing only the specific traffic required for the lab.

At this stage, only the **base firewall policy** is configured. Service-specific rules are added later when each service is deployed.

Important

UFW alone must not be treated as complete protection for Docker-published ports. Docker modifies firewall rules when containers publish ports with `-p`, and published container ports can bypass UFW filtering. In this homelab, backend services such as Loki and Alloy should be bound to `127.0.0.1` where possible. Vulnerable targets should be started only during lab sessions. If strict source-based filtering is required for Docker-published services, use

Docker-aware firewall controls such as the DOCKER-USER chain rather than relying only on UFW allow rules.

```
sudo apt install ufw -y
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow ssh
sudo ufw enable
sudo ufw status
```

Here is what each command does and why it matters:

- `sudo ufw default deny incoming` - sets the default policy to block all inbound connections. Nothing gets in unless you explicitly allow it. This is the correct default posture for any server.
- `sudo ufw default allow outgoing` - allows the server to initiate outbound connections (for package downloads, Tailscale, DNS lookups, etc.) without restriction.
- `sudo ufw allow ssh` - creates a rule allowing inbound connections on port 22 (SSH). This is critical. If you enable the firewall without allowing SSH first, you will lock yourself out of the server.
- `sudo ufw enable` - activates the firewall with the rules defined above.
- `sudo ufw status` - shows the current firewall status and all active rules. Verify that SSH is listed as allowed.

⚠ Important

Always run 'sudo ufw allow ssh' before 'sudo ufw enable'. Failing to do this will block your SSH connection and require a physical keyboard and monitor to recover access.

As new services are deployed, firewall rules should be added only for the services that need to be reachable. These rules should be restricted to the trusted LAN or Tailscale network wherever possible.

Examples of later service-specific rules include:

```
sudo ufw allow from <YOUR-LAN-SUBNET> to any port 3000 proto tcp    # Grafana
sudo ufw allow from <YOUR-LAN-SUBNET> to any port 8080 proto tcp    # DVWA
sudo ufw allow from <YOUR-LAN-SUBNET> to any port 3002 proto tcp    # Juice Shop via
Caddy
sudo ufw allow in on vboxnet0 from <YOUR-VM-IP> to any port 514 proto udp    #
Metasploitable syslog
```

These are sanitized examples. Replace 192.168.x.0/24 and <YOUR-HOST-ONLY-NETWORK> with the actual trusted lab network values.

Do not broadly expose internal backend services such as Loki on port 3100 or Alloy UI on port 12345. These should remain internal, restricted, or admin-only.

Also, Juice Shop direct port 3001 should be treated as diagnostic only. Detection testing should use Caddy on port 3002 because that path creates the access logs used by the dashboard.

Future services such as Pi-hole, Samba, and Jellyfin should receive firewall rules only if they are actually deployed later. They should not be listed as active firewall rules in the current setup.

8.4 Security Hygiene

The firewall is only one part of the security baseline. The lab should also follow these rules:

- Keep vulnerable targets private.
- Do not expose DVWA, Juice Shop, or Metasploitable to the public internet.
- Start vulnerable targets only when they are needed.
- Stop vulnerable targets after testing.
- Use Tailscale for remote administration instead of router port forwarding.
- Keep Ubuntu Server updated.
- Use strong passwords for server accounts.
- Stay within the lab boundary and only test systems owned by the user.

8.5 Troubleshooting: UFW

SSH locked out after enabling UFW

This happens if UFW is enabled before SSH is allowed.

```
sudo ufw allow ssh
```

Fix from local console:

```
sudo ufw reload
```

Service not reachable later after adding a firewall rule

This does not belong to the baseline setup, but it is a common issue during later service deployment.

Check whether the service is running:

```
docker ps
```

Check whether the port is listening:

```
sudo ss -tln | grep <port>
```

Check UFW rules:

```
sudo ufw status numbered
```

Rule is too broad

If a rule allows a service from anywhere and it should be restricted, delete the broad rule and replace it with a restricted one.

Show numbered rules:

```
sudo ufw status numbered
```

Delete a rule by number:

```
sudo ufw delete <rule_number>
```

Then add a safer restricted rule in the relevant service section.

Part 4: Logging Stack

9. Logging Stack Overview: Loki, Alloy, Grafana

9.1 Why Logging Matters

Without a logging pipeline, any attack run against DVWA, Juice Shop, or Metasploitable is just a demonstration. The tool produces output, the exploit succeeds or fails, and nothing is permanently recorded or correlated. With a logging pipeline in place, host events, container output, authentication activity, HTTP access metadata, and many attack indicators become searchable log entries that can be retrieved, queried, and analyzed at any point afterward.

This transforms the homelab from a collection of vulnerable applications into a functional mini Security Operations Center (SOC) environment. The workflow becomes: perform an attack from the Kali VM, switch to Grafana on the Acer browser, query the logs from the target that was attacked, and see the exact log entries that the attack generated. This cause-and-effect observation is what develops real analytical skill.

9.2 Why Loki, Alloy, and Grafana

The logging stack was selected specifically because it is resource-efficient enough to run comfortably alongside other services on the Dell's hardware.

Component	Role	Why This Tool
Grafana Alloy	Log Collector	Alloy is Grafana's modern replacement for Promtail. It collects logs from system log files (syslog, auth.log) and from Docker container output in real time and forwards them to Loki. It is more flexible and actively maintained compared to Promtail.
Loki	Log Storage	Loki stores and indexes logs efficiently using a label-based system rather than full-text indexing. This makes it far less memory-hungry than Elasticsearch. On this hardware, Loki can run alongside other services without causing resource contention.
Grafana	Dashboard and Query UI	Grafana connects to Loki as a data source and provides a web interface for querying logs using LogQL, building dashboards, and running live log streams. This is the interface used from the Acer browser to investigate what attacks look like in the log record.

i Note

Promtail was replaced by Grafana Alloy in this homelab. See Section 9.3 for the rationale.

Why Not ELK Stack?

The industry-standard SIEM logging stack is ELK (Elasticsearch, Logstash, and Kibana). It is powerful, full-featured, and widely used in enterprise environments. It is also completely unsuitable for this hardware. Elasticsearch alone requires a minimum of 4 GB of RAM under light load, and more realistically 6 to 8 GB in operation. On a machine with 16 GB shared between the OS, Docker, all running containers, and the logging stack itself, that headroom does not exist.

The Loki stack achieves the same core capability (real-time log ingestion, persistent storage, label-based querying, and visual dashboards) at a fraction of the resource cost. Loki's architectural trade-off is that it does not index the content of log lines, only their labels (such as which job or container they came from). This means some query patterns that are trivial in Elasticsearch require more specific LogQL expressions in Loki. For a homelab focused on learning attack-to-detection correlation, this trade-off is entirely acceptable.

9.3 Promtail Replacement Note

Promtail was originally planned as the log collector for this homelab. During implementation, Grafana Alloy was used instead because it is the actively maintained successor to Promtail. Promtail remains available and functional, but Grafana has stated that all new feature development is focused on Alloy.

Alloy provides the same core functionality as Promtail (tailing log files, collecting Docker container logs, forwarding to Loki) with additional capabilities such as more flexible configuration syntax, better support for structured log formats, and integration with other observability tools. Promtail should only be mentioned as an earlier planned option that was replaced by Alloy. All setup instructions, configuration examples, and troubleshooting in this documentation use Alloy exclusively.

9.4 Logging Directory Structure

All logging configuration files are stored under a dedicated directory in the home folder. This keeps everything in one predictable location and makes the entire logging setup easy to back up or reproduce.

```
mkdir -p ~/logging/loki
mkdir -p ~/logging/alloy
```

After running these commands, the directory structure is:

```
~/logging/
loki-config.yml      # Loki server configuration
loki/                # Loki data directory (chunks, index, compactor)
alloy/
  config.alloy       # Alloy pipeline configuration
```

9.5 Docker Network for the Logging Stack

Before starting any of the logging containers, the shared Docker network labnet must be created if it does not already exist. This network allows Loki, Grafana, and Alloy to communicate with each other by container name rather than by IP address.

```
docker network inspect labnet >/dev/null 2>&1 || docker network create labnet
```

All logging stack containers (Loki, Alloy, Grafana) will be connected to this network using the `--network labnet` flag in their docker run commands. Later, vulnerable target containers (DVWA, Juice Shop, Caddy) will be attached to the same network when they are deployed.

Container name resolution only works within a user-defined Docker network. When Alloy sends logs to Loki, it references `http://loki:3100` in its configuration. This hostname resolves automatically because both containers are on labnet.

i Note

Container name resolution (using 'loki' as a hostname) only works when both containers are on the same user-defined Docker network. If Alloy is on labnet but Loki is not, Alloy cannot reach Loki using the hostname 'loki' and will need the host machine's IP address instead. Keeping all containers on labnet avoids this entirely and makes configuration cleaner.

9.6 Loki Configuration

Loki requires a configuration file before it can start. This file tells it where to store log data, how long to retain logs, what schema version to use, and how to handle replication. The configuration used here is a single-node setup, appropriate for a homelab where there is only one server and no need for clustering or redundancy.

Understanding the Key Configuration Options

Each block in the Loki configuration controls a specific aspect of its behaviour:

- `auth_enabled: false` - Loki supports authentication for multi-tenant deployments. In this homelab there is only one user, so authentication is disabled. This simplifies setup and removes the need to pass credentials in every API call.
- `server: http_listen_port: 3100` - Loki listens for incoming log pushes and API queries on port 3100. This is the standard Loki port and matches what Alloy and Grafana are configured to use.
- `schema_config` - Defines how Loki structures its index. Schema v13 with TSDB storage is the current recommended option for new Loki deployments. The 'from' date of 2025-01-01 tells Loki this schema applies to all logs from that date onward.
- `storage_config: filesystem` - Logs are stored on the local disk rather than in cloud object storage. The path `/loki/chunks` inside the container maps to `~/logging/loki` on the Dell host through the volume mount defined in the docker run command.
- `limits_config: retention_period: 168h` - Logs older than 7 days (168 hours) are automatically deleted by the compactor. This prevents the HDD from filling up with log data over time.
- `compactor` - Handles log retention enforcement and index compaction. The `compaction_interval` of 10 minutes means Loki checks for expired logs every 10 minutes and cleans them up.

Create the Loki Configuration File

```
nano ~/logging/loki-config.yml
```

Paste the following configuration exactly as shown:

```
auth_enabled: false
server:
  http_listen_port: 3100
common:
  instance_addr: 127.0.0.1
  path_prefix: /loki
  replication_factor: 1
  ring:
    kvstore:
      store: inmemory
schema_config:
  configs:
    - from: 2025-01-01
      store: tsdb
      object_store: filesystem
      schema: v13
      index:
        prefix: index_
        period: 24h
storage_config:
  filesystem:
    directory: /loki/chunks
limits_config:
  retention_period: 168h
compactor:
  working_directory: /loki/compactor
  compaction_interval: 10m
  retention_enabled: true
  delete_request_store: filesystem
```

Fix Directory Permissions

This step is critical and easy to miss. The Loki container runs its internal process as user ID 10001, not as root. The `~/logging/loki` directory on the host is owned by the regular user account, which typically has a different UID. When Loki tries to write log chunks and index files into the mounted volume, it will fail with a permission error if the directory is not owned by UID 10001.

```
sudo mkdir -p ~/logging/loki
sudo chown -R 10001:10001 ~/logging/loki
sudo chmod -R u+rwX ~/logging/loki
```

Important

If the `chown` step is skipped and Loki is started, it will appear to be running when checked with `'docker ps'`, but `'docker logs loki'` will show repeated permission denied errors. Always fix permissions before starting Loki for the first time.

Start the Loki Container

The `docker run` command for Loki mounts two volumes: the data directory where Loki writes its chunks and index, and the configuration file. The `-config.file` flag tells Loki where to find its configuration inside the container.

```
docker run -d --name loki \
  --network labnet \
  --restart unless-stopped \
  -p 127.0.0.1:3100:3100 \
  -v ~/logging/loki:/loki \
  -v ~/logging/loki-config.yml:/etc/loki/local-config.yaml \
  grafana/loki:2.9.0 \
  -config.file=/etc/loki/local-config.yaml
```

Important

The Loki port is bound to 127.0.0.1 because Loki is a backend service. Grafana and Alloy reach Loki internally through the Docker `labnet` network using `http://loki:3100`, so Loki does not need to be exposed to the LAN. This configuration was tested with `grafana/loki:2.9.0`. If a newer Loki image is used, review the official Loki configuration and upgrade notes before changing the image tag.

Verify Loki is Running

Wait about 10 seconds for Loki to finish initialising, then check its readiness endpoint:

```
curl http://localhost:3100/ready
```

✓ Result

When Loki is ready and accepting log pushes, this command returns a single word: 'ready'. If the result is 'connection refused', Loki has not started yet. Check 'docker logs loki' for the reason.

9.7 Alloy Configuration

Alloy is the log collector. Its job is to watch specific log sources on the Dell host and inside Docker containers, and to forward everything it reads to Loki. The Alloy configuration file uses an HCL-based syntax (similar to Terraform) where sources are defined and connected to destinations using named references.

Base Alloy Configuration

This is the base Alloy configuration. It collects from three distinct sources:

- System syslog (/var/log/syslog): general system events from the Ubuntu host, including kernel messages, service start and stop events, and cron job output.
- Authentication log (/var/log/auth.log): all authentication events on the host, including SSH login attempts (both successful and failed), sudo usage, and PAM authentication results. This is the most important log source for detecting brute force or lateral movement attempts.
- Docker container logs: Docker logs may capture Juice Shop application output, but Juice Shop HTTP access detection is handled later through **Caddy JSON access logs**.

i Note

This is the base Alloy configuration. Later sections extend it with Caddy access log collection for Juice Shop and the Metasploitable rsyslog log file.

Understanding the Alloy Configuration Structure

The Alloy configuration is composed of blocks that are chained together. Each block either discovers a source, reads from it, transforms it, or delivers it somewhere. The flow for file-based logs is:

- local.file_match: defines which file paths to watch and assigns labels (job, host) to logs coming from those paths
- loki.source.file: reads the matched files and forwards log lines to a downstream receiver
- loki.write: the final delivery block that pushes logs to Loki's push API endpoint

For Docker container logs, the flow is slightly different because containers need to be discovered first:

- `discovery.docker`: connects to the Docker socket and discovers all running containers
- `discovery.relabel`: processes the container metadata to extract a clean container name label from the raw Docker metadata format
- `loki.source.docker`: reads log output from all discovered containers and forwards it
- `loki.process`: adds static labels (`job=docker`, `host=homelab`) before delivery

Create the Alloy Configuration File

```
nano ~/logging/alloy/config.alloy
```

Paste the following configuration:

```
// SYSLOG
local.file_match "syslog" {
  path_targets = [{
    "__path__" = "/var/log/syslog",
    "job"      = "syslog",
    "host"     = "homelab"
  }]
}
loki.source.file "syslog" {
  targets      = local.file_match.syslog.targets
  forward_to   = [loki.write.loki.receiver]
  tail_from_end = true
}
// AUTH LOG
local.file_match "authlog" {
  path_targets = [{
    "__path__" = "/var/log/auth.log",
    "job"      = "authlog",
    "host"     = "homelab"
  }]
}
loki.source.file "authlog" {
  targets      = local.file_match.authlog.targets
  forward_to   = [loki.write.loki.receiver]
  tail_from_end = true
}
// DOCKER LOGS
discovery.docker "containers" {
  host = "unix:///var/run/docker.sock"
}
discovery.relabel "containers" {
  targets = discovery.docker.containers.targets
  rule {
    source_labels = ["__meta_docker_container_name"]
    target_label  = "container"
    regex         = "/(.*)"
  }
}
```

```
    replacement = "$1"
  }
}
loki.source.docker "containers" {
  host      = "unix:///var/run/docker.sock"
  targets   = discovery.relabel.containers.output
  forward_to = [loki.process.docker.receiver]
}
loki.process "docker" {
  stage.static_labels {
    values = {
      job = "docker",
      host = "homelab"
    }
  }
  forward_to = [loki.write.loki.receiver]
}
// SEND TO LOKI
loki.write "loki" {
  endpoint {
    url = "http://loki:3100/loki/api/v1/push"
  }
}
}
```

Start the Alloy Container

The Alloy container requires access to three things that must be mounted from the host: its own configuration file, the `/var/log` directory where `syslog` and `auth.log` live, and the Docker socket so it can discover and read container logs. The container runs as root (`--user root`) because reading `/var/log/auth.log` requires elevated privileges on Ubuntu.

```
docker run -d --name alloy \
  --network labnet \
  --restart unless-stopped \
  --user root \
  -p 127.0.0.1:12345:12345 \
  -v ~/logging/alloy/config.alloy:/etc/alloy/config.alloy \
  -v /var/log:/var/log:ro \
  -v /var/run/docker.sock:/var/run/docker.sock \
  grafana/alloy:latest \
  run /etc/alloy/config.alloy \
  --server.http.listen-addr=0.0.0.0:12345
```

Security note:

Mounting `/var/run/docker.sock` gives Alloy visibility into Docker containers and is required for Docker log discovery in this setup. It is also a sensitive mount because access to the Docker socket can provide broad control over the Docker host. In this homelab, this is accepted as a controlled trade-off because the Dell is a private lab server. Do not expose the Alloy UI or Docker socket to untrusted users.

Breaking down the key flags:

- `-v /var/log:/var/log:ro` - mounts the host's `/var/log` directory into the container as read-only. The `:ro` flag ensures Alloy cannot accidentally write to or delete log files on the host.
- `-v /var/run/docker.sock:/var/run/docker.sock` - gives Alloy access to the Docker daemon socket, which is how it discovers and reads container log streams without needing to exec into each container.
- `--user root` - required because `/var/log/auth.log` on Ubuntu is readable only by root by default. Running Alloy as root inside the container allows it to read this file.
- `--server.http.listen-addr=0.0.0.0:12345` starts Alloy's built-in UI inside the container. The Docker port binding restricts host access to `127.0.0.1`, so the UI is available only from the Dell host unless an SSH tunnel is used.
- `tail_from_end = true` means Alloy begins reading only new log entries created after the collector starts. This avoids bulk-ingesting old logs and reduces load on the Dell's HDD. If historical ingestion is required, this setting must be changed deliberately.

Important

Alloy UI on port 12345 is an admin and debug interface. It should not be broadly exposed through UFW. Access should be restricted to local or trusted network use only. The Alloy UI is useful for troubleshooting pipeline issues but is not required for normal operation.

Verify Alloy is Working

Check Alloy's container logs immediately after starting it. A correctly started Alloy instance will not show any configuration parse errors, and it will not show 'connection refused' errors when trying to reach Loki:

```
docker logs alloy
```

Important

If a 'connection refused' error pointing to `loki:3100` appears, either Loki is not running or Alloy is not on the same Docker network as Loki. Verify with `'docker network inspect labnet'` that both containers appear in the network's container list.

9.8 Grafana Configuration

Grafana is the interface through which logs are actually interacted with. It runs as a Docker container on the Dell and is accessed from the Acer browser. Once Grafana is running and connected to Loki as a data source, any logs that Alloy has collected can be queried.

Start the Grafana Container

```
docker run -d --name grafana \  
--network labnet \  
--volume /var/log:/var/log:ro \  
--volume /var/run/docker.sock:/var/run/docker.sock \  
--user root \  
--server.http.listen-addr=0.0.0.0:12345 \  
--tail.from.end=true \  
alloy
```

```
--restart unless-stopped \  
-p 3000:3000 \  
grafana/grafana
```

Grafana stores its configuration and any dashboards created in an internal SQLite database. By default, this lives inside the container, which means it is lost if the container is removed. Dashboards can be recreated, but mounting a persistent volume (covered in Section 9.9) is the recommended approach.

Access Grafana from the Acer

From the Acer browser, navigate to `http://<YOUR-SERVER-IP>:3000`, replacing the IP with the Dell's actual LAN address. The first-time login credentials are:

```
Username: admin  
Password: admin
```

i Note

Grafana will prompt to change the default password on first login. Set a new password and save it. The admin account has full access to all dashboards and data sources.

Add Loki as a Data Source

Before any logs can be queried, Grafana needs to know where Loki is running. Navigate through the Grafana interface as follows:

- In the left sidebar, go to Connections, then Data sources
- Click Add new data source
- Search for and select Loki from the list
- In the URL field, enter: `http://loki:3100`
- Scroll down and click Save and Test

i Note

The URL uses the container name 'loki' rather than an IP address. This works because both Grafana and Loki are on the same Docker network (labnet), which allows container name resolution. If 'localhost:3100' or the Dell's LAN IP is used instead, the connection will fail, because from inside the Grafana container, localhost refers to the Grafana container itself, not the Dell host.

9.9 Grafana Dashboard Persistence

By default, Grafana stores all dashboard configurations, data source settings, and user preferences in an internal SQLite database at `/var/lib/grafana` inside the container. If the Grafana container is removed, this database is lost and all dashboards must be recreated from scratch.

Recommended: Volume Mount for Persistence

The recommended approach is to mount a Docker volume to `/var/lib/grafana` when creating the Grafana container. This stores the database on the host filesystem, making it survive container removal and recreation. Grafana commonly runs as UID 472 inside the container, so the host directory needs correct ownership before the container starts.

```
mkdir -p ~/grafana-data
sudo chown -R 472:472 ~/grafana-data
```

Then start Grafana with the volume mount:

```
docker run -d --name grafana \
  --network labnet \
  --restart unless-stopped \
  -p 3000:3000 \
  -v ~/grafana-data:/var/lib/grafana \
  grafana/grafana
```

If Grafana is already running without a volume, stop and remove the container, set up the volume directory with correct ownership, and recreate:

```
docker stop grafana
docker rm grafana
mkdir -p ~/grafana-data
sudo chown -R 472:472 ~/grafana-data
docker run -d --name grafana \
  --network labnet \
  --restart unless-stopped \
  -p 3000:3000 \
  -v ~/grafana-data:/var/lib/grafana \
  grafana/grafana
```

Minimum: Dashboard JSON Export

If volume mounting is not desired, export dashboard JSON files periodically and store them safely. Each dashboard in Grafana can be exported from the dashboard settings menu and imported back after a container rebuild.

To export a dashboard: open the dashboard in Grafana, click the settings gear icon in the top-right corner, select JSON Model from the left sidebar, copy the entire JSON content, and save it to a file on the Acer workstation or Dell host.

To import a dashboard: in Grafana, go to Dashboards in the left sidebar, click New, then Import, paste the JSON content or upload the JSON file, and click Load.

9.10 Troubleshooting: Logging Stack

Loki Container Starts but Writes No Data

The Loki container shows as running in 'docker ps' but no logs appear in Grafana, and 'docker logs loki' shows repeated permission denied errors.

Important

Root cause: The ~/logging/loki directory is owned by the host user account, but the Loki process inside the container runs as UID 10001. When Loki tries to write its chunk and index files into the mounted volume, the OS rejects the writes because UID 10001 does not own the directory.

Resolution: Run 'sudo chown -R 10001:10001 ~/logging/loki' on the Dell host, then restart the Loki container with 'docker restart loki'. Loki will immediately be able to write data.

Alloy Cannot Connect to Loki

The Alloy container starts but 'docker logs alloy' shows repeated 'connection refused' or 'no such host' errors when trying to reach loki:3100.

Important

Root cause: Alloy is not on the same Docker network as Loki. Container name resolution (using 'loki' as a hostname) only works when both containers are on the same user-defined Docker network.

Resolution: Verify that both containers are on labnet by running 'docker network inspect labnet'. If either container is missing, connect it with 'docker network connect labnet loki' or 'docker network connect labnet alloy'. Restart Alloy afterward.

Grafana Cannot Connect to Loki Data Source

In Grafana, after entering http://loki:3100 as the data source URL and clicking Save and Test, Grafana returns a connection error.

Important

Root cause: The most common causes are that Grafana is not on labnet and cannot resolve the hostname 'loki', or the URL was typed with a trailing slash or typo.

Resolution: Verify Grafana is on labnet with 'docker network inspect labnet'. If it is not listed, run 'docker network connect labnet grafana' and retry. Confirm the URL in Grafana is exactly 'http://loki:3100' with no trailing slash.

Logs Appear with Incorrect Timestamps

Log entries in Grafana show timestamps that are several hours behind the actual clock time.

⚠ Important

Root cause: The Dell server's system timezone is set to UTC rather than the local timezone. All log files on Ubuntu use the system timezone for timestamps.

Resolution: Run 'sudo timedatectl set-timezone <LOCAL_TIMEZONE>' on the Dell, replacing <LOCAL_TIMEZONE> with the correct value (for example, Asia/Kuala_Lumpur or America/New_York). Then restart all containers with 'docker restart loki alloy grafana'. New log entries will immediately reflect the correct local time.

PART 5: MONITORING DASHBOARD

10. Homelab Monitoring Dashboard

10.1 Purpose of the Monitoring Dashboard

With the base logging pipeline confirmed and logs flowing from syslog, auth.log, and Docker sources, the first Grafana dashboard built for this homelab was the **Monitoring Dashboard**. Its purpose is separate from the detection dashboards covered later. It exists to verify that the infrastructure itself is healthy and that logs are moving correctly through the **Alloy** → **Loki** → **Grafana** pipeline.

This is **not** an attack detection dashboard. It is an operational health dashboard.

In real SOC environments, monitoring and detection are separate concerns. A monitoring dashboard answers infrastructure questions: are services alive, are logs being collected, are ingestion rates normal, and are timestamps correct? A detection dashboard answers threat questions: are there attack patterns, suspicious requests, repeated login attempts, or abnormal behaviour?

The Monitoring Dashboard answers one central question:

Is the logging pipeline working correctly?

Every panel exists to verify a specific log source. If a panel goes dark or becomes unexpectedly flat, it does not automatically mean an attack happened. It means the pipeline, time range, service state, or log source should be checked before relying on later detection dashboards.

10.2 Data Sources

The initial Homelab Monitoring Dashboard uses the base log streams collected by Grafana Alloy during the core logging stack setup.

The base monitoring sources are:

- **syslog**: Ubuntu system logs collected from /var/log/syslog.
- **authlog**: Authentication-related logs collected from /var/log/auth.log.
- **docker**: Docker container logs collected through the Docker socket.

These sources are enough to confirm that the main logging pipeline is working. They show whether Alloy is reading host logs, whether Docker log discovery is functioning, whether Loki is receiving logs, and whether Grafana can query those logs.

Later sections add additional log streams for specific targets:

- **Juice Shop access logs** are added later through Caddy JSON logging.
- **Metasploitable logs** are added later through the Dell rsyslog receiver and Alloy file collection.

This means the Monitoring Dashboard starts with the base host and Docker log streams. Target-specific dashboards are created later for DVWA, Juice Shop, and Metasploitable.

Loki Stream	What It Monitors
<code>{job="syslog"}</code>	General Ubuntu host system events, including kernel messages, cron jobs, and service state changes.
<code>{job="authlog"}</code>	Host authentication events, including SSH logins, sudo usage, PAM events, and session activity.
<code>{job="docker"}</code>	Combined output from Docker containers discovered through the Docker socket.
<code>{job=~"syslog authlog docker"}</code>	Unified view across the base monitoring sources.

10.3 Panel Structure

The dashboard consists of four core panels. Each panel verifies a different part of the logging pipeline.

Panel	Query	Purpose	Visualisation
Syslog Activity	<code>sum(count_over_time({job="syslog"}[1m]))</code>	Shows the count of syslog entries per minute. A healthy Ubuntu server usually produces low but regular syslog activity from system services, kernel events, cron activity, and background processes. If this panel shows no data, check whether <code>/var/log/syslog</code> is being written and whether Alloy is collecting it.	Time series
Auth Log Activity	<code>sum(count_over_time({job="authlog"}[1m]))</code>	Shows authentication-related activity from <code>/var/log/auth.log</code> . During normal operation, the count is usually low and may include SSH activity, sudo usage, PAM events, session open or close records, and other authentication-related system activity. A sudden increase should be reviewed in the raw auth log panel.	Time series
Docker Activity	<code>sum(count_over_time({job="docker"}[1m]))</code>	Shows whether Docker container logs are being collected by Alloy and stored in Loki. A flatline does not always mean Docker log collection has failed. It may simply mean the running containers are not currently producing logs during the selected time range.	Time series
All Logs Unified View	<code>{job=~"syslog authlog docker"}</code>	Displays entries from all base monitoring sources in one chronological view. This helps confirm that syslog, authlog, and Docker logs are all reaching Loki and that timestamps are consistent.	Logs

i Note

Deduplication should remain disabled in the unified logs panel because similar log lines may repeat during normal operation. Hiding repeated entries can make ingestion gaps harder to notice.

Optional Panel Added Later: DVWA Raw Logs

This panel should be added only after DVWA is deployed.

Panel	Query	Purpose	Visualisation
DVWA Raw Logs	<code>{job="docker", container="dvwa"}</code>	Verifies that Alloy is collecting logs from the DVWA container and that those logs are searchable in Grafana. This is not part of the initial base Monitoring Dashboard because DVWA is deployed later in the documentation.	Logs

10.4 Visual Settings

All time series panels on the Monitoring Dashboard use consistent settings to keep the view readable.

Setting	Recommended Value
Line width	2
Fill opacity	20 to 30 percent
Legend	Enabled
Time range	Last 15 minutes
Refresh interval	10 seconds or 30 seconds

All log panels use the following settings:

Setting	Recommended Value
Timestamp	Visible
Deduplication	Disabled
Log order	Newest logs shown clearly
Line wrapping	Enabled when needed

Shorter time ranges are recommended because the Dell server uses a mechanical HDD. Wide Loki queries over long time periods may be slower on this hardware.

10.5 What This Dashboard Confirms

The Homelab Monitoring Dashboard confirms that the core logging pipeline is working before attack-specific dashboards are built.

It confirms the following:

- Grafana Alloy is reading host log files such as `/var/log/syslog` and `/var/log/auth.log`.
- Grafana Alloy is collecting Docker container logs through the Docker socket.
- Loki is receiving and storing the base monitoring streams.
- Grafana can query Loki successfully.
- Time ranges, timestamps, and refresh settings are working correctly.
- Raw logs can be reviewed directly when a metric panel shows unusual activity.

This dashboard is not designed to be the main attack detection dashboard. Its purpose is operational visibility. It confirms that the logging stack is healthy before DVWA, Juice Shop, and Metasploitable detection dashboards are built.

Tip

Before every attack session, check the Monitoring Dashboard first. If the core monitoring panels show no data, fix the logging pipeline before generating attack traffic. Attack activity that never reaches Loki will not appear in the detection dashboards.

10.6 Troubleshooting: Monitoring Dashboard

Issue	Likely Cause	What to Check	Fix
All panels show no data	Grafana may not be connected to Loki, or the Loki data source URL may be incorrect.	In Grafana, go to Connections > Data sources and confirm Loki exists. The Loki URL should be <code>http://loki:3100</code> . Also check <code>docker ps</code> and <code>docker network inspect labnet</code> .	Fix the Loki data source URL, confirm Grafana and Loki are on labnet, and make sure Loki is running.
Syslog panel shows a flatline	<code>/var/log/syslog</code> may not be updating, or Alloy may not be reading it.	Run <code>sudo tail /var/log/syslog</code> . Check <code>rsyslog</code> with <code>sudo systemctl status rsyslog</code> . Check Alloy logs with <code>docker logs alloy --tail 50</code> .	Fix syslog service issues, Alloy file path issues, permission errors, or Alloy configuration errors.
Auth log panel shows no data	There may be no recent authentication events, the time range may be wrong, or Alloy may not be reading <code>/var/log/auth.log</code> .	Run <code>sudo tail /var/log/auth.log</code> . Check Grafana time range. Check Alloy logs with <code>docker logs alloy --tail 50</code> .	Adjust time range, verify the auth log file path, and fix any Alloy collection errors.
Docker Activity panel is flat	Containers may not be emitting logs, the time range may be wrong, the Docker socket may not be mounted, or Alloy Docker discovery may be misconfigured.	Run <code>docker ps</code> , then <code>docker logs <container_name></code> . Check Alloy logs with <code>docker logs alloy --tail 50</code> .	Confirm containers are producing logs, verify the Docker socket mount, check Alloy Docker discovery configuration, and restart Alloy only if configuration was changed.
Wrong time range in Grafana	The selected time range may not include recent logs.	Check dashboard time selector. Test the same query in Grafana Explore.	Set the time range to Last 15 minutes and refresh the dashboard.
Dashboard loads slowly	Queries may be too wide for the Dell's HDD-based storage.	Check the selected time range and number of heavy panels.	Use Last 5 minutes or Last 15 minutes during testing. Avoid querying several days of logs unless necessary.

Important

Connecting a container to labnet is not the main fix for missing Docker logs. Alloy collects Docker logs through the Docker socket, not through the Docker network. The labnet network is important for container-to-container communication, such as Grafana reaching Loki or Caddy reaching Juice Shop, but Docker log collection depends on the Docker socket mount and Alloy configuration.

PART 6: DVWA TARGET AND DETECTION

11. DVWA Deployment

DVWA (Damn Vulnerable Web Application) is an intentionally vulnerable PHP web application created specifically as a learning tool for cybersecurity professionals and students. It presents a collection of common web application vulnerabilities (SQL injection, XSS, command injection, file inclusion, brute force, CSRF, and others) each with configurable difficulty levels that adjust how well the application defends against attacks.

Deploying DVWA as the first vulnerable target in this homelab is the right starting point because it is straightforward to set up, its vulnerabilities are well-documented, and it produces meaningful log data when attacked, which directly feeds into the logging and detection work that follows.

11.1 Why DVWA Was Added First

DVWA is an appropriate first target for several reasons. Its vulnerabilities are clearly labelled and organised by module, which makes it easy to test one attack type at a time and directly correlate what was submitted with what appears in the logs. The application is a well-known and widely used training target, runs reliably enough for this lab in Docker, and its Apache web server produces access log output in a standard format that LogQL can query without additional parsing.

It also sets the educational baseline for the rest of the lab. After DVWA, each additional target introduces something new: Juice Shop adds API-based attacks and the challenge of structured JSON logging; Metasploitable adds service-level visibility at the OS layer. DVWA teaches the fundamentals first.

11.2 Why Docker for DVWA

DVWA is run as a Docker container rather than being installed directly on the host OS. The reason is isolation and control. DVWA is intentionally insecure, and installing it directly on the host would mean that vulnerabilities in DVWA could potentially be exploited to reach the host system itself. Running it in a container reduces direct impact on the host and keeps the vulnerable application separated from the base operating system for lab purposes. However, Docker should not be treated as a complete security boundary, especially when intentionally vulnerable applications are being used.

Running DVWA as a container also makes it trivial to tear down and rebuild. If the application state becomes corrupted during attack practice, removing the container and starting a fresh one restores a clean state immediately.

11.3 DVWA Container Deployment

Pull the DVWA image from Docker Hub before running the container:

```
docker pull vulnerables/web-dvwa
```

Run the DVWA container, connected to the labnet network:

```
docker run -d \  
  --name dvwa \  
  --network labnet \  
  -p 8080:80 \  
  vulnerables/web-dvwa
```

Allow trusted LAN access to DVWA:

Breaking down each flag:

- `-d` - runs the container in detached mode, so it continues running after the terminal command returns
- `--name dvwa` - assigns a human-readable name, allowing DVWA to be referenced by name in all subsequent commands
- `--network labnet` - connects the container to the shared lab Docker network so it stays consistent with the rest of the containerized lab services. Alloy Docker log collection does not depend on labnet. Alloy collects Docker logs through the mounted Docker socket and Docker discovery configuration.
- `-p 8080:80` - maps port 8080 on the Dell host to port 80 inside the container, where DVWA's Apache web server listens

```
sudo ufw allow from 192.168.x.0/24 to any port 8080 proto tcp  
Reload UFW:  
sudo ufw reload  
Check the rule:  
sudo ufw status numbered
```

This rule allows trusted systems on the lab LAN, such as the Acer workstation, to access DVWA on port 8080. Replace 192.168.x.0/24 with the actual trusted lab subnet. Use this LAN rule only if DVWA should be reachable from trusted devices on the home lab subnet. If DVWA should be accessed remotely, prefer a temporary Tailscale-restricted rule and remove it after the session. Do not allow DVWA from anywhere.

Verify the container is running and accessible:

```
docker ps  
curl http://localhost:8080
```

The localhost test confirms that DVWA is reachable from the Dell server itself. To confirm access from the Acer workstation, open the following URL in the Acer browser:

```
http://<YOUR-SERVER-IP>:8080
```

Replace <YOUR-SERVER-IP> with the server's sanitized LAN address.

Important

Use `http://localhost:8080`, not `https`. DVWA in this configuration serves plain HTTP on port 8080. Using HTTPS will produce an SSL handshake error because no TLS certificate is configured.

Initial Database Setup

The first time DVWA loads in a browser, it will show a setup page rather than the login screen. This is expected. DVWA needs to initialise its MySQL database before it can function. Click the 'Create / Reset Database' button on this page. After the database is created, DVWA will redirect to the login page automatically.

Default login credentials:

```
Username: admin
Password: password
```

Setting the Security Level

After logging in, navigate to the DVWA Security page from the left sidebar. Set the Security Level dropdown to Low and click Submit.

The security level controls how well DVWA defends itself against attacks. At Low, the application has essentially no input validation or sanitisation, making vulnerabilities easy to trigger and clearly observable in logs. This is the correct setting for the initial learning phase. As skills develop, the security level can be increased to Medium and then High to practise defeating progressively stronger defences.

11.4 DVWA Management

DVWA is not configured to start automatically on boot. It must be started manually at the beginning of each practice session and stopped when work is done. This is a deliberate security decision: DVWA is an intentionally vulnerable application with no business being always-on. Keeping it running continuously increases attack surface unnecessarily.

```
docker start dvwa      # Start for a session
docker stop dvwa       # Stop when done
docker rm -f dvwa      # Remove container entirely (data is lost)
docker logs dvwa       # View container log output
```

11.5 Troubleshooting: DVWA Deployment

HTTPS Error When Testing DVWA Locally

When testing DVWA from the Dell command line, the curl command was accidentally run with https:// instead of http://, producing an SSL handshake error.

Important

Root cause: DVWA in this configuration serves plain HTTP. There is no TLS certificate configured. Using https:// forces a TLS handshake the server cannot complete.

Resolution: Use http://localhost:8080 for local testing on the Dell. Adding HTTPS to DVWA in this homelab has no value. It is a local-only vulnerable target and TLS would add complexity with no security or educational benefit.

DVWA Shows Database Setup Page on First Load

After navigating to the correct IP and port in the Acer browser, DVWA shows a database setup page rather than the login screen.

Note

This is expected behaviour, not an error. DVWA requires its MySQL database to be initialised before it can function. Click the 'Create / Reset Database' button on the setup page. DVWA will redirect to the login page automatically once the database is ready.

No DVWA Logs Appearing in Grafana

Querying {job="docker", container="dvwa"} in Grafana returns no results even though DVWA is running and accessible.

Possible causes:

- Alloy does not have the Docker socket mounted.
- The Alloy Docker discovery or Docker source configuration is incorrect.
- The container label used in the query does not match the actual container name.
- DVWA is running but not emitting logs during the selected time range.
- The Grafana time range does not include recent DVWA activity.
- Alloy was not restarted after a configuration change.

Check whether DVWA is running:

```
docker ps
```

Check whether DVWA is producing Docker logs:

```
docker logs dvwa --tail 50
```

Check Alloy logs:

```
docker logs alloy --tail 50
```

If the Alloy configuration was recently changed, restart Alloy:

`docker restart alloy`

Important: Connecting DVWA to labnet is not the main fix for missing Docker logs. Alloy collects Docker logs through the Docker socket, not through the Docker network. The labnet network is useful for container-to-container communication, but Docker log collection depends on the Docker socket mount and Alloy configuration.

i Note

Root cause: Multiple causes are possible. Alloy may not have the Docker socket mounted, Docker discovery may be misconfigured, the query may be using the wrong container label, DVWA may not have produced logs during the selected time range, or Alloy may need to be restarted after a configuration change. Connecting DVWA to labnet is not the main fix for missing Docker logs because Alloy collects Docker logs through the Docker socket, not through the Docker network.

12. DVWA Detection Dashboard

With the logging stack confirmed working and logs flowing from DVWA, the next step was building a structured SOC-style detection dashboard. The goal was not to build a polished product for its own sake, but to create a set of meaningful panels that reflect real detection logic: panels that change and respond when attacks actually happen.

This section documents every panel in the dashboard, the reasoning behind each LogQL query, the design decisions made along the way, and important corrections discovered during testing. In particular, the URL encoding problem discovered during XSS detection is documented in full because it represents a class of detection failure that is not obvious until you compare what was submitted with what actually appears in the log.

12.1 Dashboard Purpose

The DVWA Detection Dashboard provides visibility into attack activity against the DVWA container. It is separate from the Homelab Monitoring Dashboard, which monitors pipeline health. This dashboard is purely about threat detection: is someone brute-forcing the login? Are SQL injection payloads appearing in request logs? Is XSS content being submitted? Each section of the dashboard is designed to answer one of these questions with both a trend view and a raw evidence view.

12.2 Data Source

All detection panels draw primarily from the DVWA container log stream. The auth log stream is also used in the behaviour section for host-level correlation.

Loki Stream Label	What It Contains
<code>{job="docker", container="dvwa"}</code>	Primary detection source. DVWA's Apache access logs show HTTP request metadata, URL paths, query strings, response codes, and many visible attack indicators. Payloads submitted through URL parameters are visible in standard access logs. Payloads submitted in POST request bodies may not appear unless the application or web server is specifically configured to log them.
<code>{job="docker"}</code>	Combined log output from all running Docker containers. Useful for cross-container correlation.
<code>{job="syslog"}</code>	General Ubuntu host system events.
<code>{job="authlog"}</code>	Host-level authentication events. SSH login attempts, sudo usage, PAM authentication results.

12.3 Design Philosophy

The core principle behind this dashboard is that detection logic should be based on observable behaviour rather than on known path names or labels. DVWA's vulnerable modules live at predictable URLs such as `/vulnerabilities/brute/` and `/vulnerabilities/sqli/`, but relying on those path names in a detection query is a shortcut that would not generalise to any real system. A real web server under attack does not label incoming requests with the vulnerability type they are exploiting.

Where possible, the queries look for behavioural signals instead. A brute force attack is detected by the presence of login parameters in repeated requests from the same source IP over a short time window, not by the URL path. SQL injection is detected by the presence of injection-style syntax in the request, not by the route name. This distinction matters both for learning and for how the dashboard would translate to a real environment.

12.4 Dashboard Variable

A Grafana dashboard variable named 'job' was created to allow quick exploration of different log streams from the raw log panels without modifying individual queries. The variable is configured with Name: job, Type: Query, Data source: Loki, and Query: label_values(job). This populates a dropdown at the top of the dashboard with available job label values from Loki.

An important design decision: the variable is not wired into the detection panels. The detection panels use fixed, specific queries. Changing the job variable does not alter what a brute force or SQLi panel shows. The variable exists purely as an exploration tool for the raw log panels at the end of the dashboard.

Tip

Wiring a dashboard variable into all panels would turn the detection dashboard into a generic log browser. The value of a detection dashboard comes from its fixed, opinionated queries that reflect specific threat patterns. Keep fixed panels fixed and reserve variables for the exploratory raw log panels only.

12.5 Row 1: Overview Panels

The overview row provides a high-level picture of DVWA traffic volume and activity. These panels give an immediate visual answer to the question of whether anything notable is happening, before drilling into the specific detection rows below.

Panel: Total Requests per Minute

This stat panel shows a single count of how many log lines DVWA generated in the last one-minute window. It functions as a pulse check: at rest the number is low, and it rises sharply when any tool is actively generating traffic against DVWA. The Stat panel renders a single large number, which makes the current activity level immediately legible without needing to read a time series graph.

```
sum(count_over_time({job="docker", container="dvwa"}[1m]))
```

Visualisation type: Stat.

Panel: Request Rate Over Time

Where the stat panel shows the current moment, this panel shows the trend. A flat line indicates idle traffic. A spike indicates a burst of requests, which in the context of this lab most likely means a scan, brute force, or exploit attempt is running. This panel is most useful when left open during an active attack session, where the spike pattern of different attack tools becomes visually distinct.

```
sum(rate({job="docker", container="dvwa"}[1m]))
```

Visualisation type: Time series.

Panel: Top Attacker IPs

This panel extracts the source IP from each DVWA log line and ranks the top five most active addresses over the last five minutes. DVWA's Apache server logs requests in standard Combined Log Format. A typical log line looks like this:

```
<YOUR-SERVER-IP> - - [DD/Mon/YYYY:HH:MM:SS +0000] "GET  
/vulnerabilities/brute/?username=admin&password=test HTTP/1.1" 200 4969
```

The pattern directive in LogQL is used to extract the IP from the beginning of the line:

```
topk(  
  5,  
  sum by (ip) (  
    count_over_time(  
      {job="docker", container="dvwa"}  
      | pattern "<ip> - - <_>"  
      [5m]  
    )  
  )  
)
```

The pattern '<ip> - - <_>' captures everything before the first space-dash-dash-space sequence as a label named 'ip' and ignores the rest of the line. The topk(5, ...) wrapper returns only the five IP addresses with the highest request counts. This approach reveals source behaviour regardless of which DVWA module is being targeted.

Visualisation type: Table.

Panel: DVWA Module Activity Indicator

The attack score panel provides a single cumulative count of activity matching DVWA's main vulnerable module paths over the last hour. It counts log lines containing the terms brute, sqli, and xss_r, which appear in the URL path when DVWA's built-in modules are accessed.

```
sum(  
  count_over_time(  
    {job="docker", container="dvwa"} |~ "brute|sqli|xss_r"  
    [1h]  
  )  
)
```

Visualisation type: Stat.

i Note

This panel is intentionally DVWA-specific and that is a known limitation. It works here because DVWA's module paths are predictable. It is a lab-context indicator rather than a universal detection method. The brute force and injection panels in the rows below use more transferable detection logic.

12.6 Row 2: Brute Force Detection

The brute force detection row is built around the observation that login attempts against DVWA send the username and password parameters as URL query string values. Every login attempt, whether successful or failed, produces a request containing 'username=' in the URL. By counting these requests and grouping them by source IP, the dashboard surfaces the pattern that defines brute force: many attempts from the same source in a short time window.

Panel: Login Attempts Over Time

This panel counts all requests containing the string 'username=' within one-minute windows and plots the result as a time series. The count rises sharply when a brute force tool or script is running and drops to near-zero when it stops. Automated tools tend to produce a perfectly regular rate, while manual testing produces more irregular, lower-amplitude patterns.

```
sum(  
  count_over_time(  
    {job="docker", container="dvwa"} |= "username="  
    [1m]  
  )  
)
```

Visualisation type: Time series.

Panel: Login Attempts by IP

This panel adds attribution to the previous one. It applies the same login attempt filter but also extracts the source IP from each log line using the pattern directive, then groups the count by IP. The result shows not just that login attempts are happening but which specific address is responsible. When the Kali VM runs a brute force tool against DVWA, this panel will show the Kali IP producing a sustained count while the rest of the network remains at zero.

```
sum by (ip) (  
  count_over_time(  
    {job="docker", container="dvwa"}  
    | pattern "<ip> - - <_>"  
    |= "username="  
    [1m]  
  )  
)
```

```
)
```

Visualisation type: Time series.

Panel: Login Attempt Evidence

This panel shows the raw log lines for every request containing 'username='. It is the evidence layer: after the detection panels above signal that brute force activity is occurring, this panel allows reading the actual request lines to see the exact usernames and passwords being attempted.

```
{job="docker", container="dvwa"} |= "username="
```

Visualisation type: Logs.

12.7 Row 3: SQL Injection Detection

SQL injection attacks work by inserting SQL syntax into input fields that are then incorporated into a database query without proper sanitisation. In DVWA's logs, these payloads appear in the URL query string, making them detectable through regex matching on known SQL syntax patterns.

Panel: SQL Injection Attempts Over Time

This panel uses a case-insensitive regular expression to count requests containing common SQL injection indicators within one-minute windows. The patterns targeted cover the most commonly used SQLi primitives: a single quote (the most fundamental SQLi delimiter), the OR keyword (used to create always-true conditions), UNION and SELECT (used in union-based extraction attacks), double-dash (an SQL comment delimiter used to truncate queries), and sleep() (used in time-based blind SQLi).

```
sum(
  count_over_time(
    {job="docker", container="dvwa"}
    |~ "(?i)(union|select|\\bor\\b|'|--|sleep\\()"
```

Visualisation type: Time series.

i Note

This panel may produce occasional false positives because some Apache internal log messages also contain SQL-adjacent keywords. This is expected behaviour for signature-based detection. False positives in a real SOC are investigated, not dismissed. The raw evidence panel below allows reviewing the matching lines to determine whether they represent genuine attempts or benign log noise.

Panel: SQLi by IP

This panel adds source attribution to the SQLi detection by combining the regex filter with pattern-based IP extraction. The result is grouped by IP so that sustained injection activity from a single address is clearly visible.

```
sum by (ip) (
  count_over_time(
    {job="docker", container="dvwa"}
    | pattern "<ip> - - <_>"
    | ~ "(?i)(union|select|\\bor\\b|'|--|sleep\\()\""
    [5m]
  )
)
```

Visualisation type: Table.

Panel: SQLi Raw Evidence

This panel shows every log line matching the SQLi regex without aggregation. Reading these lines reveals the exact payload that was submitted, the URL path it was sent to, and the HTTP response code.

```
{job="docker", container="dvwa"} |~ "(?i)(union|select|\\bor\\b|'|--|sleep\\()"
```

Visualisation type: Logs.

12.8 Row 4: XSS Detection

Cross-site scripting attacks work by injecting HTML or JavaScript into a page that is then rendered by a victim's browser. Payloads submitted through URL parameters are visible in Apache access logs. Payloads submitted through POST request bodies may not appear in standard Apache access logs unless additional request-body logging is configured. For this dashboard, the strongest XSS detections come from payloads that appear in the URL, query string, or visible request metadata. When these requests hit Apache, they are logged. However, a critical discovery was made during testing: the payloads do not appear in logs as readable text. They appear URL-encoded.

The URL Encoding Problem

When a browser submits a form field or URL parameter containing special characters, it encodes them using percent-encoding. The less-than sign < becomes %3C, the greater-than sign > becomes %3E, the opening parenthesis (becomes %28, and the equals sign = becomes %3D. This means a payload like <script>alert(1)</script> appears in the Apache log as %3Cscript%3Ealert%281%29%3C%2Fscript%3E.

The initial XSS panel queries were written to match the literal character sequences <script>, onerror=, and alert(. None of these appear in the log file in that form. The panels returned no data

not because XSS was not being detected, but because the regex patterns were looking for the wrong string. Once the correct URL-encoded equivalents were identified, the panels began working correctly.

⚠ Important

This discovery is a genuine and transferable lesson. In many server-side HTTP logs, XSS payloads and other special-character injection strings may appear percent-encoded rather than as readable HTML. Detection rules should account for encoded payload forms, especially when testing URL parameters or form submissions that are encoded by the browser. Always target the encoded form in server-side log detection.

Panel: XSS Attempts Over Time

The corrected query targets the URL-encoded equivalents of common XSS tags and event handlers. %3Cscript covers the opening script tag, %3Csvg and %3Cimg and %3Cbody cover other common injection vectors, onerror and onload are event handlers that can appear without encoding in some contexts, alert%28 covers the alert function with URL-encoded parenthesis, and javascript%3A covers the javascript: URI scheme.

```
sum(
  count_over_time(
    {job="docker", container="dvwa"}
    |~ "(?i)(%3Cscript|%3Csvg|%3Cimg|%3Cbody|onerror|onload|alert%28|javascript%3A)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: XSS by IP

This panel combines the URL-encoded XSS regex with pattern-based IP extraction to identify which source address is submitting XSS payloads.

```
sum by (ip) (
  count_over_time(
    {job="docker", container="dvwa"}
    | pattern "<ip> - - <_>"
    |~ "(?i)(%3Cscript|%3Csvg|%3Cimg|%3Cbody|onerror|onload|alert%28|javascript%3A)"
    [5m]
  )
)
```

Visualisation type: Table.

Panel: XSS Raw Evidence

This panel shows the raw log lines matching the XSS patterns. The URL-encoded payloads are visible in the log lines exactly as Apache recorded them, which is useful for understanding how XSS attacks appear in server-side logs rather than in a browser's network inspector.

```
{job="docker", container="dvwa"} |~ "(?i)(%3Cscript|%3Csvg|%3Cimg|onerror|onload)"
```

Visualisation type: Logs.

12.9 Row 5: Behaviour Panels

The behaviour row captures broader activity patterns that are not specific to a single attack type. These panels are useful for correlating DVWA attack traffic with host-level events and for detecting anomalies that do not fit neatly into one category.

Panel: Request Burst

This panel counts DVWA log lines in 30-second windows and plots the result as a time series. A 30-second window is short enough that automated tools (scanners, brute force scripts, exploit frameworks) produce a clearly visible spike that stands out from normal browsing traffic. Manual interaction with DVWA produces a low, irregular count, while automated tools produce a sustained elevated plateau.

```
sum(count_over_time({job="docker", container="dvwa"}[30s]))
```

Visualisation type: Time series.

Panel: Sensitive Endpoint Activity

This panel counts requests to URLs containing terms associated with security-relevant routes in DVWA: login, admin, password, setup, instructions, and vulnerabilities. It provides a broader view of which categories of endpoint are receiving attention, useful for identifying reconnaissance activity where an attacker explores available routes before committing to a specific attack.

```
sum(
  count_over_time(
    {job="docker", container="dvwa"}
    |~ "(?i)(login|admin|password|setup|instructions|vulnerabilities)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: Host Authentication Activity

This panel shifts context from the DVWA application layer to the underlying host. It counts auth.log entries within one-minute windows, giving visibility into SSH login attempts, sudo usage, and other host-level authentication events during the same period that DVWA attacks are occurring. This is useful for correlation: if an attack against DVWA coincides with unexpected host authentication activity, that warrants investigation.

```
sum(count_over_time({job="authlog"}[1m]))
```

Visualisation type: Time series.

Panel: DVWA Raw Log Stream

This panel shows the live, unfiltered stream of all DVWA log entries. It functions as a raw evidence feed: every HTTP request, every response code, every URL path. During active attack sessions, this panel provides the most complete picture of what is happening and is the starting point for any manual investigation that the detection panels above trigger.

```
{job="docker", container="dvwa"}
```

Visualisation type: Logs.

12.10 Complete Panel Reference

The following table summarises every panel in the DVWA Detection Dashboard for quick reference.

Row	Panel Name	Viz Type	Purpose
Overview	Total Requests per Minute	Stat	Pulse check on current DVWA traffic volume
Overview	Request Rate Over Time	Time series	Traffic trend showing spikes from tool activity
Overview	Top Attacker IPs	Table	Top 5 source IPs by request volume over 5 minutes
Overview	Attack Score	Stat	Cumulative DVWA module activity over 1 hour
Brute Force	Login Attempts Over Time	Time series	Requests containing username= per minute
Brute Force	Login Attempts by IP	Time series	Login attempt count grouped by source IP
Brute Force	Login Attempt Evidence	Logs	Raw log lines for all requests containing username=
SQL Injection	SQLi Attempts Over Time	Time series	Requests matching SQL syntax patterns per minute
SQL Injection	SQLi by IP	Table	SQL injection pattern matches grouped by source IP
SQL Injection	SQLi Raw Evidence	Logs	Raw log lines matching SQLi regex patterns

Row	Panel Name	Viz Type	Purpose
XSS	XSS Attempts Over Time	Time series	Requests matching URL-encoded XSS patterns per minute
XSS	XSS by IP	Table	XSS pattern matches grouped by source IP
XSS	XSS Raw Evidence	Logs	Raw log lines matching URL-encoded XSS patterns
Behaviour	Request Burst	Time series	DVWA request count in 30-second windows
Behaviour	Sensitive Endpoint Activity	Time series	Requests to login, admin, password, vulnerability routes
Behaviour	Host Auth Activity	Time series	Auth.log entries per minute for host-level correlation
Behaviour	DVWA Raw Stream	Logs	Unfiltered live DVWA log feed for manual investigation

12.11 Testing and Observations

Brute Force Testing

DVWA's Brute Force module was used with the security level set to Low. Repeated login attempts were submitted with varying username and password combinations. The Login Attempts Over Time panel responded immediately, showing a clear spike in the time series. The Login Attempts by IP panel correctly attributed all attempts to the single source address running the test. The raw evidence panel showed log lines in the following pattern:

```
GET /vulnerabilities/brute/?username=admin&password=test&Login=Login HTTP/1.1
GET /vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1
GET /vulnerabilities/brute/?username=root&password=123456&Login=Login HTTP/1.1
```

Each line shows the source IP, the timestamp, the full URL including the submitted credentials, and the HTTP response code. The pattern of repeated requests with incrementing password values is the textbook brute force signature.

SQL Injection Testing

SQLi payloads were submitted through DVWA's SQL Injection module. The standard test payload for DVWA's Low security level is a single quote followed by OR logic:

```
1' OR '1'='1
```

The SQLi Attempts Over Time panel detected these submissions through the single-quote and OR pattern matches. The SQLi by IP panel correctly attributed the activity. Some false positive matches were observed where Apache internal messages contained SQL-adjacent terms, which is expected behaviour for signature-based detection and reinforces the value of having raw evidence panels to verify whether a match represents a genuine attempt.

XSS Testing and the URL Encoding Discovery

XSS payloads were submitted through DVWA's Reflected XSS module. The initial detection panels returned no data despite confirmed submission of payloads. Investigation of the raw DVWA log stream revealed that the payloads were appearing in logs as percent-encoded strings rather than as readable HTML. A payload submitted as `<script>alert(1)</script>` appeared in the log as `%3Cscript%3Ealert%281%29%3C%2Fscript%3E`.

The detection queries were rewritten to target the URL-encoded equivalents. After the correction, XSS submissions immediately appeared in all three XSS panels.

✓ Result

The XSS URL encoding discovery is worth documenting clearly for anyone building similar detection rules. A browser always percent-encodes form submissions containing special characters before sending them to the server. The server-side log records what was received over the network: the encoded form. Detection rules must account for the encoded representation of payloads, not the readable HTML form.

12.12 Troubleshooting: DVWA Detection Dashboard

XSS Panels Return No Data Despite Active Testing

All three XSS panels consistently returned no results even when XSS payloads were being actively submitted and confirmed to be processed by DVWA.

i Note

Root cause: The regex patterns in the XSS queries targeted literal HTML characters such as `<script>`, `onerror=`, and `alert(`. These characters are percent-encoded by the browser before transmission and appear in Apache logs as `%3Cscript`, `onerror` (event handlers sometimes pass through without encoding), and `alert%28`. The literal patterns never appear in the log file.

Resolution: Replace all XSS regex patterns with their URL-encoded equivalents as documented in the panel queries above. The corrected queries produce results immediately when XSS payloads are submitted.

SQLi Panels Show Low-Level Matches on Idle System

The SQL injection panels occasionally showed low-level match counts even when no SQLi testing was in progress.

i Note

Root cause: Apache generates internal log messages for certain events (error responses, redirect chains, configuration-related messages) that contain words matching the SQLi regex, particularly `'select'` and `'or'`. These are not attack traffic but they do satisfy the pattern. This is a known characteristic of signature-based detection (false positives), not a configuration error.

Acceptable approach: treat low-level background counts as baseline noise and focus investigation on sustained elevated counts or counts coinciding with activity in other attack panels.

Attack Score Panel Not Updating

The Attack Score stat panel was not updating despite confirmed DVWA activity visible in other panels.

i Note

Root cause: An earlier version of the Attack Score query used a more complex multi-expression structure that was not executing correctly.

Resolution: The query was simplified to a single regex covering the three main DVWA module path terms: `brute`, `sqli`, and `xss_r`. The simplified query executes reliably and the panel updates correctly.

PART 7: OWASP JUICE SHOP AND CADDY

13. OWASP Juice Shop Deployment

OWASP Juice Shop is the second vulnerable web application added to this homelab. Where DVWA is a traditional PHP application with simple, clearly labelled vulnerability modules, Juice Shop is a modern Node.js and Angular web application that behaves much more like a real-world commercial platform. It communicates through a REST API, handles authentication with JWT tokens, uses JSON request and response bodies, and exposes endpoints that reflect actual application design patterns rather than guided training exercises.

Adding Juice Shop expands the lab's coverage from classic web vulnerabilities into the territory of modern API-based attacks. It also introduces a more realistic detection challenge: because Juice Shop's traffic looks like legitimate application use, distinguishing attack patterns from normal behaviour requires more thoughtful query design than DVWA's comparatively obvious exploit paths.

13.1 Why Juice Shop Was Added

DVWA served its purpose well as a starting point. Its vulnerabilities are deliberately obvious, its logs are straightforward Apache access logs, and the attack-to-detection cycle is easy to follow. However, it has limitations as a sole target. DVWA's attack paths are predictable: the vulnerability is at `/vulnerabilities/sqli/`, the brute force module is at `/vulnerabilities/brute/`, and detection logic written for DVWA's URL structure would not generalise to any other system.

Juice Shop addresses this limitation in two ways. First, it presents vulnerabilities in a context that requires actual exploitation rather than following a guided exercise. Finding an SQL injection point in Juice Shop requires enumerating the API, understanding how search requests are constructed, and identifying which parameters are insufficiently sanitised. Second, because Juice Shop is an API application, attacks appear in logs as API calls rather than URL path visits, which requires different detection logic and teaches a more transferable skill.

i Note

The goal of running two targets simultaneously is not to double the quantity of vulnerabilities available to practise. It is to practise the SOC workflow of managing visibility across multiple applications, which involves separate log sources, separate dashboards, different query patterns, and the challenge of correlating activity across systems. This mirrors what real security teams deal with when monitoring a diverse application estate.

13.2 Difference Between DVWA and Juice Shop

These two applications teach different things and require different detection approaches. Understanding the distinction before building the detection dashboard makes the LogQL design decisions easier to follow.

Aspect	DVWA	Juice Shop
Application type	Traditional PHP, page-based	Modern Node.js/Angular, API-driven
Log source	Docker container stdout (Apache CLF)	Caddy JSON access log file
Log format	Apache Combined Log Format (plain text)	Structured JSON per request
Loki label	{job="docker", container="dvwa"}	{job="juice-shop-access"}
IP extraction	pattern directive on raw text	request_remote_ip field from json
Query approach	Raw text search and regex	Structured field parsing with json
Attack visibility	URL paths and query strings	REST/API endpoints and JSON parameters

13.3 Juice Shop Container Deployment

Juice Shop is deployed as a Docker container on the labnet network, the same network used by Loki, Alloy, Grafana, DVWA, and Caddy. This shared network is what allows Caddy to forward requests to Juice Shop using its container name as the hostname. Run the following commands on the Dell Ubuntu Server over SSH from the Acer workstation. Do not run these commands inside the Kali VM or on the Windows host.

```
docker pull bkimminich/juice-shop
```

```
docker run -d \  
  --name juice-shop \  
  --network labnet \  
  -p 127.0.0.1:3001:3000 \  
  bkimminich/juice-shop
```

Breaking down each flag:

- `-d` - runs the container in detached mode so it continues running after the terminal command returns
- `--name juice-shop` - names the container 'juice-shop'. This name becomes a resolvable hostname on labnet, which is how Caddy addresses it in the reverse proxy configuration
- `--network labnet` - connects the container to the shared lab network, enabling communication with Caddy and Alloy by container name
- `-p 127.0.0.1:3001:3000` - maps port 3001 on the Dell host, loopback only, to port 3000 inside the container. Binding to 127.0.0.1 restricts direct Juice Shop access to the Dell server itself. This port is for diagnostic purposes only and is not reachable from the Acer browser over

the LAN. To verify Juice Shop is running before Caddy is deployed, use curl `http://localhost:3001` from the Dell command line.

•

```
docker ps
```

i Note

When Juice Shop starts for the first time, it performs a startup sequence that takes 10 to 20 seconds. The application is not ready until the container output includes the message 'Server listening on port 3000'. If the URL is accessed before it is ready, the browser will receive a connection refused or loading error. Wait a moment and refresh.

13.4 Important Port Decision

There are two ways to reach Juice Shop, but only one should be used from the Acer browser. Port 3001 connects directly to the Juice Shop container, but it is bound to 127.0.0.1 on the Dell and is intended only for local diagnostic checks from the Dell itself. Port 3002 connects to Caddy, which forwards the request to Juice Shop and writes the access logs used by the detection dashboard. For browser testing, attack simulation, and detection validation, use port 3002.

⚠ Important

Port 3001 maps directly to the Juice Shop container. Traffic entering through port 3001 bypasses Caddy entirely and generates no access log entries. Any attack simulation or detection testing performed through port 3001 will produce no evidence in the Juice Shop Detection Dashboard.

Port 3002 is Caddy's listening port. All traffic entering here is logged as structured JSON before being forwarded to Juice Shop. Always use port 3002 for all testing, attack simulation, and detection work.

13.5 Troubleshooting: Juice Shop Deployment

Juice Shop Not Loading on First Start

The browser returns a connection refused or blank page when accessing Juice Shop immediately after starting the container.

i Note

Root cause: Juice Shop requires 10 to 20 seconds to initialise on first start. The application is not ready to accept connections until this sequence completes.

Resolution: Wait 20 seconds after running `docker run` and then refresh the browser. Verify the container is running with `'docker ps'` and check startup progress with `'docker logs juice-shop'`.

Juice Shop Container Not Discovered by Alloy

The Juice Shop container logs are not appearing under `{job="docker", container="juice-shop"}` in Grafana. Juice Shop's Docker stdout logs are collected by Alloy for completeness, but they do not contain useful HTTP request-level access data. The Juice Shop Detection Dashboard relies on the Caddy JSON access log, not the Juice Shop container stdout stream.

Note

Root cause: The juice-shop container is not on the labnet Docker network, or Alloy was not restarted after the container was started.

Resolution: Verify with `'docker network inspect labnet'` that juice-shop appears in the container list. If not, run `'docker network connect labnet juice-shop'`. Then restart Alloy with `'docker restart alloy'` to force re-discovery.

14. Caddy Reverse Proxy for Juice Shop

14.1 The Logging Problem

Before Caddy could be set up, a fundamental problem had to be solved: Juice Shop does not produce useful HTTP access logs in its Docker container output. When Alloy collects the Docker logs from the juice-shop container, what it receives is primarily application startup and runtime messages from the Node.js process itself: the server listening confirmation, configuration validation results, environment detection, and occasional error traces.

What does not appear is a structured record of every HTTP request that was made, who made it, what URL was requested, what the server responded with, and what query parameters or payload was submitted. This is a critical gap for detection. A SOC analyst working with logs from a web application needs request-level visibility. Without it, brute force patterns cannot be detected, injection payloads in query strings cannot be identified, and IP attribution is not possible.

Tip

This is an important real-world lesson: applications do not always emit the logs you need by default. In professional environments, security teams frequently need to add logging infrastructure (reverse proxies, log shippers, application-level middleware) to gain visibility that the application itself does not provide. Adding Caddy as a logging reverse proxy is a scaled-down version of exactly this approach.

14.2 Why Caddy Was Added

The solution to the logging problem was to place a reverse proxy in front of Juice Shop. A reverse proxy sits between the client making requests and the application serving them. The client connects to the proxy, the proxy forwards the request to the application, receives the response, and returns it

to the client. From the client's perspective, it is communicating directly with Juice Shop. In practice, every request passes through Caddy first.

Caddy is a modern, lightweight web server and reverse proxy written in Go. It has first-class support for structured access logging: when configured correctly, Caddy writes a log entry for every request it processes, containing all the fields needed for detection. The source IP, the HTTP method, the full URI including query string, the status code, the response size, the user agent, and timing information are all present in each entry.

The complete data flow after adding Caddy:

Stage	What Happens
1. Browser or Kali VM	Sends HTTP request to port 3002 on the Dell server
2. Caddy receives request	Logs the full request to the access log file immediately
3. Caddy forwards request	Proxies the request to juice-shop:3000 over the labnet Docker network
4. Juice Shop responds	Processes the request and returns a response to Caddy
5. Caddy returns response	Passes the response back to the client
6. Alloy tails log file	Reads new entries from the Caddy access log and forwards them to Loki
7. Loki stores logs	Indexes and stores the JSON log entries under job='juice-shop-access'
8. Grafana queries Loki	LogQL queries retrieve and display Juice Shop traffic in the detection dashboard

14.3 Caddy Directory Structure

Before starting Caddy, the directories it needs must exist on the Dell host. Caddy writes logs to a path inside its container, and that path is mapped to a host directory through a Docker volume mount. The Caddyfile is also loaded from the host rather than baked into the container image, so it can be modified without rebuilding the container.

```
mkdir -p ~/caddy/juice-shop/logs
mkdir -p ~/caddy/juice-shop/config
```

After running these commands, the directory tree is:

```
~/caddy/
├── juice-shop/
│   ├── config/
│   │   ├── Caddyfile          # Caddy server configuration
│   └── logs/
│       ├── juice-shop-access.log  # Written by Caddy, read by Alloy
```

14.4 Final Caddyfile

The Caddyfile is Caddy's configuration file. For this setup it needs to do two things: listen on port 3002 and forward traffic to Juice Shop, and write a structured access log for every request it receives.

```
nano ~/caddy/juice-shop/config/Caddyfile
```

Paste the following configuration:

```
:3002 {  
  log {  
    output file /var/log/caddy/juice-shop-access.log  
    format json  
  }  
  reverse_proxy juice-shop:3000  
}
```

Each part of this configuration:

- `:3002` - Caddy listens on port 3002. This is the detection-ready entry point for Juice Shop. All attack testing and monitoring should use this port.
- `output file /var/log/caddy/juice-shop-access.log` - the log is written to this path inside the Caddy container. Because of the Docker volume mount defined when the container starts, this file appears on the Dell host at `~/caddy/juice-shop/logs/juice-shop-access.log`
- `format json` - every log line is written as a complete, valid JSON object. This is the critical setting that enables structured parsing in Loki with `| json`
- `reverse_proxy juice-shop:3000` - Caddy forwards every request to the juice-shop container on port 3000. The hostname 'juice-shop' resolves because both containers are on the labnet Docker network

14.5 Why JSON Format Was Required

LogQL has two fundamentally different modes for working with log content. The first is raw text matching, using the `|=` operator for substring search and `|~` for regex. The second is structured parsing using `| json`, which treats each log line as a JSON object and extracts named fields. The structured approach is far more powerful because it allows grouping, aggregating, and filtering by specific fields such as the source IP, the HTTP method, the URI, or the status code.

For `| json` to work, every log line must be a valid, complete JSON object. The first version of the Caddyfile used Caddy's default 'console' format, which produces log lines with a plain-text timestamp prefix, a severity level, a message label, and then a JSON-like object:

```
2026/05/04 11:22:01.123 INFO http.log.access.log0 handled request  
{"request":{"remote_ip":"..."}}
```

This format looks like it contains JSON, and the JSON portion is real and valid. However, the full line starts with plain text before the JSON object begins. When Loki's `| json` parser receives this

line, it attempts to parse the entire line as a JSON document. Since the line begins with '2026' rather than '{', the parser rejects it and produces an error.

```
JSONParserErr: Value looks like object, but can't find closing '}' symbol
```

⚠ Important

This error is easy to misread as a Grafana or Loki problem. It is actually a log format problem. Loki is working correctly: it correctly identifies that the log line does not conform to the JSON format it was asked to parse. The fix is entirely on the Caddy side: changing 'format console' to 'format json' makes Caddy write each log entry as a standalone, complete JSON object on a single line, which is exactly what Loki's | json parser expects.

14.6 Start Caddy Container

```
docker run -d \
  --name caddy-juice \
  --network labnet \
  -p 3002:3002 \
  -v ~/caddy/juice-shop/config/Caddyfile:/etc/caddy/Caddyfile \
  -v ~/caddy/juice-shop/logs:/var/log/caddy \
  caddy:latest
```

Breaking down each flag:

- `--name caddy-juice` - names the container to distinguish it from any other Caddy instances that might be added later
- `--network labnet` - connects Caddy to labnet so it can reach 'juice-shop:3000' by container hostname
- `-p 3002:3002` - exposes Caddy's listening port to the Dell host, making it accessible from the Acer
- `-v ~/caddy/juice-shop/config/Caddyfile:/etc/caddy/Caddyfile` - mounts the Caddyfile from the host. Caddy reads its configuration from this path on startup
- `-v ~/caddy/juice-shop/logs:/var/log/caddy` - mounts the logs directory from the host into the container path where Caddy writes its access log. This is what makes the log file readable by Alloy on the host

Caddy on port 3002 is the only supported Juice Shop testing path because it creates JSON access logs. It must still be restricted to trusted lab access only. Do not expose port 3002 to the public internet. Restrict access to the trusted LAN, Tailscale, or a controlled lab source.

```
Example restricted UFW rule:
sudo ufw allow from <YOUR-LAB-SUBNET> to any port 3002 proto tcp
If using Tailscale-only access:
sudo ufw allow in on tailscale0 to any port 3002 proto tcp
```

14.7 Clear Old Console-Format Logs

If Caddy was initially run with 'format console' before being corrected to 'format json', the access log file will contain a mixture of old console-format lines and new JSON-format lines. Any old console-format line remaining in the file will cause | json parsing errors in Grafana. The file must be cleared so that only pure JSON entries exist.

```
truncate -s 0 ~/caddy/juice-shop/logs/juice-shop-access.log
```

The truncate command empties the file's content while keeping the file itself in place. Deleting the file would cause Alloy to lose track of it, and Caddy (with its running file handle) would either stop logging or require a restart to create a new file. Truncating to zero bytes clears the content cleanly without disrupting either process.

After truncating, restart Caddy to ensure it is running with the JSON format configuration and has fresh file handles:

```
docker restart caddy-juice
```

Important

Only truncate this file when clearing old test data or fixing the console-to-JSON format transition. Truncating removes existing local access-log evidence from the file. If the log contains evidence you want to keep for documentation, copy it to a separate evidence file before truncating.

14.8 Verify Caddy Access Logs

Before connecting Alloy, confirm that Caddy is writing well-formed JSON to the access log. Tail the log file and generate some test traffic by visiting Juice Shop through Caddy in the Acer browser:

```
tail -f ~/caddy/juice-shop/logs/juice-shop-access.log
# From the Acer browser, visit: http://<YOUR-SERVER-IP>:3002
# Each page load should produce new log lines in the terminal
# Press Ctrl+C to stop tailing
```

Important

Every log line should begin with an opening curly brace '{' and end with a closing curly brace '}'. If lines begin with a date stamp and INFO prefix instead, Caddy is still using console format.

Verify the Caddyfile contains 'format json', that the file was saved correctly, and that the container was restarted after the change. Also confirm old console-format entries were cleared with truncate.

14.9 Limitation: Caddy Does Not Capture Full Request Bodies

Caddy access logs capture request metadata: the HTTP method, the full URI including query string, the status code, the source IP, the user agent, and timing information. They do not normally capture full JSON request bodies.

This means some Juice Shop payloads that appear inside POST request bodies (such as login credentials submitted as JSON, or injection payloads in API request parameters) may not appear as readable payload strings in Loki. The URI and method will be logged, but the body content will not unless application-level logging or controlled request-body logging is added. This is a known limitation of access-log-based detection and is worth understanding before building detection queries.

14.10 Troubleshooting: Caddy

JSONParserErr When Querying Juice Shop Logs

Queries using `| json` against Juice Shop logs produce the error 'JSONParserErr: Value looks like object, but can't find closing } symbol' for most or all log entries.

Important

Root cause: The Caddyfile was using 'format console' rather than 'format json'. Console-format log lines begin with a plain-text timestamp and severity prefix before the JSON payload, making the entire line non-parseable as JSON.

Resolution: Change the Caddyfile log block to 'format json', truncate the existing log file with 'truncate -s 0 ~/caddy/juice-shop/logs/juice-shop-access.log', and restart the Caddy container with 'docker restart caddy-juice'. New log entries will be valid JSON and `| json` queries will work immediately.

Caddy Cannot Reach Juice Shop - 502 Bad Gateway

Visiting `http://<YOUR-SERVER-IP>:3002` returns a 502 Bad Gateway error from Caddy.

Important

Root cause: Either the juice-shop container is not running, or Caddy and Juice Shop are not on the same Docker network. Caddy addresses Juice Shop using the hostname 'juice-shop', which only resolves if both containers are on the same user-defined Docker network.

Resolution: Verify with 'docker ps' that juice-shop is running. Then check 'docker network inspect labnet' to confirm both caddy-juice and juice-shop appear in the containers list. If either is missing, connect it with 'docker network connect labnet <container-name>'.

No Log Entries Appearing After Visiting Juice Shop Through Caddy

Tailing the access log file shows no output even though the browser successfully loads Juice Shop through port 3002.

Important

Root cause: The volume mount for the log directory is incorrect, or the log file path in the Caddyfile does not match the mount point.

Resolution: Verify that the Caddyfile specifies 'output file /var/log/caddy/juice-shop-access.log' and that the docker run command for Caddy includes '-v ~/caddy/juice-shop/logs:/var/log/caddy'. The host directory and the container path must match this mapping exactly. If both are correct, check 'docker logs caddy-juice' for any configuration errors.

15. Juice Shop Logs to Alloy and Loki

With Caddy writing structured JSON access logs to a file on the Dell host, the next step is connecting those logs to the existing Alloy-to-Loki pipeline. Alloy already collects syslog, auth.log, and Docker container logs. Adding Caddy's access log requires two changes: updating the Alloy configuration file to define a new file source, and recreating the Alloy container with an additional volume mount so it can read the Caddy log directory.

15.1 Why File-Based Collection

When Alloy collects logs from a file rather than from Docker's log streaming API, it uses a two-component pipeline. The first component is `local.file_match`, which defines which file paths to watch and assigns Loki labels to log lines coming from those paths. The second component is `loki.source.file`, which performs the actual reading, tailing the matched files and forwarding new lines to a downstream receiver.

The labels assigned in the `file_match` block become the Loki stream selectors used in Grafana queries. For Caddy's Juice Shop access log, the job label is set to 'juice-shop-access'. This is the label used in every LogQL query in the Juice Shop Detection Dashboard. The service and host labels are additional metadata that help identify where a log came from when multiple sources are flowing into the same Loki instance.

15.2 Alloy Caddy Log Source

Open the Alloy configuration file and add the Caddy log source block above the existing `loki.write` section:

```
nano ~/logging/alloy/config.alloy
```

Add the following block above the `loki.write` section:

```
// Juice Shop access logs from Caddy (JSON format)
local.file_match "juice_shop_access" {
  path_targets = [
    {
      "__path__" = "/caddy-logs/juice-shop-access.log",
      "job"      = "juice-shop-access",
      "service"  = "juice-shop",
      "host"     = "homelab",
    },
  ]
}

loki.source.file "juice_shop_access" {
  targets      = local.file_match.juice_shop_access.targets
  forward_to   = [loki.write.loki.receiver]
  tail_from_end = true
}
```

Each part of this block explained:

- "`__path__`" = `"/caddy-logs/juice-shop-access.log"` - the path where Alloy will find the log file inside the Alloy container. This path exists because the docker run command for Alloy mounts `~/caddy/juice-shop/logs` as `/caddy-logs` inside the container
- "`job`" = `"juice-shop-access"` - the Loki stream label that all queries in the detection dashboard use: `{job="juice-shop-access"}`. Setting this in the Alloy config means it is automatically attached to every log line before it reaches Loki
- "`service`" = `"juice-shop"` and "`host`" = `"homelab"` - additional labels for filtering and identification when multiple sources flow into the same Loki instance
- `tail_from_end = true` - Alloy begins reading from the current end of the file rather than from the beginning. This prevents previously written entries from being re-ingested into Loki when Alloy restarts, which is important after truncating the log file

⚠ Important

Alloy's River/HCL configuration language requires a trailing comma after the last field in a map block. The closing 'host' entry must be followed by a comma. A missing comma will cause Alloy to fail at startup with 'missing comma in field list'. This applies to all map-style blocks in the config.

15.3 Recreate Alloy With Caddy Mount

Alloy's volume mounts are defined when the container is created using docker run. There is no way to add a new volume mount to a running container. The container must be removed and recreated. This is not a destructive operation: Alloy stores no persistent state of its own. Recreating Alloy does not delete logs already ingested into Loki. This assumes Loki persistence is configured correctly and the logs have not expired under the configured retention policy.

```
docker rm -f alloy
```

Recreate Alloy with the additional Caddy log mount included:

```
docker run -d --name alloy \  
  --network labnet \  
  --restart unless-stopped \  
  --user root \  
  -p 127.0.0.1:12345:12345 \  
  -v ~/logging/alloy/config.alloy:/etc/alloy/config.alloy \  
  -v /var/log:/var/log:ro \  
  -v /var/run/docker.sock:/var/run/docker.sock \  
  -v ~/caddy/juice-shop/logs:/caddy-logs:ro \  
  grafana/alloy:latest \  
  run /etc/alloy/config.alloy \  
  --server.http.listen-addr=0.0.0.0:12345
```

The new mount added compared to the base Alloy configuration is:

```
-v ~/caddy/juice-shop/logs:/caddy-logs:ro
```

This maps the host directory `~/caddy/juice-shop/logs` (where Caddy writes its access log) to `/caddy-logs` inside the Alloy container (where the Alloy config file's `__path__` entry points). The `:ro` suffix makes the mount read-only.

⚠ Important

The path inside the container (`/caddy-logs`) and the path in the Alloy config file (`__path__ = "/caddy-logs/juice-shop-access.log"`) must match exactly. If the mount destination differs from the config path, Alloy will report that the target file does not exist and no logs will be collected.

15.4 Verify Pipeline

Step 1: Verify Alloy Started Without Errors

```
docker logs alloy --tail 50
```

Check that the output does not contain: a 'missing comma in field list' error (Alloy config syntax problem), a 'connection refused' error pointing to `loki:3100` (Alloy cannot reach Loki), or a 'no such file or directory' error for the `caddy-logs` path (volume mount is wrong).

Step 2: Generate Test Traffic Through Caddy

From the Acer browser, visit Juice Shop through Caddy at `http://<YOUR-SERVER-IP>:3002`. Navigate through a few pages, perform a search, or attempt a login. Each action should produce entries in the Caddy access log.

Step 3: Verify Logs Appear in Grafana

In Grafana, go to Explore and run the following query:

```
{job="juice-shop-access"}
```

If log entries appear, the pipeline is working: Caddy is writing logs, Alloy is reading and forwarding them, Loki is storing them, and Grafana can retrieve them.

Step 4: Verify JSON Parsing Works

Run the following query to confirm that log lines are valid JSON and Loki can parse them into structured fields:

```
{job="juice-shop-access"} | json | __error__=""
```

✓ Result

The `|__error__=""` filter discards any log lines where the JSON parser encountered an error. If this query returns the same number of results as the raw query above, all lines are valid JSON and the pipeline is fully functional. If significantly fewer results appear, some log lines are still in non-JSON format, likely from before the log file was truncated.

Confirm All Running Containers

```
docker ps
# Expected containers and their relevant ports:
# caddy-juice    - port 3002 (Juice Shop via Caddy, for detection)
# juice-shop    - port 3001 (Juice Shop direct, diagnostic only)
# grafana       - port 3000 (Dashboard UI)
# loki          - port 3100 (Log storage)
# alloy         - port 12345 (Alloy UI and metrics)
# dvwa         - port 8080 (DVWA vulnerable app)
```

15.5 Troubleshooting: Juice Shop Log Pipeline

Alloy Shows No Such File or Directory for caddy-logs

Alloy logs show an error that `/caddy-logs/juice-shop-access.log` does not exist.

⚠ Important

Root cause: The volume mount in the docker run command does not match the path defined in the Alloy config file, or the mount was not included when recreating the Alloy container.
Resolution: Verify the docker run command includes `-v ~/caddy/juice-shop/logs:/caddy-logs:ro` and that the Alloy config file specifies `'__path__ = "/caddy-logs/juice-shop-access.log"'`. The container destination (`/caddy-logs`) and the config path (`/caddy-logs/...`) must match exactly.

Alloy Config Fails to Start with 'missing comma in field list'

The Alloy container exits immediately after starting and `'docker logs alloy'` shows a 'missing comma in field list' parse error.

⚠ Important

Root cause: A map-style block in the Alloy config is missing the required trailing comma after the last key-value pair.
Resolution: Open `~/logging/alloy/config.alloy` and check the `path_targets` block for the `juice_shop_access` file match. The last entry (such as `'host = "homelab"'`) must be followed by a comma. Save the file and recreate the Alloy container.

16. Juice Shop Detection Dashboard

With the Caddy-to-Loki pipeline confirmed working, the final phase of this Juice Shop setup is the detection dashboard in Grafana. This dashboard is entirely separate from the DVWA Detection Dashboard created earlier. The separation is deliberate: different applications produce different log formats, require different detection logic, and benefit from different dashboard layouts. Mixing them would create noise and make both less useful.

The Juice Shop dashboard uses Caddy's structured JSON access logs as its sole data source. Because these logs are valid JSON, all queries use Loki's `| json` parser to extract structured fields rather than relying on raw text pattern matching. This produces cleaner detection logic and enables richer queries. For example, grouping results by the parsed `request_remote_ip` field is a direct field lookup, whereas the DVWA dashboard had to extract IP addresses from raw text using the pattern directive.

16.1 Dashboard Purpose

This dashboard provides visibility into attack and abuse activity against Juice Shop at the HTTP request level. Because Caddy logs every request that passes through it, the dashboard can detect brute force patterns against the login endpoint, SQL injection and XSS indicators visible in logged request URIs, query strings, endpoint paths, and HTTP metadata. Payloads sent only inside POST JSON bodies are outside the visibility of this Caddy access-log setup unless request-body logging is separately implemented and validated.

16.2 How It Differs From the DVWA Dashboard

The DVWA dashboard operates on Apache-style access logs collected via Docker container output. IP extraction uses LogQL's pattern directive to parse the Combined Log Format text. Detection queries use raw text search and regex because the logs are unstructured strings.

The Juice Shop dashboard operates on Caddy's JSON access logs collected via file source. IP extraction uses the parsed `request_remote_ip` field extracted by `| json`. Detection queries use the `| json` parser to access structured fields, producing more precise filtering and grouping. Almost every query in this dashboard includes `| json | __error__=""` as a base, which ensures only cleanly parsed log lines flow through to the aggregation logic.

16.3 Performance Note

The Dell N4030 has limited CPU performance and an HDD rather than an SSD. Running multiple complex Loki queries simultaneously (especially queries combining JSON parsing, regex matching, aggregation, and IP grouping over wide time ranges) can cause Loki to fall behind on query processing. When this happens, Grafana displays the error 'too many outstanding requests'.

Tip

If Grafana shows 'too many outstanding requests', reduce the dashboard time range first. This is the most effective single change because it dramatically reduces the amount of log data Loki must scan per query.

Use time ranges of 'Last 5 minutes' or 'Last 15 minutes' while building and testing panels. Set dashboard auto-refresh to 30 seconds or 1 minute. Build and test one row at a time. Once the dashboard is complete and queries are confirmed working, wider time ranges can be used for investigation sessions.

16.4 Data Source and Base Query

Every panel on this dashboard draws from the same Loki stream. Before building any panel, verify that these base queries return results in Grafana's Explore view:

```
{job="juice-shop-access"} # Raw log lines
{job="juice-shop-access"} | json | __error__="" # Structured parsed lines
```

The second query is the foundation of almost every detection panel. The `| json` directive instructs Loki to parse each log line as a JSON document and expose its fields. The `| __error__=""` filter discards any lines where parsing fails, ensuring only clean, parseable entries flow through to the aggregation and detection logic.

16.5 Row 1: Overview

Panel: Total Requests

This stat panel shows a single count of log entries in the last one-minute window. At rest with no active testing, the count reflects background requests Juice Shop makes to itself (health checks, asset loading). During an active attack session, the count rises significantly. The panel functions as a pulse check: a high number warrants investigating the other panels.

```
sum(rate({job="juice-shop-access"}[1m]))
```

Visualisation type: Stat.

Panel: Request Rate Over Time

This panel plots request volume per minute as a time series, showing traffic trends rather than a single instantaneous count. Flat baselines broken by spikes are the expected pattern during attack sessions. Brute force tools produce a sustained elevated plateau, scanners produce sharp initial spikes that taper off, and manual testing produces irregular lower-amplitude activity. This query uses `count_over_time` rather than `rate()` to reduce computational load on the Dell's hardware.

```
sum(count_over_time({job="juice-shop-access"}[1m]))
```

Visualisation type: Time series.

Panel: Top Client IPs

This panel extracts the source IP from each log entry using the parsed `request_remote_ip` field and ranks the top five most active addresses over the last five minutes. Because the logs are structured JSON, there is no need to use the pattern directive for IP extraction: the IP is directly available as a named field after `| json` parsing.

```
topk(5,
  sum by (request_remote_ip) (
    count_over_time(
      {job="juice-shop-access"}
      | json
      | __error__=""
      [5m]
    )
  )
)
```

Visualisation type: Table.

Panel: Attack Score

The attack score panel aggregates activity matching a broad set of suspicious terms and encoded payload patterns across a five-minute window. It is a high-level triage indicator, not a precision detection signal. It combines login endpoint access, SQL injection syntax, XSS payload patterns, and sensitive endpoint names into a single count. A non-zero value indicates some combination of the targeted patterns is present in recent traffic.

```
sum(
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | ~
    "(?i)(login|union|select|%27|%22|%3Cscript|onerror|alert%28|javascript%3A|admin|ftp|debug)"
    [5m]
  )
)
```

Visualisation type: Stat.

16.6 Row 2: Login Abuse and Brute Force

Juice Shop's authentication endpoint is at `/rest/user/login`. Every login attempt, whether successful or failed, sends a POST request to this path. Brute force detection is based on the same principle as the DVWA dashboard: the behavioural pattern of many login requests from the same source IP in a short time window is the indicator.

Panel: Login Attempts Over Time

This panel counts requests matching the Juice Shop login endpoint pattern within one-minute windows. It detects both the `/rest/user/login` API path and `/login` as a fallback. A brute force attempt produces a sustained count that stands clearly above the background level of normal user activity.

```
sum(
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | request_uri =~ "(?i).*/rest/user/login|/login).*"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: Login Attempts by IP

This panel adds attribution by grouping the login attempt count by source IP. When a brute force tool is running from the Kali VM, this panel will show the Kali IP producing a sustained count while all other IPs remain at or near zero. This is the pattern that characterises brute force: single-source, sustained, high-frequency requests to an authentication endpoint.

```
sum by (request_remote_ip) (
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | ~ "(?i)(/rest/user/login|/login)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: Login Evidence

The raw evidence panel shows the actual log lines for every request matching the login endpoint filter. From these lines, the analyst can read the source IP, the full request URI, the HTTP method, the response status code, and the timestamp, providing everything needed to characterise the activity.

```
{job="juice-shop-access"} | json | __error__="" | ~ "(?i)(/rest/user/login|/login)"
```

Visualisation type: Logs.

16.7 Row 3: Injection Detection

SQL injection and XSS detection for Juice Shop follow the same conceptual approach as the DVWA dashboard: look for known injection syntax in request logs. Because Juice Shop is an API application, injection payloads often appear in URL-encoded form within query strings and API

parameters. The log lines are structured JSON, so the queries use | json rather than pattern-based extraction.

SQL Injection Detection

SQL injection payloads in Juice Shop traffic are detected by the presence of common SQL syntax in URL-encoded form. The patterns targeted include encoded single and double quotes (%27 and %22), SQL keywords (union, select), time-based injection functions (sleep()), and the tautology-based injection pattern or 1=1 in its URL-encoded variants.

Panel: SQLi Attempts Over Time

```
sum(
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | ~ "(?i)(%27|%22|union|select|sleep\\(|or%201%3D1|or\\+1\\=1)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: SQLi by IP

```
sum by (request_remote_ip) (
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | ~ "(?i)(%27|%22|union|select|sleep\\(|or%201%3D1|or\\+1\\=1)"
    [5m]
  )
)
```

Visualisation type: Table.

XSS Detection

XSS payload detection follows the same URL-encoding principle established during the DVWA dashboard work. Payloads submitted through the browser are percent-encoded before transmission. The detection targets: %3Cscript for the script tag, %3Cimg and %3Csvg for alternative injection vectors, onerror and onload as event handlers, alert%28 for the alert function, and javascript%3A for the JavaScript URI scheme.

Panel: XSS Attempts Over Time

```
sum(
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | ~ "(?i)(%3Cscript|%3Cimg|%3Csvg|onerror|onload|alert%28|javascript%3A)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: XSS by IP

```
sum by (request_remote_ip) (
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | ~ "(?i)(%3Cscript|%3Cimg|%3Csvg|onerror|onload|alert%28|javascript%3A)"
    [5m]
  )
)
```

Visualisation type: Table.

Combined Injection Evidence Panel

A single evidence panel covers both SQLi and XSS indicators, combining both regex patterns into one query. This gives a unified view of all injection-type activity for manual review without needing to switch between two separate log panels.

```
{job="juice-shop-access"} | json | __error__=""
| ~
"(?i)(%27|%22|union|select|sleep\\(|%3Cscript|%3Cimg|%3Csvg|onerror|alert%28|javascri
pt%3A)"
```

Visualisation type: Logs.

16.8 Row 4: API Abuse and Behaviour

Juice Shop is fundamentally an API-driven application. Most of its functionality is exposed through REST endpoints under `/rest/` and `/api/` paths. An attacker targeting Juice Shop will interact heavily with these endpoints, attempting to enumerate available resources, probe for missing authentication, or abuse endpoints that lack rate limiting.

Panel: API Request Burst

This panel counts requests to the `/rest/` and `/api/` endpoint prefixes within 30-second windows. A 30-second window is short enough to surface automated behaviour clearly: tools generate API requests at a rate that produces a clearly elevated count compared to the low background of normal browsing. Manual exploration produces a low, irregular count; automated tools produce a sustained plateau.

```
sum(
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | ~ "(?i)(/rest/|/api/)"
    [30s]
  )
)
```

Visualisation type: Time series.

Panel: Sensitive Endpoint Access

This panel tracks requests to endpoint names associated with administrative functions, user data, and application internals. The keywords targeted (admin, account, basket, user, payment, address, ftp, debug, config) represent paths an attacker would specifically seek out during reconnaissance. Elevated counts, particularly from a single IP, indicate targeted probing rather than incidental browsing.

```
sum(
  count_over_time(
    {job="juice-shop-access"}
    | json
    | __error__=""
    | request_uri =~
    "(?i).*(admin|account|basket|user|payment|address|ftp|debug|config).*"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: HTTP Status Code Breakdown

This panel groups requests by HTTP status code and shows the count for each over time. Status code patterns reveal specific attack behaviours. A high volume of 401 responses indicates failed authentication, correlating with brute force. A high volume of 404 responses indicates path enumeration or scanning for non-existent endpoints. A spike in 500 responses can indicate that injection payloads are causing server-side errors, which is itself a detection signal.

```
sum by (status) (  
  count_over_time(  
    {job="juice-shop-access"}  
    | json  
    | __error__=""  
    [1m]  
  )  
)
```

Visualisation type: Time series. Each status code appears as a separate coloured series on the graph.

16.9 Row 5: Investigation

The investigation row provides raw log access for manual analysis. When the detection panels above flag suspicious activity, the investigation panels are where the analyst reads the actual evidence.

Panel: Full Raw Log Stream

This panel shows all Juice Shop access log entries without any filtering or parsing. It is the starting point for investigation and the fallback when parsed queries produce unexpected results. Raw lines here can be compared against parsed results to diagnose format issues.

```
{job="juice-shop-access"}
```

Visualisation type: Logs.

Panel: Parsed Clean Logs

This panel shows only log entries that parse successfully as JSON and contain no parser errors. This is the cleanest view of structured log data and the most useful for reading individual request details, with all fields visible and legible as a full JSON object per request.

```
{job="juice-shop-access"} | json | __error__=""
```

Visualisation type: Logs.

Panel: Suspicious Activity Logs

This panel combines all detection signal patterns into a single filter and shows the matching raw log entries. It is the equivalent of a SIEM alert triage queue: all lines that triggered at least one detection signature, in chronological order for analyst review. Reading these lines after an attack session provides the full picture of what was submitted, when, and from where.

```
{job="juice-shop-access"}
| json
| __error__=""
| request_uri =~
"(\?i).*(admin|union|select|%3Cscript|%3Cimg|%3Csvg|onerror|debug|ftp).*"

```

Visualisation type: Logs.

16.10 Complete Panel Reference

Row	Panel Name	Viz Type	Purpose
Overview	Total Requests	Stat	Current request volume in the last minute
Overview	Request Rate Over Time	Time series	Traffic trend showing spikes from tool activity
Overview	Top Client IPs	Table	Top 5 most active source IPs over 5 minutes
Overview	Attack Score	Stat	Combined suspicious activity indicator over 5 minutes
Login Abuse	Login Attempts Over Time	Time series	Requests to /rest/user/login per minute
Login Abuse	Login Attempts by IP	Time series	Login attempt count grouped by source IP
Login Abuse	Login Evidence	Logs	Raw log lines for all login endpoint requests
Injection	SQLi Attempts Over Time	Time series	Requests matching SQL injection syntax patterns
Injection	SQLi by IP	Table	SQL injection matches grouped by source IP
Injection	XSS Attempts Over Time	Time series	Requests matching URL-encoded XSS patterns
Injection	XSS by IP	Table	XSS matches grouped by source IP
Injection	Injection Evidence	Logs	Combined raw evidence for all injection indicators
API Abuse	API Request Burst	Time series	/rest/ and /api/ requests in 30-second windows
API Abuse	Sensitive Endpoint Access	Time series	Requests to admin, user, payment, ftp, debug routes
API Abuse	Status Code Breakdown	Time series	Request count grouped by HTTP status code
Investigation	Full Raw Log Stream	Logs	All Juice Shop access log entries unfiltered

Row	Panel Name	Viz Type	Purpose
Investigation	Parsed Clean Logs	Logs	Successfully parsed JSON log entries only
Investigation	Suspicious Activity Logs	Logs	Lines matching any detection signature pattern

16.11 Key Lessons

Applications Do Not Always Produce the Logs You Need

Juice Shop's default Docker output was insufficient for security monitoring. Adding a reverse proxy to generate access logs is a legitimate and commonly used technique in real environments. When an application does not emit the telemetry a security team needs, the correct approach is to add infrastructure that generates that telemetry from the outside: through a proxy, an agent, or a network tap, rather than modifying the application.

Log Format Determines What Detection Is Possible

The difference between Caddy's console format and JSON format was the difference between broken detection and working detection. Console-format logs could be read as raw text but could not be parsed as structured data, which limited query expressiveness significantly. JSON-format logs enable field-level extraction and grouping, producing much cleaner and more powerful detection queries. In any logging setup, prefer structured formats where possible.

Truncate Before Querying After a Format Change

When a log format changes mid-file, old entries in the non-JSON format remain and break | json queries run against the full log history. The correct remediation is to truncate the file to zero bytes so only new entries in the correct format are present. This avoids continuous JSONParserErr noise from old entries.

Different Applications Require Different Detection Logic

The DVWA dashboard used Apache log format and the pattern directive for IP extraction. The Juice Shop dashboard uses JSON log format and the parsed request_remote_ip field. These are genuinely different approaches because the underlying log structures are different. A SOC analyst working across a mixed application estate must understand the log format of each application and write appropriate detection logic for each one, rather than applying a single template.

Hardware Limits Must Shape Query Design

Running JSON parsing, regex matching, aggregation, and IP grouping simultaneously across wide time ranges on older hardware with an HDD will exhaust Loki's query capacity. The solution is not to abandon complex detection but to be disciplined about time ranges and query load during development. Using 5-minute windows while building, and expanding only for deliberate investigation sessions, keeps the lab functional without compromising detection capability.

16.12 Troubleshooting: Juice Shop Detection Dashboard

'Too Many Outstanding Requests' Error in Grafana

Grafana displays a 'too many outstanding requests' error when loading the dashboard or running queries.

Important

Root cause: Multiple complex queries running simultaneously over a wide time range (such as 24 hours) exceed Loki's processing capacity on this hardware. The Dell's CPU and HDD cannot sustain parallel JSON parsing, regex matching, and aggregation across large log volumes quickly enough.

Resolution: Reduce the dashboard time range to Last 5 minutes or Last 15 minutes. Set auto-refresh to 30 seconds or 1 minute. Build and test panels one at a time rather than loading the full dashboard simultaneously. The error resolves immediately when the query load is reduced.

json Queries Return No Fields Despite Logs Appearing in Raw View

Queries using `| json` return no structured fields (request_remote_ip, status, and other expected fields are absent) even though the raw log view shows entries.

Important

Root cause: The logs visible in the raw view are old console-format entries that predate the switch to JSON format. The `| json` parser fails on these lines and the `__error__` field contains 'JSONParserErr'. Because the filter `| __error__=""` is used, all such lines are excluded, leaving no results.

Resolution: Check whether the log file was truncated after changing the Caddyfile to 'format json'. Generate fresh traffic through port 3002, then re-run the query. Only new entries written after the format change will be valid JSON.

request_remote_ip Shows Unexpected IP Values

The Top Client IPs table shows IP addresses that do not correspond to the expected Acer or Kali source, for example an internal Docker gateway address rather than the real client IP.

Note

Root cause: When Caddy is running inside Docker and requests arrive from outside Docker via the host network bridge, the source IP in the log may reflect the Docker bridge gateway (typically 172.17.0.1 or similar) rather than the true client IP. This is a known characteristic of Docker networking, not a Caddy misconfiguration.

For this homelab's purposes, where all attacking traffic comes from within the lab network, the logged IP is still useful for distinguishing sources. In a production deployment, Caddy would be configured to trust and forward the X-Forwarded-For header from a front-end load balancer.

PART 8: METASPLOITABLE 2

17. Metasploitable 2 Design Decision and Virtualization Choice

The previous two phases of this homelab focused entirely on web application visibility. DVWA and Juice Shop both expose vulnerabilities at the HTTP layer and produce logs that record request-level activity. Detection on those targets is about identifying suspicious patterns in web traffic: injection payloads in query strings, repeated login attempts against authentication endpoints, API enumeration behaviour. That is one important layer of what a real SOC monitors, but it is not the whole picture.

Real infrastructure also runs network services: SSH for remote access, FTP for file transfer, Telnet for legacy management, Samba for file sharing. These services produce their own authentication events, session logs, and connection records, entirely separate from any web application logging. An attacker targeting a real system will often probe these services before or instead of the web application, and a SOC analyst must be able to detect and investigate activity at this layer too. Metasploitable 2 brings this dimension into the homelab.

17.1 Why Metasploitable 2 Was Added

Metasploitable 2 is a deliberately vulnerable Linux virtual machine distributed by the Metasploit project. Unlike a Docker container running a single vulnerable application, it is a full Linux OS with dozens of intentionally misconfigured and vulnerable services installed and running by default: SSH with weak credentials, an anonymous-access FTP server, a plaintext Telnet daemon, a vulnerable Samba installation, an old vulnerable Apache/PHP stack, and many others. It is exactly the kind of target that teaches service-level attack and detection skills that web application labs cannot provide.

17.2 Why Metasploitable 2 Instead of Metasploitable 3

Several alternative options were considered. Metasploitable 3 is newer and more feature-rich, but it requires significantly more resources to run. Active Directory lab environments are excellent for learning Windows-domain attacks, but they require at least two VMs and substantial RAM. Wazuh and other SIEM platforms with built-in agents would provide richer detection, but they would consume resources the Dell's hardware cannot spare.

Metasploitable 2 fits the hardware constraints of this lab precisely. It runs comfortably on 512 MB of RAM, which is low enough to co-exist with Loki, Alloy, Grafana, DVWA, Juice Shop, Caddy, and the Ubuntu host system without causing memory pressure. It is stable, well-documented, and has been the standard target for service-level attack practice in the security community for years. Its log output from syslog, SSH, FTP, and Telnet is readable and directly usable for detection without any special parsing.

i Note

The rule applied throughout this homelab when making tool and component choices applies equally here: choose what the hardware can actually sustain, not what sounds most impressive. A Metasploitable 2 VM running reliably at 512 MB RAM produces better learning outcomes than a Metasploitable 3 installation that makes the Dell unresponsive.

17.3 Why VirtualBox Was Selected

Metasploitable 2 runs as a virtual machine rather than a Docker container. This is because it is a full Linux operating system image. It includes a kernel, init system, multiple running services, and persistent state that a container cannot model. VirtualBox is the virtualisation platform used for this purpose.

VirtualBox was chosen over KVM for this phase because it is simpler to install and configure on Ubuntu Server 22.04 for this class of hardware, requires no kernel module setup beyond what the package handles automatically, and has straightforward host-only networking that integrates cleanly with the isolation requirements for a deliberately vulnerable VM. KVM remains an option for future phases where performance is more critical, but the additional complexity is unnecessary here.

17.4 What Not to Include as Installed

Proxmox is not installed and must not be described as part of the current setup. KVM may only be mentioned as an earlier or future alternative. The current implementation uses VirtualBox exclusively. Wazuh, Splunk, and ELK are not installed and do not appear as components of the Metasploitable phase.

17.5 Troubleshooting and Design Notes

The direct Alloy syslog listener approach for Metasploitable was tested during the design phase and abandoned. Metasploitable 2 uses legacy syslogd with RFC 3164 format, which Alloy's parser does not fully support and produced errors during testing. Additionally, both Alloy and rsyslog attempted to bind to UDP port 514, and only one process can hold a port at a time. The final architecture uses rsyslog to receive and write logs to a file, then Alloy tails that file. This decision is documented in full in Section 19.

18. Metasploitable 2 Deployment

18.1 Install VirtualBox

VirtualBox is installed directly on the Dell server using the standard Ubuntu package repository. All VirtualBox management on the headless server is done through the VBoxManage command-line tool.

```
sudo apt update
sudo apt install virtualbox -y
```

18.2 Download and Extract Image

The Metasploitable 2 image is distributed as a ZIP archive containing a VMDK (Virtual Machine Disk) file. It is available from SourceForge under the Metasploit project files. After downloading the archive, extract it to retrieve the disk image:

```
# The downloaded archive: Metasploitable2-Linux.zip
# After extraction: Metasploitable.vmdk (the virtual disk file used by VirtualBox)
```

18.3 Create VM

The VM is created with a deliberately minimal specification to keep resource consumption as low as possible. Metasploitable 2 does not need substantial resources. It is running 2006-era Linux services, not modern workloads. Every megabyte of RAM allocated to the VM is a megabyte unavailable to Loki, Grafana, and the other running containers.

```
VBoxManage createvm --name metasploitable2 --ostype Ubuntu --register
VBoxManage modifyvm metasploitable2 --memory 512 --cpus 1 --vram 16
```

Setting	Value and Reason
VM Name	metasploitable2 - used in VBoxManage commands to start and stop the VM
OS Type	Linux, Ubuntu (32-bit) - Metasploitable 2 is a 32-bit system based on Ubuntu 8.04
RAM	512 MB - sufficient for all Metasploitable services, minimal impact on host resources
CPU	1 Core - single core is enough; Metasploitable runs no CPU-intensive workloads
Video Memory	16 MB - minimum allocation; the VM runs headless so no display is ever rendered
Disk	Metasploitable.vmdk attached directly - no new virtual disk created
Start Mode	Headless - no GUI window opened, VM runs as a background process on the server

18.4 Attach VMDK

```
mkdir -p ~/vms/metasploitable2
# Move or copy Metasploitable.vmdk into:
# ~/vms/metasploitable2/Metasploitable.vmdk

VBoxManage storagectl metasploitable2 \
  --name "SATA Controller" \
  --add sata \
  --controller IntelAhci \
  --port 0 \
  --device 0 \
  --type hdd \
  --medium ~/vms/metasploitable2/Metasploitable.vmdk
```

Replace the path only if your extracted VMDK is stored somewhere else. Use an absolute or home-relative path that exists on the server.

18.5 Host-Only Network Setup

The network configuration for Metasploitable 2 is one of the most important decisions in this phase. A deliberately vulnerable system with dozens of exploitable services running must never have a path to the public internet or to the home network's general traffic. Exposing Metasploitable on the main home LAN would mean any device on that network could reach it.

The solution is a VirtualBox host-only network. A host-only network is a private network that exists only between the Dell host machine and any VMs connected to it. It is completely isolated from the home LAN, from the internet, and from other Docker containers unless a specific routing path is explicitly created.

Address	Assigned To
<YOUR-HOST-ONLY-NETWORK>	Host-only network subnet, exists only between the Dell host and VMs on this adapter
<YOUR-HOST-ONLY-ADAPTER-IP>	Dell host machine's interface on the host-only adapter, where rsyslog listens for forwarded logs
<YOUR-VM-IP>	Metasploitable 2 VM, assigned automatically by VirtualBox DHCP

```
VBoxManage list hostonlyifs
VBoxManage modifyvm metasploitable2 --nic1 hostonly --hostonlyadapter1 vboxnet0
```

Important

The host-only network is the correct isolation model for this lab. Metasploitable must be reachable from the Dell host so rsyslog can receive its forwarded logs, but it must not be reachable from anything else. A bridged or NAT network would expose Metasploitable to the wider home network or internet, which is not acceptable for a system running this many intentional vulnerabilities.

18.6 Start VM Headless

On a server with no graphical interface, VirtualBox VMs are started using the VBoxManage command-line tool with the `--type headless` flag. This starts the VM as a background process. It runs and consumes resources exactly like a normal VM, but no display window is opened and no GUI is rendered.

```
VBoxManage startvm metasploitable2 --type headless
# Verify the VM is running:
VBoxManage list runningvms
# Stop the VM when not in use:
VBoxManage controlvm metasploitable2 poweroff
```

Important

Like DVWA and Juice Shop, Metasploitable 2 should only be started when it is needed for an active practice session. It runs dozens of vulnerable services that represent real attack surface. Leaving it running at all times when not in use is unnecessarily risky, even within the isolated host-only network.

18.7 Connectivity Validation

After starting the VM, verify that the Dell host can reach Metasploitable on the host-only network before attempting to configure syslog forwarding:

```
ping <YOUR-VM-IP>
# Verify SSH works (required for editing Metasploitable configuration):
ssh -o StrictHostKeyChecking=no msfadmin@<YOUR-VM-IP>
# Default credentials: msfadmin / msfadmin
```

18.8 SSH Compatibility Fix

Modern OpenSSH clients (version 8.8 and later) have disabled several legacy algorithms that Metasploitable 2's old OpenSSH server requires for handshaking. If the SSH connection fails with a message about unsupported algorithms, add the following to `~/.ssh/config` on the Dell host:

```
Host <YOUR-VM-IP>
  KexAlgorithms +diffie-hellman-group1-sha1
  HostKeyAlgorithms +ssh-rsa
  PubkeyAcceptedAlgorithms +ssh-rsa
```

The `+` prefix adds these algorithms back without disabling modern ones for all other hosts. This configuration applies exclusively to the Metasploitable address.

18.9 Terminal Compatibility Fix

Metasploitable 2 uses an old version of bash and a minimal terminal environment. If the local terminal type is set to a modern value such as xterm-256color, some terminal operations may fail or display garbled output. Set the TERM variable to the basic xterm value for the duration of the SSH session:

```
export TERM=xterm
```

This only affects the current SSH session and does not persist after disconnecting. It tells the remote system to treat the terminal as a basic xterm rather than a modern colour-capable variant that Metasploitable's old ncurses libraries do not support.

18.10 Troubleshooting: Metasploitable Deployment

SSH Connection Rejected Due to Deprecated Algorithms

Attempting to SSH into Metasploitable from the Dell produces an error about unsupported key exchange algorithms or host key types.

Important

Root cause: Modern OpenSSH clients (version 8.8 and later) have disabled legacy algorithms including diffie-hellman-group1-sha1 for key exchange and ssh-rsa for host key verification, which Metasploitable 2's old OpenSSH server requires.

Resolution: Add a Host block to ~/.ssh/config on the Dell host specifying 'KexAlgorithms +diffie-hellman-group1-sha1' and 'HostKeyAlgorithms +ssh-rsa' for <YOUR-VM-IP>. The + prefix adds these algorithms back without affecting connections to other hosts.

VM Not Reachable After Start

The ping command to <YOUR-VM-IP> produces no response after starting the VM.

Note

Root cause: Metasploitable 2 takes 30 to 60 seconds to fully boot before its network interface becomes active. If the ping is run immediately after VBoxManage startvm, the VM may not have finished initialising.

Resolution: Wait 60 seconds after starting the VM and try again. If the issue persists, verify with 'VBoxManage list runningvms' that the VM is in the running state, and check that the VirtualBox host-only adapter is configured with the <YOUR-HOST-ONLY-NETWORK> subnet.

PART 9: METASPLOITABLE LOGGING PIPELINE

19. Metasploitable Logging Architecture

19.1 Why Docker Log Collection Does Not Apply

Getting Metasploitable's logs into Loki requires solving a problem that did not exist with DVWA and Juice Shop. Those applications ran as Docker containers on the Dell itself, and Alloy could collect their logs directly using Docker socket integration. Metasploitable is a separate virtual machine with its own operating system, running on a different IP address. Its logs cannot be collected through the Docker socket.

The solution is syslog forwarding. This is a mechanism built into Unix and Linux systems where the syslog daemon can be instructed to send a copy of every log message it receives to a remote host over the network. Metasploitable's syslog daemon is configured to forward all logs to the Dell host. On the Dell, rsyslog receives those forwarded logs and writes them to a dedicated log file. Alloy then tails that file and ships entries to Loki.

This three-stage design (VM forwards to host, host writes to file, Alloy reads file) is more stable and more representative of real-world practice than attempting to make Alloy directly ingest UDP syslog packets. It cleanly separates responsibilities: syslog handles transport, rsyslog handles reception and file writing, and Alloy handles forwarding to Loki.

19.2 Final Working Pipeline

```
Metasploitable syslogd
  down UDP 514 (over vboxnet0 host-only network)
Dell rsyslog (imudp module, bound to <YOUR-HOST-ONLY-ADAPTER-IP>)
  down
/var/log/metasploitable/metasploitable.log
  down
Alloy file tail (/var/log/metasploitable/metasploitable.log)
  down
Loki {job="metasploitable-syslog"}
  down
Grafana (Metasploitable Detection Dashboard)
```

19.3 Why Direct Alloy Syslog Was Rejected

The initial design for this phase attempted to configure Alloy with a syslog listener component that would receive UDP syslog packets directly from Metasploitable. This approach was abandoned for two reasons.

The first reason is protocol compatibility. Metasploitable 2 runs syslogd, an extremely old syslog implementation that formats its messages according to RFC 3164, a format from 1999 that differs significantly from the modern RFC 5424 standard. Alloy's syslog parser is designed for RFC 5424 and has limited tolerance for the older format. During testing, Alloy produced parsing errors for many Metasploitable log messages rather than forwarding them cleanly to Loki.

The second reason is port conflict. Both Alloy and rsyslog attempted to bind to UDP port 514, the standard syslog port. Only one process can hold a UDP port at a time. When rsyslog was running

as a system service (which it is by default on Ubuntu), Alloy's listener startup failed because the port was already in use.

 Tip

The rsyslog file-based approach more closely mirrors what enterprise SIEM deployments look like. In real environments, a dedicated log collector (rsyslog, syslog-ng, or Fluentd) receives and normalises logs from multiple sources and writes them to files or databases. A SIEM agent like Alloy then reads those normalised files. This separation of collection from forwarding is a sound architectural pattern, not a workaround.

19.4 Troubleshooting and Design Notes

All troubleshooting related to the direct Alloy syslog listener approach is documented in Section 21.7. That approach does not appear as an active component anywhere else in this document. The rsyslog file-based pipeline is the final and only active architecture for Metasploitable log collection.

20. Configure Metasploitable Syslog Forwarding

20.1 Identify Logging Service

Metasploitable 2 uses `/etc/syslog.conf` to configure its `syslogd` daemon. Unlike modern Ubuntu systems which use `rsyslog` and `/etc/rsyslog.conf`, Metasploitable uses the older `sysklogd` package. The correct configuration file to edit is `/etc/syslog.conf` on the Metasploitable VM.

Important

Do not edit `/etc/rsyslog.conf` on Metasploitable. That file either does not exist or is not the active configuration. The correct file is `/etc/syslog.conf`. This is one of the most common points of confusion when working with legacy syslog on Metasploitable 2.

20.2 Add Forwarding Rule

SSH into Metasploitable and edit the syslog configuration:

```
ssh msfadmin@<YOUR-VM-IP>
export TERM=xterm
sudo nano /etc/syslog.conf
```

Add the following line at the end of the file, below all existing rules:

```
*.* @<YOUR-HOST-ONLY-ADAPTER-IP>
```

Breaking down this rule:

- `*.*` - the facility and severity selector. The asterisk in each position is a wildcard meaning all facilities and all severity levels. This instructs `syslogd` to forward every log message regardless of what service generated it or how severe it is.
- `@` - tells `syslogd` to forward to a remote host using UDP. A single `@` means UDP; `@@` would mean TCP. Metasploitable's old `syslogd` only supports UDP, so a single `@` is correct here.
- `<YOUR-HOST-ONLY-ADAPTER-IP>` - the Dell host's IP address on the host-only network. This is the address where `rsyslog` listens for incoming syslog messages.

20.3 Restart sysklogd

After saving the file, restart the syslog daemon on Metasploitable. Metasploitable 2 predates `systemd` and uses the legacy SysV init system. The restart command uses `/etc/init.d/` scripts rather than `systemctl`.

```
sudo /etc/init.d/sysklogd restart
```

i Note

Using `systemctl` to restart services on Metasploitable will not work for most services. Always use the `/etc/init.d/` scripts. This is a key difference between Metasploitable 2 (which uses SysV init) and modern Ubuntu systems (which use systemd).

20.4 Generate Test Log

From inside Metasploitable (via SSH), generate a test log message using the `logger` command:

```
logger "METASPLOITABLE FORWARDING TEST"
```

Back on the Dell host, tail the log file to confirm the test message arrived:

```
sudo tail -f /var/log/metasploitable/metasploitable.log
```

i Note

The `rsyslog` receiver on the Dell must be configured before this test will work. If the log file does not exist yet, complete Section 21 first and then return to this step.

20.5 Troubleshooting: Metasploitable Syslog Forwarding

Forwarding Rule Has No Effect

After adding the `*.* @<YOUR-HOST-ONLY-ADAPTER-IP>` rule and restarting `syslogd`, no messages arrive at the Dell.

⚠ Important

Root cause: Common causes are that the line was not added to `/etc/syslog.conf` (not `/etc/rsyslog.conf`), or the file was not saved, or `syslogd` was not restarted after the change. **Resolution:** Re-open `/etc/syslog.conf` on Metasploitable and confirm the line `*.* @<YOUR-HOST-ONLY-ADAPTER-IP>` appears at the end. Save and run `'sudo /etc/init.d/syslogd restart'`. Then run `'logger "test"'` and check the Dell's log file.

syslogd Restart Fails

The `/etc/init.d/syslogd restart` command produces an error or has no effect.

i Note

Resolution: Try stopping and starting the service separately with `'/etc/init.d/syslogd stop'` followed by `'/etc/init.d/syslogd start'`. Also check that the `syslog.conf` syntax is correct: the forwarding line must start with `*.*` followed by a single space and then `@<YOUR-HOST-ONLY-ADAPTER-IP>`.

21. Configure Dell rsyslog Receiver

21.1 Install rsyslog

rsyslog may already be installed on Ubuntu Server 22.04 as it is the default syslog implementation. Install it explicitly to be certain:

```
sudo apt install rsyslog -y
```

21.2 Create Dedicated Log File

The log directory and file must exist before rsyslog can write to them, and they must have the correct ownership so that rsyslog's process user has write access.

```
sudo mkdir -p /var/log/metasploitable
sudo touch /var/log/metasploitable/metasploitable.log
sudo chown syslog:adm /var/log/metasploitable
sudo chmod 750 /var/log/metasploitable
sudo chown syslog:adm /var/log/metasploitable/metasploitable.log
sudo chmod 640 /var/log/metasploitable/metasploitable.log
```

Why these specific settings:

- `chown syslog:adm` - rsyslog runs as the 'syslog' system user. The log file must be owned by this user or rsyslog will receive a permission denied error when attempting to write messages. The 'adm' group allows members of that group (including the regular user account) to read the log file.
- `chmod 640` on the file - the owner (syslog) has read and write access. The group (adm) has read access. Others have no access. This is the standard permission pattern for system log files.
- `chmod 750` on the directory - the owner (syslog) has read, write, and execute access. The group (adm) has read and execute access. Others have no access at all. This is intentionally more restrictive than 755 because the Metasploitable log directory should not be readable by arbitrary users on the system.

21.3 Final rsyslog Receiver Config

Rather than modifying rsyslog's main configuration file, a separate configuration file is created in `/etc/rsyslog.d/`. rsyslog loads all `.conf` files in this directory, so placing a new file here is the clean, non-destructive way to add configuration without touching the defaults.

```
sudo nano /etc/rsyslog.d/30-metasploitable.conf
```

Paste the following configuration:

```
# Load the UDP input module
module(load="imudp")

# Listen for incoming syslog on UDP port 514, bound to the host-only interface
input(type="imudp" port="514" address="<YOUR-HOST-ONLY-ADAPTER-IP>")

# If the message came from Metasploitable, write it to the dedicated log file
if $fromhost-ip == '<YOUR-VM-IP>' then /var/log/metasploitable/metasploitable.log
& stop
```

Each part of this configuration explained:

- `module(load="imudp")` - loads the UDP input module for rsyslog. Without this, rsyslog cannot receive UDP syslog messages. The module must be loaded before any UDP input rules can be defined.
- `input(type="imudp" port="514" address="<YOUR-HOST-ONLY-ADAPTER-IP>")` - activates a UDP listener on port 514, bound specifically to the Dell's vboxnet0 interface. Binding to `<YOUR-HOST-ONLY-ADAPTER-IP>` rather than all interfaces ensures the listener only accepts traffic from the host-only network, not from the broader home LAN.
- `if $fromhost-ip == '<YOUR-VM-IP>'` - filters incoming messages to only those originating from Metasploitable's IP address. This prevents logs from any other device that might send syslog to the Dell from being mixed into the Metasploitable log file.
- `/var/log/metasploitable/metasploitable.log` - the file where matched messages are written. This is the file that Alloy will tail.
- `& stop` - after writing the matching message to the file, rsyslog stops processing it further. Without this directive, rsyslog would also write Metasploitable's messages to its normal system log files alongside the Dell's own system messages.

21.4 Restart rsyslog

The `-N1` flag performs a dry-run validation of the rsyslog configuration without starting or restarting the daemon. If the configuration is valid, this command produces no output or prints a brief validation message and exits cleanly. If there is a syntax error, it prints the error before you restart the service. Run this check before restarting rsyslog after editing files in `/etc/rsyslog.d/`.

```
sudo rsyslogd -N1
sudo systemctl restart rsyslog
```

21.5 Verify Reception

From inside Metasploitable (via SSH), generate a test log message:

```
logger "METASPLOITABLE FORWARDING TEST"
```

Back on the Dell host, tail the log file to confirm the test message arrived:

```
sudo tail -f /var/log/metasploitable/metasploitable.log
```

✓ Result

If the test message appears in the tail output, the syslog pipeline is confirmed working: Metasploitable generated a log message, syslogd forwarded it over UDP to the Dell, rsyslog received it on port 514, the IP filter matched it, and it was written to the correct file. If nothing appears, work through Section 21.6.

21.6 UFW Rule for Metasploitable Syslog

If UFW is active on the Dell, a specific firewall rule must be added to allow incoming UDP traffic from Metasploitable on port 514, restricted to the vboxnet0 interface only. A broad unrestricted allow rule for port 514 is not appropriate here.

```
sudo ufw allow in on vboxnet0 from <YOUR-VM-IP> to any port 514 proto udp
sudo ufw reload
sudo ufw status numbered
```

⚠ Important

Do not use 'sudo ufw allow 514/udp' as the rule for Metasploitable. That would allow UDP port 514 from any source on any interface, which unnecessarily exposes the syslog port on the home LAN interface as well. The correct rule restricts the allow to the vboxnet0 interface and the specific Metasploitable IP.

21.7 Troubleshooting: rsyslog Receiver

UDP Port 514 Already in Use

Alloy fails to start with an error about port 514 already being bound, or rsyslog fails to start with a similar error.

⚠ Important

Root cause: Only one process can hold a UDP port at a time. If the Alloy configuration still contains a syslog listener block pointing to port 514, it will conflict with rsyslog's imudp module.

Resolution: Remove any syslog listener configuration from the Alloy config file. Alloy's role in this architecture is exclusively file-based. It reads from the file that rsyslog writes. It does not receive UDP syslog directly. rsyslog handles all UDP ingestion.

rsyslog Writes Nothing to the Log File

The Metasploitable log file exists and has correct permissions, rsyslog is running, but no messages appear in the file even after sending logger test messages from Metasploitable.

Important

Root cause: Several possible causes. First, verify that Metasploitable's syslogd is actually forwarding to the correct IP by checking `/etc/syslog.conf` on Metasploitable for the `.* @<YOUR-HOST-ONLY-ADAPTER-IP>` line. Second, verify rsyslog loaded the configuration file without errors: `'sudo journalctl -u rsyslog | tail -20'`. Third, check that no firewall rule on the Dell is blocking incoming UDP on port 514: `'sudo ufw status'`. Fourth, confirm rsyslog was restarted after the configuration file was created.

Permission Denied Error in rsyslog Log

rsyslog logs show a permission denied error when attempting to write to `/var/log/metasploitable/metasploitable.log`.

Important

Root cause: The log file or directory is not owned by the syslog user. rsyslog runs as the syslog system user and can only write to files that this user owns or has write permission to. **Resolution:** Run `'sudo chown syslog:adm /var/log/metasploitable/metasploitable.log'` and `'sudo chmod 640 /var/log/metasploitable/metasploitable.log'`, then restart rsyslog with `'sudo systemctl restart rsyslog'`.

RFC 3164 Format Causing Alloy Parse Errors

Alloy logs show repeated parse errors for messages from Metasploitable, or messages are arriving in Loki with garbled or missing fields.

Note

Root cause: This is the expected behaviour when Alloy is configured with a syslog protocol listener rather than a file source. Metasploitable's old syslogd uses RFC 3164 format, which Alloy's parser does not fully support.

Resolution: This is not an issue in the file-based architecture used here. Alloy reads the file that rsyslog wrote. rsyslog handles the protocol parsing and produces clean log lines in the file. Alloy simply tails the file and forwards lines as plain text to Loki, bypassing the protocol compatibility issue entirely.

22. Alloy Collection for Metasploitable Logs

22.1 Why File Tailing Was Used

With rsyslog writing Metasploitable's logs to `/var/log/metasploitable/metasploitable.log`, the final step in the pipeline is configuring Alloy to tail that file and forward entries to Loki. The same file-based collection pattern used for Caddy's access log applies here, with one key difference: the Metasploitable log file is in `/var/log/metasploitable/`, which is a subdirectory of `/var/log/`. Since the `/var/log` volume is already mounted read-only into the Alloy container, no additional volume mount is needed.

22.2 Final Alloy Block

Open the Alloy configuration file and add the following block above the `loki.write` section:

```
nano ~/logging/alloy/config.alloy
```

```
// Metasploitable 2 syslog forwarded via rsyslog
local.file_match "metasploitable_logs" {
  path_targets = [
    {
      "__path__" = "/var/log/metasploitable/metasploitable.log",
      "job"      = "metasploitable-syslog",
      "host"     = "metasploitable2",
      "service"  = "syslog",
    },
  ]
}

loki.source.file "metasploitable_logs" {
  targets      = local.file_match.metasploitable_logs.targets
  forward_to   = [loki.write.loki.receiver]
}
```

The key label set by this block is `job='metasploitable-syslog'`. This is the stream selector used in all Grafana queries for the Metasploitable Detection Dashboard. The host label is set to `'metasploitable2'` to distinguish these logs from the Dell host's own syslog, which uses `'homelab'` as the host label.

22.3 Recreate Alloy

Recreate the Alloy container with the updated configuration. Because the Metasploitable log file is within `/var/log/`, which is already mounted into the Alloy container via `-v /var/log:/var/log:ro`, no new volume mount is required. Recreate Alloy with the same command used after the Caddy integration, which already includes all necessary mounts:

```
docker rm -f alloy
```

```
docker run -d --name alloy \
  --network labnet \
  --restart unless-stopped \
  --user root \
  -p 127.0.0.1:12345:12345 \
  -v ~/logging/alloy/config.alloy:/etc/alloy/config.alloy \
  -v /var/log:/var/log:ro \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -v ~/caddy/juice-shop/logs:/caddy-logs:ro \
  grafana/alloy:latest \
  run /etc/alloy/config.alloy \
  --server.http.listen-addr=0.0.0.0:12345
```

i Note

The `-v /var/log:/var/log:ro` mount covers the entire `/var/log` directory tree on the Dell host, including `/var/log/metasploitable/`. This is why no additional mount is needed. Alloy can already see `/var/log/metasploitable/metasploitable.log` at the path configured in the `file_match` block.

22.4 Verify in Grafana

After recreating Alloy, verify that Metasploitable logs are appearing in Grafana. In Grafana Explore, run:

```
{job="metasploitable-syslog"}
```

If entries appear, the full pipeline is confirmed: Metasploitable generated logs, syslogd forwarded them over UDP, rsyslog received and wrote them to file, Alloy read the file and forwarded entries to Loki, and Loki stored them for Grafana to query.

To generate fresh entries for this verification, attempt an SSH connection to Metasploitable using a wrong username, which immediately generates an SSH authentication event in the syslog:

```
# From the Dell host:
ssh wronguser@<YOUR-VM-IP>
```

```
# Then query Grafana:
{job="metasploitable-syslog"} |~ "sshd"
```

22.5 Troubleshooting: Alloy Metasploitable Collection

No Results for `{job="metasploitable-syslog"}` in Grafana

The query returns no results even though rsyslog is receiving messages and writing to the log file.

⚠ Important

Root cause: Either the Alloy config block was not saved correctly before recreating the container, or the path in the file_match block does not match the actual file location.

Resolution: Verify the Alloy config with 'cat ~/logging/alloy/config.alloy' and confirm the metasploitable_logs block is present with __path__ =

"/var/log/metasploitable/metasploitable.log". Check 'docker logs alloy' for any errors. Confirm the file exists on the Dell host with 'ls -la /var/log/metasploitable/'.

PART 10: METASPLOITABLE DETECTION DASHBOARD

23. Metasploitable Detection Dashboard

The Metasploitable Detection Dashboard is the fourth dedicated detection dashboard in this homelab, following the Homelab Monitoring Dashboard, the DVWA Detection Dashboard, and the Juice Shop Detection Dashboard. Its data source, detection logic, and investigation focus are all different from the others.

Where the DVWA and Juice Shop dashboards focus on HTTP request patterns, URL-encoded payloads, and web application behaviour, the Metasploitable dashboard focuses on operating system and network service activity: SSH authentication events, FTP and Telnet connection records, session lifecycle events, and service-level anomalies. This is the layer of real infrastructure that sits beneath the web application and generates its own stream of security-relevant events entirely independently of any web traffic.

23.1 Dashboard Purpose

This dashboard provides visibility into attack activity against Metasploitable's network services. It detects brute force attempts against SSH, anonymous FTP access, Telnet connections, reconnaissance scanning patterns, and general authentication abuse. Unlike the web application dashboards, which work from structured HTTP access logs, this dashboard works from raw syslog messages in plain text format, which requires a different detection approach.

23.2 Design Philosophy

A design decision that applies across every dashboard in this homelab, but is especially important here, is the principle of behaviour-based detection rather than label-based detection.

Metasploitable's syslog does not contain messages that say 'brute force attack detected' or 'SSH scan in progress'. What it contains is raw evidence: failed password messages, connection refused entries, session opened and closed records, authentication failure notifications.

Detection logic is built by asking what the behaviour of an attack looks like in raw log terms. A brute force attack against SSH does not announce itself as a brute force. It appears as a large number of 'Failed password' messages in a short time window, often involving multiple different usernames, all originating from the same IP. The detection panel does not look for the phrase 'brute force'. It counts 'Failed password' occurrences per minute, which is the actual observable evidence of that behaviour.

i Note

This approach, reasoning from observable evidence to attack behaviour, is precisely how analysts work in real SIEM environments. Enterprise SIEMs do not receive logs that say 'attack occurred'. They receive the same raw system events that Metasploitable generates, and detection rules identify attack patterns by correlating event types, frequencies, and sources. Building detection logic this way in the homelab develops the same analytical thinking that real SOC work requires.

23.3 Data Source

Every panel on this dashboard uses the following Loki stream selector:

```
{job="metasploitable-syslog"}
```

This label was set in the Alloy configuration file during the pipeline setup in Section 22. It isolates Metasploitable's log entries from all other streams in Loki. The Dell host's syslog, the auth log, all Docker container logs, and the Juice Shop Caddy logs all use different job labels and will not appear in queries using this selector.

23.4 Row 1: Overview

Panel: Total Events

This stat panel counts all log entries from Metasploitable in the last five minutes. In an idle state, the count reflects background syslog activity: cron job events, keepalive messages, and periodic service status logging. During an active attack simulation, the count rises significantly. A non-zero count also confirms the pipeline is working. Suggested thresholds: 0 to 5 is green (idle baseline), 5 to 15 is yellow (elevated activity), above 15 is red (high activity, investigate).

```
sum(count_over_time({job="metasploitable-syslog"}[5m]))
```

Visualisation type: Stat.

Panel: Event Rate Over Time

This panel plots the count of log entries per one-minute bucket. The shape of the graph is as informative as the count. A flat baseline with a sudden steep spike is characteristic of a scan or burst attack. A gradual sustained rise indicates something ongoing such as a running brute force tool. Repeated small spikes at regular intervals may indicate automated probing.

```
sum(count_over_time({job="metasploitable-syslog"}[1m]))
```

Visualisation type: Time series. Line width 2, fill opacity 20 percent, legend enabled.

Panel: Suspicious Activity Score

The suspicious activity score aggregates matches against a broad set of security-relevant terms across a five-minute window. The keywords targeted cover the most common indicators of abuse activity in system logs. This is a triage indicator rather than a precision detection signal. A high number tells you something is worth investigating, and the row-specific panels below identify what kind of activity it is.

```
sum(  

```

```
count_over_time(
  {job="metasploitable-syslog"}
  |~ "(?i)(failed|invalid|authentication
failure|denied|refused|root|anonymous|error|warning|session opened)"
  [5m]
)
```

Visualisation type: Stat. Thresholds: 0 is green, 5 is yellow, 15 is red.

23.5 Row 2: SSH and Authentication

SSH is one of the most frequently attacked services on any network-connected Linux system. Metasploitable exposes SSH with known weak default lab credentials and no rate limiting or lockout policy, making it an effective target for demonstrating brute force authentication attacks and observing exactly how those attacks appear in system logs.

Panel: Failed Login Attempts

This panel counts log entries matching common authentication failure patterns per one-minute window. A normal system with no active attacks produces a low and irregular count from occasional automated checks. A brute force tool produces a sustained, elevated, near-flat rate that stands distinctly above the baseline.

```
sum(
  count_over_time(
    {job="metasploitable-syslog"}
    |~ "(?i)(failed password|authentication failure|invalid user|login incorrect)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: Root Login Attempts

This panel filters specifically for log entries referencing the root account in the context of failed or attempted authentication. Attackers commonly target root as the first account because a successful root login gives immediate full system access. Any count above zero represents deliberate targeting.

```
sum(
  count_over_time(
    {job="metasploitable-syslog"}
    |~ "(?i)(failed password for root|invalid user root|authentication
failure.*root)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: SSH Evidence Logs

The evidence panel surfaces the raw log lines for all SSH and authentication-related entries. Each line shows the exact failure message, the username that was attempted, the source IP, and the timestamp. Reading this panel after a brute force simulation makes the attack pattern immediately legible.

```
{job="metasploitable-syslog"} |~ "(?i)(sshd|failed password|authentication failure|invalid user|root)"
```

Visualisation type: Logs. Timestamp visible, deduplication off, newest first.

23.6 Row 3: FTP and Telnet

Metasploitable runs an FTP server (vsftpd) with anonymous access enabled and a Telnet daemon. FTP transmits credentials in plaintext and often has anonymous access enabled by default on older configurations. Telnet transmits everything in plaintext including session content. Both are commonly targeted during service enumeration and exploitation.

Panel: FTP Activity

This panel counts log entries related to FTP service activity per minute. A baseline of zero or near-zero activity is expected at rest. Anonymous FTP matches specifically indicate anonymous login attempts, which are a notable signal on a system that should not be used for legitimate anonymous file access.

```
sum(
  count_over_time(
    {job="metasploitable-syslog"}
    |~ "(?i)(ftp|vsftpd|proftpd|anonymous)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: Telnet Activity

This panel monitors for Telnet daemon activity. Telnet activity on any modern network is inherently suspicious because Telnet should not be used for legitimate remote access anywhere that SSH is available. On Metasploitable, Telnet access attempts from the Kali VM during attack simulations will generate entries here.

```
sum(
  count_over_time(
    {job="metasploitable-syslog"}
    |~ "(?i)(telnet|in.telnetd|login)"
    [1m]
  )
)
```

Visualisation type: Time series.

Panel: FTP and Telnet Evidence

The combined evidence panel shows raw log lines for all FTP and Telnet-related entries. Combining both into one panel is more useful than two separate evidence panels because service-level attacks often involve both, and correlating those events within the same panel view is faster than switching between tabs.

```
{job="metasploitable-syslog"} |~
"(?i)(ftp|vsftpd|proftpd|anonymous|telnet|in.telnetd)"
```

Visualisation type: Logs.

23.7 Row 4: Reconnaissance

Reconnaissance is typically the first phase of an attack. Network scanners like Nmap generate high volumes of connection attempts to many ports in a short period, producing a characteristic spike in service connection log entries. The shape of this spike (brief and spanning many service types) distinguishes it from brute force activity (sustained and targeting a single service).

i Note

Nmap scans may create visible service logs when scanned services log connection attempts, authentication prompts, or refused sessions. This dashboard provides service-log visibility, not full packet-level network detection. For packet-level detection, a tool like Suricata or Zeek would be needed, which is listed as a future option in the roadmap.

Panel: Service Connection Activity

This panel counts connection, session, and service interaction events per minute. Under normal conditions these counts are low. When an Nmap scan is running, many services attempt to handle

or reject incoming connections simultaneously, generating a sharp spike. The spike is typically brief (matching the scan duration) and wide (spanning many service types).

```
sum(  
  count_over_time(  
    {job="metasploitable-syslog"}  
    |~ "(?i)(connection|connect|session opened|session closed|refused)"  
    [1m]  
  )  
)
```

Visualisation type: Time series.

Panel: Authentication Burst Detection (30s)

This panel uses a 30-second window rather than the standard one-minute window. The shorter window is more sensitive to bursts. It captures the initial spike of a brute force attack more clearly because it does not average high activity across a full minute alongside quieter periods. A brute force tool that fires 50 requests in 10 seconds produces a count of 50 in the 30-second window, whereas a 1-minute window would show it spread across the full minute.

```
sum(  
  count_over_time(  
    {job="metasploitable-syslog"}  
    |~ "(?i)(authentication|login|session opened|session closed|failed)"  
    [30s]  
  )  
)
```

Visualisation type: Time series.

Panel: Reconnaissance Evidence

The raw evidence panel for reconnaissance shows connection, session, and login events in chronological order. After running an Nmap scan, this panel will show a cluster of connection and refused entries with timestamps tightly packed together, the distinctive pattern of a scanner hitting many ports in rapid succession.

```
{job="metasploitable-syslog"} |~  
"(?i)(connection|refused|session|opened|closed|login)"
```

Visualisation type: Logs.

23.8 Row 5: Investigation

The investigation row provides two complementary views for manual log analysis: a filtered view showing only the highest-value suspicious entries, and a completely unfiltered view showing every log line. The filtered view reduces noise and helps an analyst prioritise; the unfiltered view ensures nothing is hidden and supports deep investigation when context beyond the filtered results is needed.

Panel: Suspicious Logs

This panel filters the log stream to only lines matching a broad set of security-relevant terms. It covers authentication failures, access denials, root account references, anonymous access, and error conditions. The result is a reduced stream that highlights the most likely security-relevant events without requiring the analyst to read through every background system message. This panel is the equivalent of a SIEM alert triage queue.

```
{job="metasploitable-syslog"} |~  
"(?i)(failed|invalid|denied|refused|authentication|root|anonymous|error|warning)"
```

Visualisation type: Logs.

Panel: Full Raw Logs

The full raw log panel applies no filter and displays every log entry forwarded from Metasploitable. This panel is essential when an event in the suspicious panel requires surrounding context to interpret, or when the suspicious panel shows nothing but something may still be wrong. The raw stream is the ground truth.

```
{job="metasploitable-syslog"}
```

Visualisation type: Logs. Timestamp visible, deduplication off, newest first.

23.9 Complete Panel Reference

Row	Panel	Viz Type	Detection Purpose
Overview	Total Events	Stat	Pipeline health and overall activity level
Overview	Event Rate	Time series	Activity trend - shape reveals attack type
Overview	Suspicious Score	Stat	Composite security indicator across all log types
SSH / Auth	Failed Login Attempts	Time series	Brute force detection via failure frequency
SSH / Auth	Root Login Attempts	Time series	Root account targeting indicator
SSH / Auth	SSH Evidence Logs	Logs	Raw authentication log lines for manual review
FTP / Telnet	FTP Activity	Time series	FTP service use including anonymous access attempts
FTP / Telnet	Telnet Activity	Time series	Telnet daemon connections, legacy service monitoring
FTP / Telnet	FTP and Telnet Evidence	Logs	Combined FTP and Telnet raw log lines
Reconnaissance	Service Connection Activity	Time series	Service connection volume, Nmap scan indicator
Reconnaissance	Auth Burst (30s)	Time series	Short-window burst detection for rapid brute force
Reconnaissance	Reconnaissance Evidence	Logs	Connection and session events, scan activity visible
Investigation	Suspicious Logs	Logs	Filtered high-value events for analyst triage
Investigation	Full Raw Logs	Logs	Unfiltered complete log stream, ground truth view

23.10 Attack Simulation and Validation

SSH Brute Force Simulation

From the Dell host or the Kali VM (if routing to the Metasploitable network is configured), attempt SSH logins with incorrect credentials. A single failed attempt generates a 'Failed password' log entry in Metasploitable's syslog. Multiple rapid attempts will produce a visible spike in the Failed Login Attempts panel.

```
# Single failed login attempt from the Dell host:  
ssh wronguser@<YOUR-VM-IP>
```

```
# For a more realistic brute force simulation, Hydra can be used from Kali:  
hydra -l root -P /usr/share/wordlists/rockyou.txt ssh://<YOUR-VM-IP>
```

✓ Result

When running Hydra or similar brute force tools, the Failed Login Attempts panel should show a sustained elevated count for the duration of the tool's execution, followed by a return to baseline when the tool stops. This cause-and-effect observation (tool running, count rises, tool stops, count falls) is the core learning outcome of this simulation.

Reconnaissance Scanning

Running an Nmap scan against Metasploitable generates service connection and refusal events across many ports simultaneously. The Service Connection Activity panel should show a sharp spike during the scan.

```
# Service version scan at moderate speed (T2 reduces log flooding):  
nmap -T2 -sV <YOUR-VM-IP>
```

```
# Basic port scan:  
nmap <YOUR-VM-IP>
```

Manual Log Injection for Pipeline Testing

When verifying the pipeline without running attacks, the logger command on Metasploitable can inject a test message directly into the syslog stream:

```
# SSH into Metasploitable and run:  
logger "MANUAL PIPELINE TEST MESSAGE"
```

```
# Then query in Grafana:  
{job="metasploitable-syslog"} |= "MANUAL PIPELINE TEST MESSAGE"
```

23.11 Detection Limitation

Nmap scans may create visible service logs when scanned services log connection attempts, authentication prompts, or refused sessions. This dashboard provides service-log visibility, not full packet-level network detection. An Nmap scan that does not cause any service to log a connection will not appear in this dashboard. For full packet-level detection, a dedicated IDS such as Suricata or Zeek would be required. These are listed as future optional additions in the roadmap.

23.12 What This Phase Adds to the Homelab

Adding Metasploitable 2 with the rsyslog forwarding pipeline and detection dashboard brings a genuinely different category of visibility into the homelab. The previous phases all operated at the HTTP layer: web requests, URL patterns, and response codes. Metasploitable operates at the system and network service layer, which is an entirely separate attack surface with different evidence, different detection patterns, and different investigation workflows.

The homelab now covers three distinct visibility layers simultaneously:

- Web application layer: DVWA and Juice Shop producing HTTP access logs, captured through Docker container output and Caddy reverse proxy logging, visible in two separate detection dashboards
- System and service layer: Metasploitable producing syslog entries from SSH, FTP, Telnet, and other services, forwarded via rsyslog, visible in the Metasploitable Detection Dashboard
- Infrastructure health layer: the Homelab Monitoring Dashboard confirming that all logging pipelines are functioning and all log sources are active

Tip

The ability to correlate events across dashboards using source IP and timestamp is one of the most important skills this lab now supports practising. When reviewing evidence after an attack session, bring up the Metasploitable Detection Dashboard and the DVWA Detection Dashboard side by side. Find the IP responsible for the Metasploitable brute force attempts. Check whether that same IP appears in the DVWA logs. Check the timestamps. This is how SOC analysts reconstruct attack sequences from raw log evidence.

23.13 Troubleshooting: Metasploitable Dashboard

No Data for {job="metasploitable-syslog"}

All panels return no data even though the Metasploitable VM is running and the rsyslog pipeline was verified.

Important

Root cause: The most common causes are that the Alloy config block was not saved before recreating the container, or the Alloy container was not recreated after the config change, or rsyslog is not forwarding messages to Loki because the file is not being written.

Resolution: Check 'docker logs alloy' for errors. Verify 'cat ~/logging/alloy/config.alloy' shows the metasploitable_logs block. Confirm 'sudo tail /var/log/metasploitable/metasploitable.log' shows recent entries. If the file is empty, the rsyslog pipeline is the issue. If the file has content but Grafana shows nothing, the Alloy configuration is the issue.

Dashboard Shows Events but Panels Are Not Detecting Attack Activity

The raw log panels show Metasploitable entries but the SSH failed login or FTP panels remain at zero during confirmed attack activity.

Note

Root cause: The regex patterns in the detection panels may not match the exact wording of the log messages produced by Metasploitable's specific service versions.

Resolution: Open the Full Raw Logs panel and read the actual log lines produced during the attack. Identify the exact failure message format (for example, 'Failed password for invalid user' or 'authentication failure'). Adjust the regex in the detection panel to match. The source documentation panels in the investigation row are designed specifically for this process.

PART 11: CROSS-DASHBOARD SOC WORKFLOW

24. Multi-Source Investigation Workflow

With all four dashboards operational, the homelab supports something that neither a single vulnerable application nor a single detection dashboard can provide on its own: the ability to correlate attack evidence across multiple separate log sources. This is the core skill of intermediate SOC analyst work, and it is what the full lab architecture was designed to make possible.

Standard investigation sequence:

1. Open the Homelab Monitoring Dashboard and confirm log ingestion is healthy.
2. Identify which detection dashboard shows abnormal activity.
3. Set Grafana to the exact time window of the test or suspected event.
4. Review the overview row first for spikes.
5. Pivot to source-IP, endpoint, service, or status-code panels.
6. Open raw evidence panels and confirm the matching log lines.
7. Record timestamp, source identifier, target service, request path or service name, status/result, and matching rule.
8. Check the other detection dashboards for activity in the same time window.
9. Document whether the activity is confirmed, suspicious, benign test traffic, or unsupported by evidence.

24.1 Why Separate Dashboards Exist

Each dashboard in this homelab was built to answer a specific question about a specific source. The Homelab Monitoring Dashboard answers: is the logging pipeline healthy?

The DVWA Detection Dashboard answers: is anyone attacking the DVWA web application?

The Juice Shop Detection Dashboard answers: is anyone attacking the Juice Shop API?

The Metasploitable Detection Dashboard answers: is anyone attacking the network services on the vulnerable VM?

Keeping these separate is deliberate. Mixing them would create visual clutter, make query logic harder to follow, and destroy the clarity that makes each dashboard useful. In a real SOC, analysts work across multiple dashboards and data sources simultaneously. The skill is not in having one dashboard that shows everything; it is in knowing which dashboard to open first given a particular alert, and how to pivot between sources when the investigation requires it.

24.2 Correlation by Timestamp and Source IP

The two main correlation anchors across the detection dashboards are timestamp and source identity. However, source IP must be treated carefully because it is not equally reliable across every log source.

In DVWA logs, the client IP is usually visible in the Apache-style access log. In Juice Shop logs, Caddy's `request_remote_ip` may sometimes show a Docker bridge or gateway address instead of the real client IP, depending on Docker NAT behaviour. In Metasploitable logs, source IP visibility depends on whether the specific service records the remote address and whether the attack path is routed correctly.

For this reason, timestamp is the most reliable universal anchor across all dashboards. Source IP is still useful, but only after it is validated against raw evidence. A source IP should not be treated as confirmed attacker attribution unless the raw logs support that interpretation.

Cross-dashboard correlation should therefore use multiple indicators together:

- timestamp and time window
- source IP or source identity, where validated
- target system
- affected service or endpoint
- request path or service name
- status code or authentication result
- repeated behaviour pattern
- raw evidence from the relevant dashboard

If a controlled route from Kali to the Metasploitable host-only network has been configured, the same attacker identity may appear across DVWA, Juice Shop, and Metasploitable evidence. If no route exists, Metasploitable testing should be performed from the Dell host, and cross-dashboard correlation should focus on timestamp, target sequence, service activity, and known test behaviour rather than assuming the Kali IP appears in every dashboard.

DVWA and Juice Shop include source-oriented panels where the log format supports IP extraction. Metasploitable correlation is more service-log focused unless a dedicated source-IP extraction panel has been created and validated from the raw syslog format. For example, a root login attempt panel shows account targeting behaviour, but it is not automatically an IP attribution panel unless the query extracts and groups by source IP.

A strong correlation finding is not simply “the same IP appeared twice.” A stronger finding is when multiple pieces of evidence align within the same time window. For example, DVWA may show repeated brute-force activity, Juice Shop may show a burst of login or API requests, and Metasploitable may show failed SSH authentication attempts around the same period. If the source identity is validated in each log source, this supports a multi-target attack pattern. If the source identity is not reliable in one source, the conclusion should be limited to a time-based and behaviour-based correlation.

Tip

The most effective way to practise multi-source correlation is to run an attack session that touches multiple targets, then investigate only from the dashboards without referring to what you did during the attack. Open the Monitoring Dashboard first to confirm all pipelines are healthy. Then open each detection dashboard and review elevated panels, timestamps, raw evidence, source indicators, affected services, and repeated behaviour. The goal is to reconstruct the sequence from evidence, not assumption.

24.3 Example Attack Storyline

The following describes a realistic multi-stage attack sequence and how it appears across the dashboards. This is an example investigation scenario, not a step-by-step attack guide.

Stage 1: Reconnaissance

An attacker begins with an Nmap scan of Metasploitable. In Grafana, the Metasploitable Detection Dashboard shows a sharp spike in the Service Connection Activity panel. The Reconnaissance Evidence panel shows a burst of connection and refused entries with tightly packed timestamps. The overview Attack Score rises. The source IP can be read from the raw evidence.

Stage 2: Service Attack

The attacker pivots to SSH brute force against Metasploitable. The Failed Login Attempts panel shows a sustained elevated count. The Root Login Attempts panel rises if root is being targeted. The SSH Evidence Logs panel shows repeated 'Failed password' messages from the same source IP.

Stage 3: Web Application Attack

The same attacker then targets DVWA. The DVWA Detection Dashboard shows a spike in Login Attempts Over Time if brute force is attempted against the web login, and the SQLi or XSS panels if injection payloads are submitted. The Top Attacker IPs panel shows the same IP address seen in the Metasploitable logs. Timestamps in both dashboards confirm the sequence: Metasploitable activity came first, DVWA activity followed.

Stage 4: API Abuse

The attacker also probes Juice Shop. The Juice Shop Detection Dashboard shows elevated API Request Burst counts and Sensitive Endpoint Access. The Top Client IPs table shows the attacker's IP again. Cross-referencing timestamps across all three detection dashboards confirms that all activity originated from the same source within the same session window.

24.4 Evidence Handling

After an attack session, the key evidence handling workflow is:

- Note the approximate time window during which the attack occurred
- Open each detection dashboard and set the time range to cover that window
- In each dashboard, identify the source IP or IPs responsible for elevated panel activity
- Use the raw evidence (Logs) panels to read the actual log lines and confirm the nature of the activity
- Record the timestamps, source IPs, destination services, and attack types in a written summary
- Cross-reference timestamps across dashboards to reconstruct the attack sequence

i Note

Grafana's Explore view supports ad-hoc investigation without being constrained to a pre-built dashboard. After identifying a suspicious IP in one dashboard, switching to Explore and running queries filtered to that IP across multiple job labels is a fast way to see the full picture. For example: `{job=~"docker|metasploitable-syslog"} |= "<Attacker_IP>"` would show all entries from that IP across both Docker container logs and Metasploitable syslog simultaneously.

24.5 Troubleshooting and Workflow Notes

Cross-Dashboard Timestamps Do Not Align

Events that appear to be from the same attack session show timestamps that are several minutes apart across dashboards.

⚠ Important

Root cause: The server's timezone may have been changed after some containers were started, or containers were restarted at different times and picked up different timezone settings.

Resolution: Verify the Dell's current timezone with `timedatectl`. Restart all containers with `'docker restart loki alloy grafana dvwa caddy-juice juice-shop'`. Metasploitable is a VM, not a container, and does not need a Docker restart for timezone changes.

Grafana Time Range Does Not Show Expected Activity

Querying a time range known to contain attack activity returns no results.

i Note

Root cause: The time range selector in Grafana defaults to relative ranges (Last 1 hour, Last 5 minutes). If the attack occurred earlier than the selected window, it will not appear.

Resolution: Use the absolute time range option in Grafana's time picker. Click the time range selector, choose 'Absolute time range', and enter the exact start and end times that cover the attack session. This ensures the query covers the correct period regardless of when it is run.

PART 12: OPERATIONS AND MAINTENANCE

25. Daily Operation Runbook

This section documents the standard procedures for starting, operating, verifying, and stopping the homelab safely. Following this runbook ensures that services are started in the correct order, that the monitoring pipeline is confirmed working before attack traffic is generated, and that vulnerable targets are shut down cleanly after each session.

25.1 Start Core Stack

Core services (Loki, Alloy, Grafana) are configured with `--restart unless-stopped` and will start automatically when Docker starts after a reboot. After a manual start or on first setup, bring them up in this order:

```
# Start Loki first (Alloy needs it to be ready)
docker start loki
```

```
# Wait 10 seconds, then start Alloy
docker start alloy
```

```
# Start Grafana
docker start grafana
```

```
# Verify all three are running
docker ps
```

```
# Confirm Loki is ready
curl http://localhost:3100/ready
```

i Note

Starting Loki before Alloy matters. If Alloy starts before Loki is ready, it will attempt to connect to `loki:3100`, fail, and may back off with a delay before retrying. Starting Loki first avoids this. The `--restart unless-stopped` policy handles this automatically on boot by giving Docker time to bring up Loki before Alloy tries to connect.

25.2 Start Vulnerable Targets

Vulnerable targets are started manually, session by session. Do not start them unless actively using the lab.

```
# DVWA (start only when testing DVWA)
docker start dvwa
```

```
# Juice Shop and Caddy (start both together)
```

```
docker start juice-shop
docker start caddy-juice
```

```
# Metasploitable VM (start only when testing service-level attacks)
VBoxManage startvm metasploitable2 --type headless
```

```
# Verify Metasploitable is booting (wait 60 seconds before pinging)
VBoxManage list runningvms
ping -c 3 <YOUR-VM-IP>
```

Important

Never start Juice Shop without also starting Caddy. Juice Shop started without Caddy is only accessible on port 3001 (diagnostic port). No access logs will be generated and no detection will work. Always start both containers together for any Juice Shop testing session.

25.3 Verify Dashboards Before Testing

Before generating any attack traffic, open the Homelab Monitoring Dashboard in Grafana and confirm that all pipeline panels are showing data:

- Syslog Activity panel shows a non-flat line (background system activity is flowing)
- Auth Log Activity panel shows some activity
- Docker Activity panel shows non-zero counts
- All Logs Unified View shows entries from multiple sources

Tip

If any panel is dark or flat before starting a test session, resolve the pipeline issue first. Attack traffic that never reaches Loki is lost and cannot be investigated afterward. A two-minute pipeline check before each session prevents this from happening.

25.4 Stop Lab Safely

Stop vulnerable targets immediately when a session ends. Stop core services only when the Dell server needs to be powered down or restarted.

```
# Stop vulnerable targets after each session
docker stop dvwa
docker stop caddy-juice juice-shop
VBoxManage controlvm metasploitable2 poweroff
```

```
# Stop core stack only when shutting down the server
docker stop grafana alloy loki
```

25.5 After Reboot Checklist

After the Dell server reboots, run through the following checklist before starting any lab session:

- Verify Docker service is running: `sudo systemctl status docker`
- Check that core containers started automatically: `docker ps` (should show loki, alloy, grafana)
- If any core container is missing, start it manually and check docker logs for the reason it did not start
- Confirm Loki is ready: `curl http://localhost:3100/ready` (should return 'ready')
- Open Grafana in browser and verify the Loki data source connection is still working
- Check the Homelab Monitoring Dashboard for all panels showing data
- Verify Tailscale is connected: `tailscale status`
- Check UFW rules are still in place: `sudo ufw status`
- Start vulnerable targets only when needed for the session

i Note

Core services (Loki, Alloy, Grafana) have `--restart unless-stopped` policies and will start automatically after a reboot. Vulnerable targets (DVWA, Juice Shop, Caddy, Metasploitable) do not auto-start. This is intentional. Always confirm the monitoring pipeline is healthy before starting vulnerable targets.

25.6 Troubleshooting: Operations

Core Container Does Not Start After Reboot

One of the core containers (Loki, Alloy, Grafana) does not appear in 'docker ps' after a reboot even though `--restart unless-stopped` was set.

⚠ Important

Root cause: The container may have been stopped with 'docker stop' before the reboot, which sets it to the 'stopped' state. The unless-stopped policy does not restart containers that were manually stopped before the shutdown.

Resolution: Start the container manually with 'docker start <name>'. Check 'docker logs <name>' for any startup errors. If the container exits immediately after starting, the logs will contain the reason.

Grafana Shows No Data After Reboot

All Grafana panels return no data even though Loki and Alloy appear to be running.

⚠ Important

Root cause: Alloy may have started before Loki was ready and failed to establish a connection. Alloy will retry, but there may be a delay.

Resolution: Run 'docker logs alloy --tail 30' and look for connection errors or 'ready' confirmations. Run 'curl http://localhost:3100/ready' to confirm Loki is up. If Loki is ready and Alloy still shows errors, restart Alloy with 'docker restart alloy'.

26. Log Rotation and Storage Management

26.1 Why Log Rotation Matters

Two log files in this lab are written to indefinitely without automatic cleanup: the Caddy JSON access log for Juice Shop and the Metasploitable rsyslog log file. Unlike the Docker container logs and the Dell host's syslog (which are managed by their respective systems), these two files will grow without bound until disk space is exhausted on the Dell's 1 TB HDD.

Loki's retention policy (168 hours, 7 days) handles the logs once they are inside Loki. But it does not affect the source files on disk. A lab session that generates heavy Juice Shop or Metasploitable traffic can produce megabytes of log data per hour. Over weeks of use, this accumulates significantly.

26.2 Files Needing Rotation

File	Written By	Risk Without Rotation
~/caddy/juice-shop/logs/juice-shop-access.log	Caddy container	Grows with every Juice Shop request, no upper bound
/var/log/metasploitable/metasploitable.log	rsyslog on Dell host	Grows with every Metasploitable syslog event, no upper bound

26.3 Suggested logrotate Plan

The standard Linux log rotation tool is logrotate. The following configuration handles both files. Create a new logrotate configuration file:

```
sudo nano /etc/logrotate.d/homelab
```

Paste the following configuration:

```
/home/<YOUR_USER>/caddy/juice-shop/logs/juice-shop-access.log
/var/log/metasploitable/metasploitable.log
{
    daily
    rotate 7
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
```

Each directive explained:

- daily - rotate the log once per day
- rotate 7 - keep 7 rotated copies before deleting the oldest. Combined with daily, this keeps one week of history on disk.
- compress - compress rotated log files with gzip to reduce disk usage
- delaycompress - delay compression of the most recently rotated file by one cycle. This prevents compressing a file that a process may still have open.
- missingok - do not report an error if the log file does not exist (useful if Metasploitable has not been used recently and the file is empty or missing)
- notifempty - do not rotate empty log files
- copytruncate - copy the current log file to the rotated name and then truncate the original to zero bytes. This is important for the Caddy log because Caddy keeps the file handle open. Truncating rather than renaming means Caddy continues writing to the same path without needing a restart.

⚠ Important

Replace <YOUR_USER> in the Caddy log path with the actual username on the Dell server. The logrotate configuration must use the full absolute path to the file. A relative path will not work.

Test the logrotate configuration without actually rotating:

```
sudo logrotate --debug /etc/logrotate.d/homelab
```

The debug output will show which files would be rotated and confirm the configuration is being parsed correctly.

27. Backup and Persistence

27.1 What Must Be Backed Up

The following items represent the lab's configuration state. Losing them would require significant reconstruction effort. They should be backed up to the Acer workstation or another location outside the Dell.

Item	Location on Dell	How to Back Up
Loki configuration	~/logging/loki-config.yml	Copy to Acer via scp or Tailscale file share
Alloy configuration	~/logging/alloy/config.alloy	Copy to Acer via scp or Tailscale file share
Caddyfile	~/caddy/juice-shop/config/Caddyfile	Copy to Acer via scp
rsyslog Metasploitable config	/etc/rsyslog.d/30-metasploitable.conf	Copy to Acer via scp
SSH config for Metasploitable	~/.ssh/config	Copy to Acer via scp
Grafana dashboards	Inside Grafana container or ~/grafana-data volume	Export as JSON from Grafana UI
UFW rules	System-managed	Save output of 'sudo ufw status verbose'

27.2 Grafana Dashboard Export

Grafana dashboards are the most time-consuming items to recreate from scratch. The LogQL queries, panel layouts, visualisation settings, and thresholds all require manual re-entry if lost. Export them regularly.

To export each dashboard:

- Open the dashboard in Grafana
- Click the settings gear icon in the top-right corner of the dashboard
- Select JSON Model from the left sidebar
- Copy the entire JSON content
- Save it to a file named after the dashboard on the Acer workstation

Recommended export names:

```
homelab-monitoring-dashboard.json
dvwa-detection-dashboard.json
juiceshop-detection-dashboard.json
metasploitable-detection-dashboard.json
```

To import a dashboard after a container rebuild: in Grafana, go to Dashboards in the left sidebar, click New, then Import, paste the JSON content or upload the JSON file, and click Load.

27.3 Container Persistence

The following table summarises the persistence state of each container and what happens when it is removed and recreated:

Container	Persistence	What Is Lost on Removal
Loki	Volume mounted to <code>~/logging/loki</code>	Nothing, log data is on the host filesystem
Alloy	Config file mounted from host	Nothing, Alloy has no state of its own
Grafana	Volume mounted to <code>~/grafana-data</code> (if set up)	Nothing if volume is mounted; all dashboards if not
DVWA	No persistent volume	Database state resets; must click Create/Reset Database again
Juice Shop	No persistent volume	Application state resets; progress on challenges is lost
Caddy	Config and log directory mounted from host	Nothing, config and logs are on the host filesystem

Note

Grafana is the only core service where data loss on container removal is a significant risk. The `~/grafana-data` volume mount described in Section 9.9 protects against this. If that volume mount is in place, removing and recreating the Grafana container preserves all dashboards, data source connections, and settings.

27.4 Troubleshooting: Backup and Persistence

Grafana Dashboards Lost After Container Rebuild

After removing and recreating the Grafana container, all dashboards are gone and only the default Grafana welcome screen appears.

Important

Root cause: The Grafana container was recreated without the `~/grafana-data` volume mount, or the volume mount directory has incorrect ownership.

Resolution: Stop and remove the Grafana container. Verify `~/grafana-data` exists and is owned by UID 472 (run `'ls -la ~/ | grep grafana-data'`). If ownership is wrong, run `'sudo chown -R 472:472 ~/grafana-data'`. Recreate Grafana with the `-v ~/grafana-data:/var/lib/grafana` mount. If the directory was lost entirely, restore from the JSON exports and reimport each dashboard.

PART 13: SECURITY, SCOPE, AND LIMITATIONS

28. Lab Security Rules

A homelab that hosts intentionally vulnerable software carries real responsibility. The following principles govern how this lab is operated safely. They should be understood by anyone working with a similar setup.

28.1 Keep Vulnerable Targets Private

DVWA, Juice Shop, Metasploitable 2, and all other vulnerable applications in this lab are hosted exclusively on the local network or on isolated network segments. No port forwarding is configured on the home router. No DDNS or public domain name points to this server. No VPN exit node exposes these services externally.

Important

If any of these targets were reachable from the public internet, they could be exploited by anyone who found them, not just the intended user. Before adding any form of external access to this homelab in the future, every service must be reviewed for whether it is safe to expose. Vulnerable targets are never exposed externally under any circumstances.

28.2 Start Vulnerable Targets Only When Needed

DVWA, Juice Shop, Caddy, and Metasploitable are not configured to start automatically on boot. They must be started manually at the beginning of each practice session and stopped with the commands in Section 25.4 when the session ends. This minimises the window of exposure and conserves system resources.

28.3 Lab Boundary

All attacks in this lab are performed against services running on the Dell server or the Metasploitable VM on the host-only network. Never perform attack tool scans or exploitation attempts against any system outside the lab network. This is both a legal boundary and an ethical one. Port scanning or exploiting systems you do not own is illegal in most jurisdictions regardless of intent.

Important

The lab boundary is: the Dell server, Docker containers running on the Dell, and the Metasploitable VM on the VirtualBox host-only network. Everything outside this boundary, including the home router, other devices on the home network, and anything on the internet, must not be targeted.

28.4 Tailscale Access Scope

Tailscale provides encrypted remote access to the Dell server from the Acer workstation across any network. Only services that are legitimately exposed for administrative access should be reachable

via Tailscale. Grafana (port 3000) and SSH (port 22) are the intended Tailscale-accessible services. Vulnerable targets (DVWA on 8080, Juice Shop via Caddy on 3002) are accessible via Tailscale because they are on the same machine, but they should only be accessed when deliberately conducting a lab session.

28.5 Keep the Host System Updated

Run 'sudo apt update && sudo apt upgrade -y' regularly, at minimum weekly. The host OS is the foundation of everything running on the server. An unpatched kernel or vulnerable system library can undermine all other security controls. Docker images should also be pulled fresh periodically to incorporate upstream security fixes.

28.6 Troubleshooting and Safety Notes

UFW Rules Missing After Reboot

After a reboot, 'sudo ufw status' shows that rules have reverted or UFW appears disabled.

i Note

Root cause: UFW may not have been enabled persistently, or the service did not start correctly after the reboot.

Resolution: Run 'sudo ufw enable' to ensure it is active, then re-add any rules that are missing. Check 'sudo systemctl status ufw' to confirm the service is running. Verify rules with 'sudo ufw status numbered'.

29. Detection Limitations

Every detection system has limitations. Understanding what this lab's detection can and cannot do is as important as understanding what it can do. These limitations are not failures of the design; they are inherent characteristics of the chosen approach and the hardware constraints. Recognising them is part of developing mature security judgement.

29.1 DVWA Limitations

- Detection relies on Apache access logs captured from Docker container stdout. POST body content for some vulnerabilities (command injection via form submission) may not appear in full in the access log, only the URL path.
- Attack Score is based on DVWA-specific URL path terms and would not generalise to a real application.
- SQLi pattern detection produces false positives from Apache internal log messages containing SQL-adjacent keywords. These must be distinguished from genuine attack traffic using the evidence panels.
- All detection is signature-based. Novel attack variants that do not match the regex patterns will not be detected.

29.2 Juice Shop Limitations

- Caddy access logs capture request metadata (method, URI, status, source IP, user agent) but do not capture JSON request bodies. Attack payloads submitted inside POST bodies (such as credentials or API request parameters) are not visible as payload strings in Loki.
- Detection depends on Caddy running on port 3002. Traffic sent to port 3001 (direct Juice Shop access) bypasses Caddy and produces no detection logs.
- The source IP visible in Caddy logs may reflect the Docker bridge gateway rather than the true client IP in some network configurations. This is a Docker networking characteristic.
- All detection is signature-based against URL-encoded payload patterns. Payloads submitted in ways that do not produce recognisable URL-encoded strings may not be detected.

29.3 Metasploitable Limitations

- Detection is based on syslog messages generated by services on Metasploitable. Not all service activity produces syslog output. Some attacks against Metasploitable may leave no log trace at all.
- Nmap scans produce visible log entries only when scanned services log connection attempts. A scan targeting ports with no active service or with silent rejection will not produce syslog entries.
- This dashboard provides service-log visibility, not full packet-level network detection. There is no visibility into network-layer events (TCP SYN floods, ARP scanning) that do not result in a service logging a message.
- The rsyslog pipeline introduces a small forwarding delay. Log entries may appear in Grafana a few seconds after the actual event, which is not a problem for investigation but means live streaming is not millisecond-accurate.

29.4 Platform Limitations

- Loki does not index log line content, only labels. This makes some query patterns that are trivial in Elasticsearch require more specific LogQL expressions in Loki. Queries cannot perform arbitrary full-text search across all labels simultaneously without specifying a stream selector first.
- On the Dell's HDD hardware, complex queries over wide time ranges can produce 'too many outstanding requests' errors. Wide time range investigation must be performed with care.
- Grafana dashboard persistence depends on the ~/grafana-data volume mount being in place. Without it, a container rebuild loses all dashboards.
- Log rotation for Caddy and Metasploitable log files must be implemented manually via logrotate. Without it, log files grow without bound.

29.5 What Would Improve Detection Later

- Application-level request body logging in Juice Shop: would expose POST payload content currently invisible in Caddy access logs

- Fail2Ban integration: would automatically block repeated failed authentication attempts at the host firewall level, adding an active response layer to the passive detection currently in place
- Grafana alerting rules: would send notifications when detection panels exceed defined thresholds, removing the need for manual dashboard monitoring
- Suricata or Zeek: would provide packet-level network detection for Metasploitable that the current syslog-based approach cannot deliver
- Tailscale subnet routing to Metasploitable: would allow Kali VM attacks from the Acer to reach Metasploitable over the host-only network, enabling more realistic attack simulation

PART 14: CURRENT STATE AND FUTURE ROADMAP

30. Current Completed State

30.1 Completed Components

The following components are fully deployed and confirmed operational as of this document version:

- Ubuntu Server 22.04.5 LTS: base OS, static IP, SSH, correct timezone configured
- Docker: container platform with user added to docker group, Docker service enabled on boot
- Tailscale: both Dell and Acer enrolled in the same tailnet, secure remote access confirmed
- UFW: default-deny inbound, SSH allowed, service-specific allow rules, vboxnet0 restricted rule for rsyslog
- Docker network labnet: shared user-defined network, all relevant containers connected
- Grafana Loki: running with retention policy, volume-mounted data directory, ready endpoint confirmed
- Grafana Alloy: collecting syslog, auth.log, Docker container logs, Caddy access logs, and Metasploitable rsyslog file
- Grafana: connected to Loki data source, volume-mounted data directory, all dashboards built
- DVWA: deployed on port 8080, database initialised, security level set to Low
- Homelab Monitoring Dashboard: four panels confirming pipeline health across all three base log streams
- DVWA Detection Dashboard: five rows covering brute force, SQL injection, XSS, and behaviour panels
- OWASP Juice Shop: deployed on labnet, accessible on port 3001 for diagnostics
- Caddy: reverse proxy for Juice Shop, JSON access log format confirmed, running on port 3002
- Juice Shop JSON Log Pipeline: Alloy tailing Caddy logs, {job="juice-shop-access"} confirmed in Grafana
- Juice Shop Detection Dashboard: five rows covering login abuse, injection, API abuse, and investigation
- VirtualBox: installed on Dell, host-only network configured with vboxnet0 at <YOUR-HOST-ONLY-ADAPTER-IP>
- Metasploitable 2: VM deployed, SSH access confirmed with legacy algorithm fix, syslog forwarding configured
- rsyslog receiver: 30-metasploitable.conf active, binding to <YOUR-HOST-ONLY-ADAPTER-IP> on UDP 514
- Metasploitable Log Pipeline: syslogd forward confirmed, rsyslog file write confirmed, Alloy tail confirmed, {job="metasploitable-syslog"} confirmed in Grafana
- Metasploitable Detection Dashboard: five rows covering SSH/auth, FTP/Telnet, reconnaissance, and investigation

30.2 Final Service Table

Component	Status	Port/Address	Log Source	Dashboard	Start Mode
Ubuntu Server 22.04.5	Active	Static LAN IP	Host OS	Monitoring	Always on
Docker (labnet)	Active	-	-	-	Auto on boot
Grafana Loki	Active	3100	-	-	Auto (restart unless-stopped)
Grafana Alloy	Active	127.0.0.1:12345	-	-	Auto (restart unless-stopped)
Grafana	Active	3000	-	All dashboards	Auto (restart unless-stopped)
rsyslog	Active	UDP 514 on vboxnet0	-	-	Auto (systemd)
DVWA	Manual	8080	{job="docker",container="dvwa"}	DVWA Detection	Manual per session
Juice Shop	Manual	3001 (diag only)	{job="docker"}	Juice Shop Detection	Manual per session
Caddy (Juice Shop)	Manual	3002	{job="juice-shop-access"}	Juice Shop Detection	Manual per session
VirtualBox	Installed	Host-only vboxnet0	-	-	Manual VM start
Metasploitable 2	Manual	<YOUR-VM-IP>	{job="metasploitable-syslog"}	Metasploitable Det.	Manual per session
Tailscale	Active	100.x.x.x mesh	-	-	Auto on boot
UFW	Active	Default deny inbound	-	-	Auto on boot

31. Future Work and Roadmap

The following improvements and additions are planned for future phases of this homelab. They are organised by category and priority. None of the items in this section are currently installed or active.

31.1 Short-Term Improvements

These items directly improve the stability and maintainability of the current lab and should be implemented in the next maintenance window:

- Grafana dashboard JSON export: export all four dashboards as JSON files and store them on the Acer workstation. This protects against data loss if the Grafana container is rebuilt without a volume mount.
- Grafana persistent volume confirmation: verify that `~/grafana-data` is correctly mounted and owned by UID 472. Run a test by removing and recreating the Grafana container and confirming dashboards survive the rebuild.
- logrotate for Caddy and Metasploitable logs: implement the configuration from Section 26 and confirm it rotates correctly. Schedule a weekly manual check of both log file sizes until this is active.
- Restart policy cleanup: audit all containers and confirm that core services have `--restart unless-stopped` and vulnerable targets do not. Document the final policy in the operations runbook.

31.2 Security and Monitoring Improvements

These items add active defence and alerting capability to the currently passive detection setup:

- Fail2Ban: install and configure Fail2Ban on the Dell host to automatically block IP addresses that exceed authentication failure thresholds against SSH. This adds an active response layer to the existing passive detection.
- Grafana alerting: configure alert rules on key detection panels (Failed Login Attempts, SQLi Attempts Over Time, Suspicious Activity Score) with threshold-based triggers. Test alert notifications to confirm they fire correctly.
- Better alert thresholds: review and tune the thresholds on all stat panels (Attack Score, Suspicious Activity Score, Total Events) based on observed baseline activity during real lab sessions.

31.3 Home Server Improvements

These items expand the Dell server's role as a practical home server alongside its security lab function:

- Pi-hole: network-wide DNS filtering and ad blocking for all devices on the home network. Deploy as a Docker container on labnet, or on the host network depending on DNS configuration requirements.
- Samba: local file sharing to make a shared directory accessible from the Acer workstation and other home network devices. Useful for transferring files between machines without needing SCP or Tailscale.

- Jellyfin: local media server for streaming video and audio. Direct play only (no transcoding) given the Dell's CPU limitations. Deploy as a Docker container with the media directory mounted from the host.

31.4 Advanced Security Improvements

These items add more sophisticated detection and attack simulation capability to the lab:

- Optional packet-level IDS: Suricata or Zeek would provide network-layer detection for the Metasploitable target that the current syslog-based approach cannot deliver. This would require either a dedicated network tap or running the IDS on the Dell host with access to the vboxnet0 interface.
- Optional Tailscale subnet routing to Metasploitable: configure Dell IP forwarding between the tailscale0 and vboxnet0 interfaces, and advertise the <YOUR-HOST-ONLY-NETWORK> subnet as a Tailscale route. This would allow the Kali VM on the Acer to reach Metasploitable directly over Tailscale, enabling more realistic remote attack simulation.
- Optional Windows target: if hardware allows (unlikely on current RAM allocation), a minimal Windows Server VM could be added to the VirtualBox host-only network to provide Active Directory attack practice. This would require significant additional RAM and is a long-term consideration only.
- Optional structured portfolio screenshots: capture and annotate Grafana dashboard screenshots showing real attack-to-detection correlation evidence. This produces concrete portfolio artefacts demonstrating the lab's detection capability to reviewers.

31.5 Items Not Currently Planned

The following items were considered during the design phase and are not part of the current roadmap. They are documented here to avoid confusion if they appear in earlier draft documents:

- Proxmox: not installed, not planned. VirtualBox is sufficient for the current virtualisation needs.
- Full ELK stack (Elasticsearch, Logstash, Kibana): not installed, not planned. Hardware constraints make this infeasible on the Dell N4030. Loki provides sufficient capability for this lab's learning objectives.
- Wazuh: not installed, not planned. Wazuh was evaluated and rejected due to resource requirements.
- Splunk: not installed, not planned.
- Unbound DNS resolver: was mentioned in early planning notes as a complement to Pi-hole. Not part of the current plan. If Pi-hole is added in a future phase, whether to include Unbound will be evaluated at that time.